

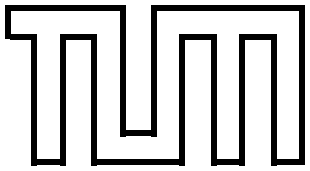
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Voblint: Towards a Verified Goblint-Style Analysis Pipeline in Isabelle/HOL

Manuel Lerchner



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Voblint: Towards a Verified Goblint-Style Analysis Pipeline in Isabelle/HOL

Voblint: Auf dem Weg zu einer verifizierten Analysepipeline im Stil von Goblint in Isabelle/HOL

Author:	Manuel Lerchner
Supervisors:	Alexandra Graß, M.Sc. Prof. Dr. Helmut Seidl
Examiner:	Prof. Dr. Francisco Javier Esparza Estaun
Submission Date:	01.12.2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 01.12.2026 Manuel Lerchner

Acknowledgements

I would like to thank my supervisors, Alexandra Graß and Helmut Seidl, for their guidance and support throughout this thesis. I am especially grateful to Alexandra for the many discussions, detailed feedback, and ideas that shaped the project, and I greatly appreciated the freedom to explore the topic in depth and to follow directions that emerged along the way. Javier Esparza kindly agreed to examine this thesis.

I am also grateful to Michael Petter for helping to scope the project and shape its initial direction, and to Julian Erhard for ideas and feedback that brought the formalization closer to Goblint. I also thank Sarah Tilscher for developing and maintaining the verified TD solver formalization on which this work builds.

I also acknowledge the many contributors to Goblint, whose work on the analyzer provided both a practical reference point and much of the inspiration for this thesis, as well as the developers of Isabelle and the open-source tooling around it. In particular, the Isabelle interfaces offered by AutoCorrode and Kevin Kappelmann's MCP server made it practical to integrate AI-assisted development into a formalization of this scale.

I am also grateful to the members of the Chair for Programming Languages, Compiler Construction and Specification Formalisms. I first came to know the chair through the course Functional Programming and Verification, and it became an important part of my time at TUM. Working there twice as a teaching assistant and once as a tutor gave me insight into both teaching and research, and showed me how much I enjoy teaching formal methods.

Finally, I would like to thank my family and friends. My family gave me the freedom to focus fully on my work and encouraged me through every stage of my studies. My friends provided encouragement, welcome distractions, and regular reminders that there is a world outside Isabelle, and their company made the more intense periods of work much easier. Their patience, friendship, and support have meant a great deal to me throughout my time at TUM.

Abstract

An abstract interpreter over-approximates a program's executions, so any property that holds for the over-approximation holds in every run. Goblint, an abstract interpreter for C, builds a control-flow graph of the program, translates the graph into a system of equations over abstract values, and solves that system with a generic fixpoint solver [1], [2]. Its guarantee therefore depends on four conditions: a control-flow graph that simulates the source program, equations that over-approximate its executions in every calling context, a solver result that is a post-solution of the equations, and verdicts that follow from that result. For Goblint's top-down solver, only the third condition had been verified [3], [4].

We present Voblint, an Isabelle/HOL formalization of a context-sensitive interprocedural analyzer. Its soundness proof covers all four conditions for VIMP, a small C-like language with global and local integer variables and recursive procedures with parameters and return values. The main theorem combines the proofs of the four conditions. It states that if the analyzer returns a report, the report covers every store that a finite execution of the program reaches from an initial store with zeroed globals, and that every definite verdict at the corresponding program point holds for that store. We also prove that a check reported `DEAD` is unreachable and that a divisor reported as an error is zero wherever its point is reached. At a point without an arithmetic warning or error, no divisor is zero. To our knowledge, Voblint is the first mechanized analyzer whose soundness proof connects a source semantics to a side-effecting constraint system and its verified solver.

The main difficulty is context sensitivity. A context-sensitive analyzer computes a separate invariant for each program point and calling context. The collecting semantics, however, merges the stores of all calls to a procedure, so it cannot say which stores a given context must cover. To solve this, we introduce activation traces, inspired by the local traces of Schwarz et al. [5]. Each trace follows one procedure activation, and a context policy decides to which contexts it belongs. If the policy admits every reachable call in some context, the union of the per-context sets of stores equals the collecting semantics, so splitting by context loses no execution.

The proof is modular and follows Goblint's architecture. Domains, context policies, analyses, and the solver each have their own proof obligations, and one soundness theorem combines them. A numeric domain only has to implement a shared interface and prove its abstract operations sound. The framework builds transfer functions from these operations and proves them sound once for all domains, as in Nipkow and Klein's generic abstract interpreter [6, §13.5]. The main theorem holds for every offered choice of analysis, context policy, and placement of program globals.

The analysis function is exported to OCaml, runs in a command-line interface and a browser playground, and is tested on 377 regression programs. Proofs by evaluation show that the main theorem is non-vacuous on concrete programs.

The main result is partial correctness: termination of the solver is not proved. We trust everything outside the formalization: the hand-written OCaml code, including the parser and the rendering, Isabelle's code generator, and the OCaml and WebAssembly toolchains. We document how VIMP differs from C11 [7].

Use of Generative AI

The Leiden Declaration on Artificial Intelligence and Mathematics recommends that authors transparently disclose the automated tools they use, including large language models [8]. This section follows that recommendation.

Substantial parts of this work were carried out with generative AI tools, primarily Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor, under the author's direction and review. These tools contributed a large share of the implementation-level material. The author chose the research direction, selected and revised the designs, and approved the definitions and theorem statements.

In the Isabelle development, the tools generated many proofs, carried out refactorings across theories, and transformed tactic-style proofs and Sledgehammer output into structured Isar. They proposed designs and drafted planning documents for larger changes. For the thesis, they assisted with literature searches, checked citations and claims against cited sources, and drafted and revised the chapter structure, text, and figures.

Separately, Grammarly was used to check spelling and grammar in the thesis text. The Isabelle interfaces of AutoCorrode [9], used following the human-guided workflow of Kappelmann et al. [10], served as proof-development infrastructure.

The assistance also extended to design. To formalize calling contexts, the concrete semantics is based on traces instead of states, inspired by the local traces of Schwarz et al. [5]. The state-based collecting semantics of the prototype records the stores that reach a node, but not which procedure activation contains each store (Section 4.1). Language models helped adapt the thread-local setting of that work to procedure activations. The requirements for the activation traces of Section 4.2.1 were refined interactively with them: how traces are generated, that a callee trace begins only at a call, that a trace retains its call history, and that calling contexts are derived from traces.

For the analysis framework, agents inspected Goblint's OCaml sources to identify interfaces the formalization could approximate, such as local and global unknowns, unknowns indexed by node and context, and the enter/combine protocol at calls (Section 15.5). The source language and its compilation, the activation traces, the coverage contract, and the corresponding proofs have no direct counterpart in Goblint and were developed for this work. The formalization reuses the Isabelle/HOL formalization of Goblint's top-down solver and its partial-correctness proof by Tilscher et al. [4].

A major part of the development effort concerned finding suitable definitions, interfaces, invariants, and locale boundaries rather than discharging individual proof steps. During the first weeks, a small prototype mirrored Goblint's structure while leaving proof obligations as `sorry`. Once the definitions and locale boundaries had stabilized, many of the remaining proof obligations were short and were discharged with substantial agent assistance.

The assistants' output was not reliable on its own. Agents proposed incorrect lemmas and, in one case, introduced a premise into the end-to-end theorem that no program satisfied, making the theorem vacuous until the solver run was redesigned. Audits of theorem statements and full batch builds exposed these errors. The non-vacuity instances of Section 13.1 now demonstrate that the premises of the main theorems can be satisfied.

Agents also wrote much of the tooling around the formalization, such as the command-line interface, the website, and the regression suite with its ports of Goblint regression tests. They also wrote the checks that keep the theories, the generated material, the analyzer,

the website, and this thesis consistent (Section 14.1). These checks proved especially useful because they replaced repeated manual inspection with conditions that every change had to satisfy, and they exposed errors that manual reading had missed.

The table below quantifies the recorded part of this assistance, following the session-log analysis of Bryant et al. [11, Section 5]. The counts measure interaction with the tools, not authorship or the proportion of the work attributable to them.

	Claude Code		**Codex**	
	main	subagent	main	subagent
sessions	110	451	121	47
assistant messages	52,374	22,755	15,063	484
tool calls	56,474	27,952	25,235	2,402
shell	27,525	17,568	–	–
Isabelle batch builds	726	97	361	1
Isabelle server (I/Q, PIDE)	24,149	5,032	18,251	1,247
file edits and writes	850	1,340	600	22

Recorded automated-assistant activity during development, counted on 2026-09-25 from the local session logs (2026-08-24 to 2026-09-25 for Claude Code, 2026-06-27 to 2026-09-21 for Codex). No local logs are available for earlier Claude Code sessions, ChatGPT in the browser, or Cursor, so these are not counted. The author sent 2,434 messages to Claude Code and 1,227 to Codex. Shell edits count as shell calls. Interactions through the Isabelle server, including theory edits, count as server calls. Co-author lines in commit messages were not kept consistently and are not used as a measure.

Isabelle/HOL accepts every theorem in the development. Proofs by evaluation, used for some facts about fixed programs, additionally rely on Isabelle's code generator (Section 12.5). This establishes the stated propositions relative to their definitions and assumptions, but does not establish that those definitions faithfully capture the intended language or analysis. Definitions and theorem statements were therefore also checked against example programs, the regression suite, and the documented differences between VIMP and C11. Related-work claims were checked against the cited sources, often with agent assistance and subsequent author review.

Responsibility for the final claims, framing, and treatment of related work lies with the author alone.

Contents

Acknowledgements i

Abstract ii

Use of Generative AI iii

Part I: The Problem

1. Introduction 2

1.1. From executions to static guarantees . . . 2
1.2. State of the art . . . 4
1.3. Contributions . . . 7
1.4. Scope and limitations . . . 8
1.5. Outline . . . 9

2. Background 10

2.1. Executions and the collecting semantics . . . 10
2.2. Lattices and fixpoints . . . 11
2.3. Abstract interpretation . . . 12
2.4. Constraint systems . . . 14
2.5. Widening and narrowing . . . 15
2.6. Side-effecting constraint systems . . . 16
2.7. The verified top-down solver . . . 17
2.8. Isabelle/HOL mechanisms used in this development . . . 18

Part II: What Must Be Over-Approximated

3. Programs and Control-Flow Graphs 22

3.1. VIMP, and what it leaves out . . . 22
3.2. Executing a source program . . . 25
3.3. Scope and adequacy of the source semantics . . . 27
3.4. The procedure-aware control-flow graph . . . 28
3.5. Compilation . . . 31
3.6. Relating the two executions . . . 33

4. Traces and the Coverage Contract 35

4.1. Why contexts need traces . . . 35
4.2. Traces . . . 37
4.3. Calling contexts . . . 40
4.4. The coverage contract . . . 43

Part III: Analyses

5. Abstract Domains **46**

5.1. What an abstract value means . . . 47
5.2. From values to stores . . . 48
5.3. Evaluating expressions . . . 51
5.4. Answering queries . . . 52
5.5. Learning from a guard . . . 53
5.6. The complete domain interface . . . 54
5.7. A carrier the interface excludes . . . 56

6. What an Analysis Supplies **57**

6.1. Transfers at edges and calls . . . 57
6.2. Facts without a program point . . . 58
6.3. The analysis contract . . . 59
6.4. Program globals: flow-sensitive or flow-insensitive . . . 60

7. How Analyses Cooperate **61**

7.1. Questions and answers . . . 62
7.2. Proving an analysis against any channel . . . 63
7.3. Many analyses over one state . . . 64
7.4. What a new analysis has to prove . . . 65

Part IV: From Analyses to Verdicts

8. Equations, Contexts, and Routing **67**

8.1. Unknowns and right-hand sides . . . 67
8.2. Routing and context policies . . . 69
8.3. Soundness of the generated system . . . 71

9. Solving the Equations **72**

9.1. Adapting the equations to the solver . . . 72
9.2. A demand-driven solve . . . 75
9.3. What a terminating solve guarantees . . . 76
9.4. Merging contributions to global unknowns . . . 78
9.5. Making abstract states executable . . . 79
9.6. Why termination is not proved . . . 81

10. What a Report Guarantees **82**

10.1. From solution to report . . . 82
10.2. Verdicts . . . 83
10.3. The chain at one check . . . 85
10.4. The source-level theorem . . . 86
10.5. What a verdict says about executions . . . 87

Part V: Instances and Practice

11. Five Domains and an Order Analysis **90**

11.1. The implemented domains . . . 90
11.2. One loop, five answers . . . 91
11.3. The reduced product . . . 91
11.4. The order analysis . . . 93

12. From Theorem to Tool **94**

12.1. From theorem to code . . . 94
12.2. The generated code . . . 95
12.3. The unverified shell . . . 96
12.4. Tracing the exported solver . . . 97
12.5. The trust boundary . . . 97

13. Evaluation **99**

13.1. Non-vacuity . . . 99
13.2. Do the obligations exclude real errors? . . . 100
13.3. Precision and cost on concrete programs . . . 100
13.4. Tests of the unverified parts . . . 103

14. Engineering and Tooling **104**

14.1. The Voblint repository . . . 104
14.2. `isar-tools`: source tooling for Isabelle . . . 106
14.3. Changes to the verified solver . . . 107

Part VI: Assessment

15. Related Work **109**

15.1. Verified analyzers: Verasco and the CompCert line . . . 109
15.2. Mechanized abstract interpretation . . . 110
15.3. Verified fixpoint solvers . . . 111
15.4. Context sensitivity and trace partitioning . . . 112
15.5. Goblint, local traces, and thread-modular analysis . . . 112
15.6. Where Voblint sits . . . 114

16. Conclusion **115**

16.1. Discussion . . . 115
16.2. Limitations . . . 116
16.3. Outlook . . . 117

Bibliography **119**

PART I

The Problem

1. Introduction

A static analyzer answers questions about every execution of a program at once: whether a divisor can be zero at some point, whether a program point is reachable at all, or whether a check always holds. The analyzer is itself a program and can contain errors. Such an error can produce a confident answer that real executions contradict. In Goblint issue 1156, Goblint claimed that `c % 2 == 1` holds for $c \in \{-5, -7\}$ [12]. However, the truncating remainder of C11 [7, §6.5.5p6] gives -1 for both values. Pull request 1161 fixed the congruence domain, which had ignored the sign of the dividend [13]. Such bugs are particularly bad because a user who trusts the answer has no reason to look for the error. An imprecise answer, in contrast, only leaves the check undecided. In this thesis, we build Voblint, an executable analyzer whose answers are proved sound for every execution of the analyzed program, and we state exactly under which assumptions this proof holds.

Formal verification states such a claim as a proposition over explicit definitions, and a proof assistant checks its proof [6]. Isabelle/HOL constructs every theorem through a small trusted inference kernel, following the LCF approach [14], so the kernel accepts a proof regardless of who or what wrote it. This matters now that LLM coding agents produce large formalizations. Bryant et al. [11] report agents that wrote over 85,000 lines of Isabelle/HOL covering Munkres' general topology, with all 806 results proved. This thesis was developed with AI assistance as well (see Use of Generative AI), and its proofs pass the same check.

The kernel guarantees only that the formal statement follows from the definitions. Whether the definitions model the intended objects, and whether the statement expresses the intended claim, is left to human review. A review of parts of the Munkres formalization found definitions that are logically weaker than the textbook's. They are harmless only because each theorem assumes the missing constraints again [11, §8.1].

Verification projects therefore explicitly state what lies outside the proof. The seL4 proof relates the kernel's C implementation to an abstract specification in Isabelle/HOL and names the compiler, assembly code, boot code, cache management, and hardware as assumptions [15]. Voblint draws its boundary at the source language. Its main theorem starts from *source executions*, the executions of a program under the small-step semantics `pstep` of Voblint's input language VIMP. Every later representation is connected to source executions by proof, so only `pstep` needs an adequacy argument. That argument states which fragment of C it models, where it departs from C11 (Section 3.3), and why no existing verified semantics serves as the anchor.

1.1. From executions to static guarantees

Software is usually checked by testing. A failing test demonstrates a bug, but a passing test suite has observed only finitely many executions [16, §1.4.1]. A static analysis reasons about all executions instead. At each program point, it computes an *abstract state*. An abstract state is a finite representation of a set of program states, for instance, an interval that

contains every value a variable takes whenever execution reaches the point. The analysis is *sound* if the abstract state contains every state that occurs in a real run, although it may contain more. If a sound abstract state contains no state that violates a check, no execution violates it, tested or not (Figure 1.1). The converse does not hold. An abstract state can contain violating states that no execution reaches, and the analysis then cannot decide the check, even when every execution satisfies it.

Suppose the analysis computes $x \in [43, +\infty]$ at a point that divides by `x`. Every execution that reaches this point has $x \geq 43$, so the division is safe. If it computes only $x \in [0, 100]$, the abstract state contains $x = 0$, and the analysis cannot prove the division safe. However, every real execution may still have $x \geq 43$, for instance, because of a relation to another variable that an interval cannot express.

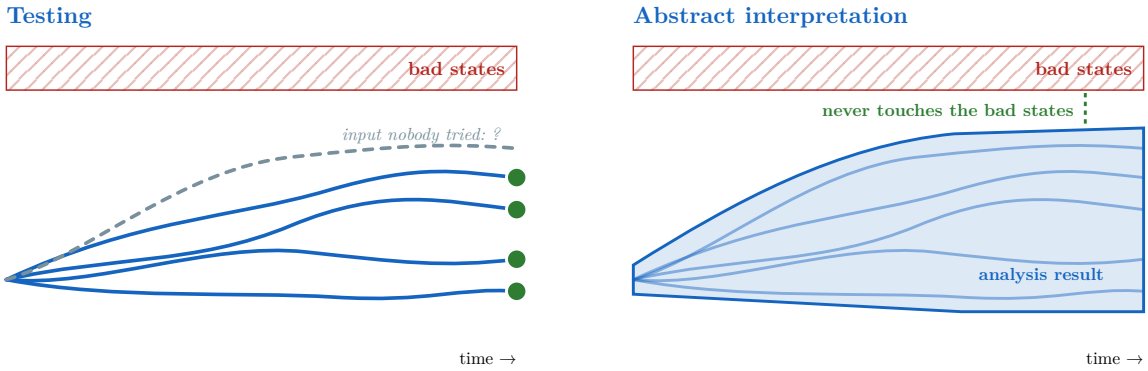
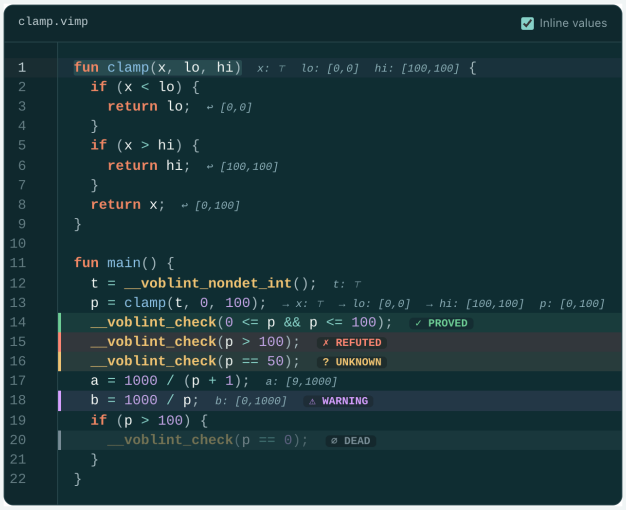


Figure 1.1: Testing and static analysis, schematically. Each curve is one execution, and the vertical axis stands for the program state over time. Left: tests observe only the runs they execute, and a run on an input nobody tried (dashed) stays unknown. Right: a sound analysis result (shaded) contains every run, tried or not, and may also contain states that no run reaches. Because it does not overlap the bad states, no execution can reach them.

For a general-purpose language, this incompleteness is unavoidable. By Rice's theorem, no algorithm decides a nontrivial semantic property for every program of a Turing-complete language [16, §1.3.3], [17]. A sound analysis may therefore raise false alarms and leave checks undecided, but it never makes a false claim. It is useful as long as it decides the checks of interest often enough [16, §1.3.5]. Since every sound analysis over-approximates the same executions, analyses can specialize. One tracks intervals, another parities, and another relations between variables. If any one of them proves a check, the check holds in every run. An analysis can also separate the activations of a procedure by their *calling context*, an abstraction of how the call was reached, such as the call site or the abstract state at entry. It then keeps one abstract state per program point and context. A richer domain or more contexts can decide more checks, but they also make the analysis more expensive. If there are many contexts, the analysis may not finish in practice.

Voblint answers every check that the user writes into the source program as a `check` statement. It also warns at every division and remainder where it cannot exclude a zero divisor. Figure 1.2 shows a run in the browser playground together with what each answer guarantees. Only `DEAD` makes a claim about reachability, and `REFUTED` is not a verified counterexample. Each run fixes an analysis configuration. It consists of the activated analyses (the domains Sign, Interval, Parity, Congruence, and the reduced-product domain

Int, and an order analysis), the context policy that chooses the contexts, the rule that combines contributions to global values, and the placement of the program's globals. The run shown uses intervals without calling contexts.



Line	Answer	What it guarantees
14	PROVED	the condition holds whenever a run reaches it
15	REFUTED	the condition fails whenever a run reaches it; not a counterexample
16	UNKNOWN	nothing
20	DEAD	no run reaches the check
18	WARNING	a zero divisor could not be excluded; no run is shown to divide by zero
17	none	no run reaching this point divides by zero

Figure 1.2: One analyzer run on a clamp function, as the browser playground shows it, with the interval domain and no calling contexts. The analysis covers every value of `t` at once. The screenshot opens the run in the playground, as does the `VIMP` ↗ tag on every later VIMP listing (claim `intro-answers`, fixture `24-site-figures/precision/48-clamp_every_answer.vimp`).

1.2. State of the art

Most abstract interpreters in practical use have no machine-checked soundness proof and rely on testing instead. Livshits et al. observe that virtually all published whole-program analyses purposely make unsound choices [18, p. 44]. Such an analysis over-approximates most language features but under-approximates a few features that experts know well, for instance, reflection in Java, `eval` in JavaScript, or `setjmp`/`longjmp` in C. The authors call such analyses *soundy*. They treat this as an engineering choice for precision and scalability, and they ask authors to state clearly the nature and extent of the unsoundness, instead of mentioning it only in passing [18, p. 45].

Table 1.1 compares Voblint with six analyzers that have no machine-checked soundness proof. Five of them claim to be sound. Astrée, for instance, proves the absence of run-time errors in C programs of up to 132,000 lines [25]. Pulse-X is a bug finder instead, and its under-approximating soundness theorem is proved on paper [23, pp. 81:2–81:3]. The tools differ most in how they treat procedures. Four of them inline the callee at every call and therefore need no context policy [20, §5.3], [21, §5.1], [22, §3], [25, §5.3]. Inlining cannot handle unbounded recursion, and these tools exclude recursion, unroll it to a bounded depth, rely on user-written contracts, or assume that a recursive call may change any value [20, §6.3.9], [25, §5.3], [26, Limitations], [27, Analysis Assumptions]. Goblint, like Voblint, keeps one abstract state per program point and calling context and analyzes recursive programs. It is therefore the closest model for Voblint, but only its solver has a mechanized proof [3], [4].

Such tools are checked against executions. Cuoq et al. [28] compare Frama-C's value analysis with compiled execution on programs generated by Csmith and report fifty bugs found and fixed. By differential testing, which flags answers on which analyzers disagree, Klinger et al. [29] find soundness or precision issues in four of six analyzers, and Kaindlstorfer et al. [30] find 16 soundness issues in seven of eight, including MOPSA. Since its first participation in SV-COMP in 2021, Goblint has never produced a false negative on the competition's benchmark suite [2]. Still, its developers report soundness issues that the benchmarks missed [31, §3.4.6]. The Goblint defect at the start of this chapter is an error in one abstract operation. Voblint proves a soundness obligation for each of its abstract operations, which excludes this kind of error.

CompCert provides a precedent for a theorem about a delivered tool. It proves that its compiled code behaves as the source semantics specifies, and it lists the components that remain trusted [32]. Verasco carries the approach to a static analyzer for C [33]. For Goblint, local traces give each thread a concrete semantics against which its thread-modular analyses are proved sound on paper [5], [34], [35, Ch. 6].

Table 1.2 compares the mechanized abstract interpreters most closely related to Voblint. Their proofs cover small or intermediate languages, from While and IMP to C#minor. The differences that matter for Voblint lie in recursion, contexts, and the fixpoint computation. Verasco raises an alarm where a recursive call can occur [33], and three of the works have no procedures at all. No work in the table lets the context policy give one call several contexts, which an analysis needs when it splits one call into several alternatives. Only Cachera et al. compute the fixpoint with a verified solver, and that solver handles ordinary inequations, not side-effecting ones [37]. Only two works specify the constraints without computing them, and they have no executable analyzer [39], [40].

Tool	Language	Stance	Calls	Recursion	Relational	Combination	Mechanized
Astrée [19]	C subset	sound	inlining	not used	octagons	channels	none
Eva [20]	C	sound	inlining	contract	Apron	via values	none
MOPSA [21]	C, Python	sound	inlining	unsupported	Apron	products, queries	none
IKOS [22]	C/C++	sound	inlining	assumptions	Apron	fixed products	none
Pulse-X [23]	C	bug finding	summaries	unrolled	pure conditions	one domain	none
Goblint [24]	C	sound	per context	analyzed	Apron	products, queries	TD algorithm
Voblint	VIMP	proved sound	per context	covered	order analysis	products, queries	end to end

Table 1.1: Unverified analyzers and Voblint, as described by each tool's papers and documentation. *Stance* is the tool's own soundness claim. Under *calls*, inlining reanalyzes the callee at every call, a summary is computed once per procedure, and *per context* means one abstract state per program point and context. *Recursion* says how a recursive call is treated. *Analyzed* means the tool analyzes recursive calls, and *covered* means that Voblint's theorem includes them. *Relational* names the domains that relate variables. Apron is a library of relational numeric domains, and Voblint's order analysis tracks which variable is at most which. *Combination* names how domains exchange facts. Astrée's domains send each other constraints through reduction channels, Eva's domains communicate through the abstract values of expressions, a product runs domains side by side on one state, and a query lets one analysis ask another for a fact at the current point. *Mechanized* records what a proof assistant has checked. *End to end* means from source executions to the verdicts of the exported function, for every answer it returns (partial correctness) and with the trusted components of Section 12.5.

Voblint's architecture follows Goblint, an abstract interpreter for multithreaded C programs [1], [2]. Goblint defines analyses independently of the generic solvers that compute their results. The interface between the two is a side-effecting constraint system [2], [42], [43]. Such a system relates unknown abstract values by inequalities, and the inequality of one unknown may also contribute to other unknowns. Goblint's top-down solver, including its extension to side effects, has been formalized and proved partially correct in Isabelle/HOL [3], [4]. When it terminates, it returns values that satisfy the inequalities of the queried unknown and of every unknown it transitively reads (`part_post_solution`). Whether that system describes the program, and whether the verdicts read off its solution hold, lies outside the solver's theorem.

Calling contexts add another gap, since the per-context abstract states of an analyzer have no counterpart in an ordinary concrete semantics. Suppose `main` calls `bump(5)` and `bump(4)`, and `bump` returns its argument plus one. An analysis whose contexts record the argument at entry claims the result 6 in the first context and 5 in the second. The collecting semantics at the point where `bump` returns holds both results. It forgets how each store was reached, so it does not say which store belongs to which context. To state these claims, the concrete semantics must record the context of every *activation*, one execution of a procedure from its call to its return.

To our knowledge, no prior mechanized analyzer connects a source semantics to a side-effecting constraint system and its verified solver. Among the works of Table 1.2, Dabrowski and Pichardie are closest to Voblint in their treatment of contexts. Their context function is a parameter of the concrete semantics, but it gives each call exactly one context. Their analyses are constraints that no solver computes [39, §7]. Voblint's theorem combines four properties that no work in the table has together. It covers recursive procedures, takes

Work	Prover	Language	Recursion	Contexts	Fixpoint	Executable
Verasco [33]	Coq	C#minor	alarm	per call site	structural	yes
Blazy et al. [36]	Coq	Cminor CFG	covered	intraprocedural	checked	yes
Cachera et al. [37]	Coq	Java Card	covered	none	solver	yes
Cachera, Pichardie [38]	Coq	While	no procedures	none	structural	yes
Dabrowski, Pichardie [39]	Coq	Java-like	covered	object-sensitive	specified	no
Nipkow, Klein [6]	Isabelle	IMP	no procedures	none	iterator	in prover
Lammich, Müller-Olm [40]	Isabelle	flowgraphs	exact	summaries	specified	no
Franceschino et al. [41]	F*	IMP	no procedures	none	structural	yes
Voblint	Isabelle	VIMP	covered	set per call	TD solver	yes

Table 1.2: Mechanized abstract interpreters with machine-checked soundness proofs, as described in each work. *Prover* is the proof assistant and *Language* the analyzed language. *Recursion* says how a recursive call is treated: *alarm* raises an alarm, *covered* means the theorem includes recursive programs, *exact* interprets calls and returns exactly, and *no procedures* means the language has none. *Contexts* says how calls are distinguished: *per call site* reanalyzes a function at every call site, *object-sensitive* computes contexts by a function, *summaries* computes one summary per procedure, *set per call* lets the context policy return several contexts for one call, and *none* and *intraprocedural* mean that calls are not distinguished or not analyzed. *Fixpoint* says how the result is computed: *structural* iterates over the program's syntax, *iterator* over the whole annotated program, *checked* verifies the result of an untrusted iterator, *solver* and *TD solver* use a verified solver, and *specified* only states the constraints. *Executable* says whether the analyzer runs outside the prover. Chapter 15 gives the details.

the context policy as a parameter that may give several contexts to one call, is proved through a verified solver, and holds for the executable analyzer. It is also stated over source executions, whereas Verasco's theorem concerns C#minor, an intermediate language of CompCert. Voblint claims no better precision than any of these tools.

Voblint builds on prior work. The solver and the soundness proofs of its update rules come from the Isabelle formalization of Tilscher et al. [4]. Voblint uses a copy of it with changes made for this thesis (Section 14.3). The update rules come from Stemmler et al. [44], side-effecting constraint systems from Seidl et al. [42] and Apinis et al. [43], and the local/global analysis architecture from Goblint. Widening, narrowing, and the reduced product are well studied [45], [46]. The derivation of transfer functions from sound value operations follows Nipkow and Klein [6, Sects. 13.5, 13.7], and the activation traces adapt the local traces of Schwarz et al. [5] to procedure activations. Isabelle's code generator [47] produces the OCaml code the tools run.

1.3. Contributions

The contribution is the Isabelle/HOL formalization of Voblint and its machine-checked soundness proof, available at `github.com/ManuelLerchner/Voblint-Verification-Master-Thesis` and archived on Zenodo [48]. Figure 1.3 shows the chain of inclusions that the proof establishes at one program point, from source executions to the reported verdicts. Each step may add states that no execution reaches, which costs precision, but must not lose a reachable state. Errors such as a compiler that drops an edge or a return to the wrong caller would lose such a state, and the proof excludes them.



Figure 1.3: Soundness at a program point v as nested sets. A valid trace records, besides the stores of a graph run, which call created each activation, and every graph run is represented by one. Splitting the node collecting semantics by context loses no store. The source-level theorem chains these inclusions for every report the analyzer returns (`run_voblint_source_sound`, Chapter 10).

The formalization makes four contributions. The first is the end-to-end theorem itself. The other three make this theorem possible or show that it is not vacuous.

- *End-to-end soundness.* Whenever the analysis function `run_voblint` returns a report, its definite verdicts (`PROVED` and `REFUTED`) hold for every source execution from an initial store with zeroed globals that reaches the corresponding program point, under every analysis configuration the function offers (`run_voblint_source_sound`, Section 10.4). Companion theorems justify `DEAD` and the absence of arithmetic warnings (Chapter 10).
- *A concrete semantics of calling contexts.* An activation trace records the path of one activation together with the activation that called it. Therefore, the contexts that a

policy assigns to a call can be read off the concrete execution. A policy may assign several contexts to one call. Assume that an analysis claim meets the coverage contract. This contract includes that every call from a store the claim contains gets at least one context. Then the stores collected per context together are exactly the stores of the node collecting semantics, which ignores contexts (`node_collect_eq_Union_activation_collect`, Section 4.4).

- *Separately verified components.* Domains, context policies, analyses, and the solver are each verified against their own interface, and generic theorems compose them for every configuration. A numeric domain proves only its primitive operations sound, and the transfer functions derived from them are sound for every such domain (`sound_nonrelational_ops`). A context policy proves only that its contexts agree with the concrete semantics and that every call from a store that the claim contains gets a context. Analyses that ask one another for facts can be combined under conditions on their answers (Chapter 7). The solver enters the proof only through proved facts about its result (Section 9.3.3).
- *Needed assumptions and nonempty results.* For several conditions of the proof, a theorem shows that dropping or weakening the condition admits an answer that a concrete execution contradicts. Examples are an analysis result that misses a store that some run reaches, or an abstract remainder that excludes the concrete one. Therefore, none of these conditions can simply be dropped (Section 13.2). One example is dropping the requirement that every call from a store the claim contains gets a context (`total_dropped_unsound`). Since a soundness theorem would also hold for an analyzer that answers `UNKNOWN` everywhere, further theorems show definite verdicts on concrete programs (Section 13.1) and that one context policy is strictly more precise than another on one program (Section 13.3).

1.4. Scope and limitations

Voblint is a Goblint-style analyzer with several context policies, over a small language with parameters, return values, and recursive procedures. It isolates the parts of Goblint's architecture that the proof is about: calling contexts, side-effecting constraint systems, configurable domains, analyses that answer one another's queries, and the top-down solver. It is not a verification of Goblint itself. *End-to-end* means that the proof reaches from source executions to the result that the analysis function returns. Figure 1.4 places each stage of the analyzer relative to the proof.

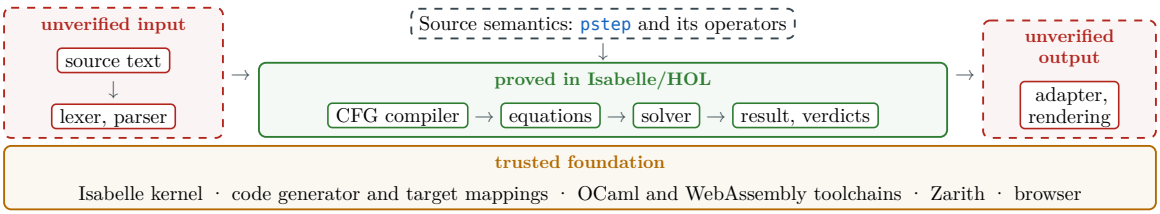


Figure 1.4: Where the proof starts and stops, for accepted programs whose solve terminates. The adequacy of `pstep` is argued (Section 3.3). The handwritten adapters connect the generated code to the command line and the browser, Zarith is OCaml's arbitrary-precision integer library, and Section 12.5 gives the exact boundary.

The theorem has the following limits (Section 16.2).

- *Language and properties.* VIMP has no pointers, heap, or threads. Its integers are unbounded, and division by zero is defined. Therefore, a verdict does not always hold for a C program with the same text (Section 3.3). Voblint checks assertions and division by zero, while Goblint also checks overflows, data races, memory safety, memory leaks, and program termination [31, §3.3].
- *Adequacy and trusted components.* We argue, but do not prove, that `pstep` models the intended language (Section 3.3). The parser, the code generator, and the target toolchains lie outside the proof (Section 12.5).
- *Partial correctness.* The theorem covers every report the analyzer returns. Termination of the solve is not proved for every program (Section 9.6).
- *Precision and cost.* No completeness or general precision ordering is proved (Section 13.3). The executable uses the same data representations as the proofs, and we report no time or memory measurements (Section 16.2).
- *Goblint and evidence.* The correspondence to Goblint is architectural, and no theorem transfers to Goblint's implementation (Section 15.5). The regression corpus is small and was written or adapted for this work (Section 13.4).

1.5. Outline

After Chapter 2, the thesis follows the nested sets of Figure 1.3 from the inside out. Part II fixes what an analysis must over-approximate: VIMP and its compilation to a control-flow graph (Chapter 3), and activation traces indexed by calling context with the coverage contract that analyses must meet (Chapter 4). Part III develops what each analysis contributes, without contexts or a solver: the abstract domains (Chapter 5), the interface through which an analysis supplies its transfer functions (Chapter 6), and the combination of analyses that query one another (Chapter 7).

Part IV turns any analysis that meets this interface into a sound result. Chapter 8 generates the equations of a program, Chapter 9 computes their solution with the verified solver, and Chapter 10 composes these parts into the source-level theorem. Part V puts the theorem to use. Chapter 11 instantiates it for five domains and an order analysis. Chapter 12 follows `run_voblint` to the delivered tools and their trust boundary. Chapter 13 tests the premises and obligations, the precision, and the unverified code, and Chapter 14 describes the tooling. Part VI compares Voblint with prior work (Chapter 15) and closes with limitations and future work (Chapter 16).

2. Background

Chapter 1 described soundness informally: an analysis result must contain every state that a real run reaches, and it may contain more. Such a result is an *over-approximation*. To prove that Voblint computes one, we need precise definitions of the runs, of the abstract states, and of how an analyzer computes them from a system of constraints. We introduce them using one small program, which we follow from its executions to the result of the solver.

2.1. Executions and the collecting semantics

An analysis describes the states a program can be in at each of its points, so we first make the points explicit. A *control-flow graph* has the program points as nodes and an edge for every way control can pass from one point to the next, labeled with the assignment or condition executed on the way. Figure 2.1 shows the graph of a loop that counts `i` from 0 to 5, our running example. The loop becomes a cycle through h, b, and t, and its *loop head* h is the node through which control enters and exits the cycle.

```
1 i = 0;                                    VIMP
2 while (i < 5) {
3   i = i + 1;
4 }
```

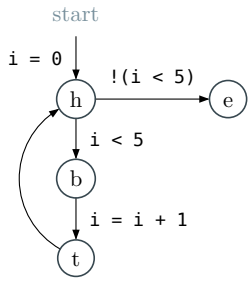


Figure 2.1: The counting loop and its control-flow graph (schematic). Nodes are program points: h is the loop head, b the start of the body, t the point after the increment, and e the point after the loop. Edges carry the assignment or the condition that leads from one point to the next. The edge from t back to h has no effect.

The state of an execution is a *store*, which maps each variable to its value, here the value of i. An execution starts at the entry of the graph in an initial store and follows a path, updating the store at every edge. The counting loop reaches h once before the first iteration and again after each run of the body, so i takes exactly the values $0, 1, ..., 5$ there. Collecting, for every program point, the stores of all executions that reach it gives the *collecting semantics* $\mathcal{C}$ [45, §4], [49, §3.4], which maps every program point v to the set $\mathcal{C}(v)$ of stores reached there. Here $\mathcal{C}(h)$ holds the stores with $i \in \{0, ..., 5\}$. For programs with procedures, Chapter 4 defines Voblint's version over execution traces (`node_collect`).

The collecting semantics is the exact answer an analysis approximates. An analysis is *sound* if, at every program point, its result describes every store that the collecting semantics gives there. It may describe more stores, which costs precision, but never fewer. Soundness thus relates three levels: the individual executions, the collecting semantics that

gathers the stores they reach, and the abstract states that an analysis computes. Figure 1.3 refines these levels for Voblint.

2.2. Lattices and fixpoints

At h, the intervals $[0, 5]$ and $[0, 10]$ both contain every value i takes there (the collecting semantics $\{0, ..., 5\}$), but $[0, 10]$ also admits $i = 7$, which no execution reaches. A description is *more precise* than another if it admits fewer states. Ordered sets make this comparison formal. In an ordered set $(\mathbb{D}, \leq)$ (`order`), $a \leq b$ means that a is at least as precise as b [6, Def. 10.23]. For sets of concrete states, the order is inclusion.

Two descriptions reaching a point along different paths must be combined into one that covers both. Their *join* $a \sqcup b$ is the least upper bound, the most precise element above both (`semilattice_sup`). At the loop head, the value before the loop and the value after the body are combined this way. Dually, the *meet* $a \sqcap b$ is the greatest lower bound. An analysis uses it at a branch to restrict the incoming value to the values the condition admits. If the analysis knew only $[0, 10]$ at h, entering the body would restrict it to $[0, 10] \sqcap [-\infty, 4] = [0, 4]$. Soundness does not need this refinement, since $[0, 10]$ itself is sound at b, but it makes the result more precise, and Goblint and Voblint both refine at branches. The least element $\bot$ lies below every element, and a top element $\top$, if one exists, lies above every element. In abstract interpretation, $\bot$ describes no states and $\top$ describes all of them. A *complete lattice* (`complete_lattice`) has a join $\bigsqcup X$ and a meet $\bigsqcap X$ for every subset X [6, Def. 13.1], [49, Def. 2.5].

Voblint's solver, however, requires only a join semilattice with a least element (`bounded_semilattice_sup_bot`) that carries the widening and narrowing operators of Section 2.5, so joins need to exist only for pairs. A numeric domain adds a top element. Refinement at a branch adds a pairwise intersection that keeps every value both operands admit (`sound_intersection`, Section 5.5).

An analysis computes the description at a point from the descriptions at its predecessors, by a function f on $\mathbb{D}$. The classical theory requires f to be *monotone* (`mono`), so that $a \leq b$ implies $f(a) \leq f(b)$ [6, Def. 10.26], [49, Def. 2.9] and more precise input never yields less precise output. For a monotone f, the iterates $\bot, f(\bot), f(f(\bot)), ...$ form an ascending chain. Each of them lies below every fixpoint. The verified solver that Voblint reuses, however, does not assume monotonicity.

A *fixpoint* of f is an element with $f(d) = d$, and a *post-fixpoint* is an element with $f(d) \leq d$, so applying f once more yields nothing above d. By the Knaster–Tarski theorem, a monotone f on a complete lattice has a least fixpoint $\operatorname{lfp} f$, the meet of all post-fixpoints [50, Thm. 1] (`lfp`, `lfp_unfold`), so every post-fixpoint lies above $\operatorname{lfp} f$. For a monotone f on a finite-height order, the iteration from $\bot$ stabilizes after finitely many steps [51, §1.5], and an iterate that no longer changes is the least fixpoint (`lfp_Kleene_iter`).

An analysis does not have to reach $\operatorname{lfp} f$. Every post-fixpoint of an f that includes the initial states and approximates each step soundly already contains the collecting semantics (Section 2.3).

2.3. Abstract interpretation

The correctness of an abstract result needs a meaning for its elements. Let U be a set of concrete objects, such as integers or stores (`store`), and A an *abstract domain*, an ordered set of *domain elements* whose underlying set is its *carrier*. A domain element is an *abstract value* when it describes the values of one variable, such as an interval for one integer, and an *abstract state* when it describes stores. A concretization $\gamma : A \to \mathcal{P}(U)$ gives each domain element its meaning. It is monotone, so a less precise element admits at least the same objects: $a \leq b$ implies $\gamma(a) \subseteq \gamma(b)$ (`gamma_mono`). Soundness at a program point v (Section 2.1) then becomes an inclusion. If a is the analysis result at v, then

$$\mathcal{C}(v) \subseteq \gamma(a),$$

the consistency requirement between an abstract and a concrete interpretation in [45, §6], [49, Def. 2.11].

The running example uses the *interval domain* [45, §9.2], [49, §4.5]. Its elements are $\bot$ and the intervals $[l, u]$ with $l \in \mathbb{Z} \cup \{-\infty\}$, $u \in \mathbb{Z} \cup \{+\infty\}$ and $l \leq u$, and an interval denotes the integers between its bounds:

$$\gamma([l, u]) = \{n \in \mathbb{Z} \mid l \leq n \leq u\}, \quad \gamma(\bot) = \emptyset.$$

The order is inclusion of these sets, so $[l, u] \leq [l', u']$ holds exactly when $l' \leq l$ and $u \leq u'$. The join $[l, u] \sqcup [l', u'] = [\min(l, l'), \max(u, u')]$ is the smallest interval containing both. An interval cannot have holes, so the join may contain values that neither operand contains: $[0, 0] \sqcup [5, 5] = [0, 5]$. The meet intersects the bounds. This is the textbook domain. Voblint's representation `ivl` admits every pair of bounds, so $\bot$ is one of several empty pairs (Section 5.1).

At the loop head, both $[0, 5]$ and $[0, 10]$ are sound (Figure 2.2), as is every interval that contains $[0, 5]$. An analysis aims for a precise one, because a more precise element admits fewer values and so decides more checks: $[0, 5]$ proves `i <= 5` at the loop head, and $[0, 10]$ does not.

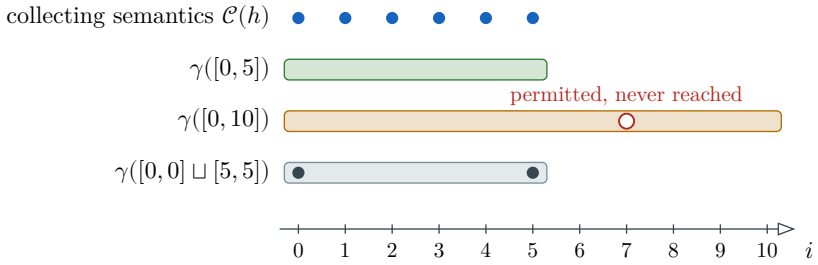


Figure 2.2: Concretization of intervals at the loop head. Both $[0, 5]$ and $[0, 10]$ contain the collecting semantics $\mathcal{C}(h)$, and $[0, 10]$ also permits $i = 7$. The join of $[0, 0]$ and $[5, 5]$ adds the values in between.

Much of abstract interpretation also assumes an abstraction function $\alpha : \mathcal{P}(U) \to A$ that forms a *Galois connection* with γ and gives every set of states a best abstraction [46, §5.3], the most precise domain element whose concretization contains the set. Not every useful domain has such best abstractions, and Cousot and Cousot treat a Galois connection as the ideal case that efficient representations do not always meet [52, §5]. Voblint uses γ alone, like the verified analyzers of Jourdan et al. and of Nipkow [33, §2], [53, §5.2], since its soundness theorem claims coverage and nothing about optimal precision. The

concretization exists only in the logic. Since its result is in general an infinite set, it is never executed, and the analyzer itself is defined from executable operations only, so that it can be exported and run as a standalone program.

The analysis replaces each concrete operation by an abstract one, a *transfer function* $f^\sharp : A \to A$, which must not lose any state that the concrete operation can produce. In general, a program's steps form a relation $\to$ on U, and one state may have several successors, even infinitely many, for example, when the next value is read from user input. The *post-image* $\mathrm{post}(X) = \{s' \mid \exists s \in X. s \to s'\}$ collects the successors of all states in X. The transfer function is sound if, for every a,

$$\mathrm{post}(\gamma(a)) \subseteq \gamma\big(f^\sharp(a)\big),$$

so stepping from any state that a describes leads to a state that $f^\sharp(a)$ describes [16, §4.2.2], [45, §6], [49, Def. 2.15]. The square of Figure 2.3 thus commutes only up to inclusion. In the counting loop, the edge from b to t executes `i = i + 1`. Over $\mathbb{Z}$, the interval transfer $[l, u] \mapsto [l + 1, u + 1]$ is sound for it, and applied to $[0, 4]$ it yields $[1, 5]$, which contains every successor $i = 1, ..., 5$. Soundness composes along paths [49, Thm. 2.6], and at control-flow joins the upper-bound laws of the join keep the states from both predecessors (`gamma_sup_ub1`, `gamma_sup_ub2`).

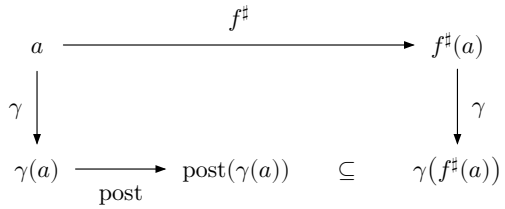


Figure 2.3: Soundness of a transfer function $f^\sharp$: the square commutes up to inclusion.

Sound transfer functions make every post-fixpoint that contains the initial states a sound result [49, Thm. 2.8]. The standard argument uses one domain element that describes the whole program. It works with *configurations*, each a program point together with a store, so that one relation $\to$ describes every step of the program, each moving an execution along one edge. The constraint systems of Section 2.4 then split this element into one abstract state per program point. In the counting loop, a configuration is a pair (p, n) of a program point and the value of i, so $U = \{\mathrm{start}, h, b, t, e\} \times \mathbb{Z}$. With the set I of initial configurations, the reachable configurations form the least fixpoint of

$$F(X) = I \cup \mathrm{post}(X)$$

[16, §4.1.1, Thm. 4.1]. A set with $F(X) \subseteq X$ contains the initial configurations and the successors of all its elements, so no execution can leave it, and $\mathrm{lfp}\, F$ is the smallest such set. In the counting loop, executions begin at the node `start`, with any value of i, so $I = \{(\mathrm{start}, n) \mid n \in \mathbb{Z}\}$. Iterating F from $\emptyset$ adds one step at a time: $(h, 0)$, then $(b, 0)$, $(t, 1)$, $(h, 1)$, and so on, up to $(h, 5)$ and finally $(e, 5)$. The least fixpoint contains (h, n) exactly for $n \in \{0, ..., 5\}$, which is $\mathcal{C}(h)$, since $\mathcal{C}$ groups the reachable configurations by their program point.

Now take a domain element a with $I \subseteq \gamma(a)$ and $f^\sharp(a) \leq a$, for a sound $f^\sharp$. Soundness of $f^\sharp$ and monotonicity of γ give $F(\gamma(a)) \subseteq I \cup \gamma\big(f^\sharp(a)\big) \subseteq \gamma(a)$. So $\gamma(a)$ is a post-fixpoint of F and, by Knaster–Tarski, contains $\mathrm{lfp}\, F$, every reachable configuration. Here F works

on sets of configurations and $f^\sharp$ on domain elements, and the soundness of $f^\sharp$ relates the two through γ.

Voblint's proof uses the same principle on a set of execution traces defined inductively as a least fixpoint, and proceeds by induction over its rules. Analyzers such as Goblint keep one abstract state per program point [16, §4.2.1] and state one inequality per point, whose solution plays the role of a.

2.4. Constraint systems

An analysis needs an abstract state at every program point, and the state at one point depends on the states at its predecessors. The analysis therefore states its result as a system of constraints. Each *unknown* of the system stands for the abstract state at one program point, and each constraint says what the unknown must cover.

The counting loop has one variable, so an abstract state is an interval for i. (With several variables, a *non-relational* state maps each variable to its own abstract value.) With one unknown per program point of Figure 2.1, each unknown must cover what the edges entering its point produce:

$$h \geq [0,0] \sqcup t, \qquad b \geq h \sqcap [-\infty, 4],$$
$$t \geq b +^\sharp [1,1], \quad e \geq h \sqcap [5, +\infty].$$

The assignment `i = 0` yields $[0,0]$ at h, the condition `i < 5` restricts the branch into the body, and its negation restricts the exit. Substituting b and t into the first inequality gives one for the loop head, $h \geq f(h)$, with

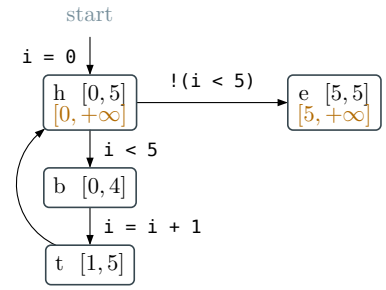
$$f(d) = [0,0] \sqcup \big((d \sqcap [-\infty, 4]) +^\sharp [1,1]\big),$$

whose post-fixpoints appear in Figure 2.4b. A solution assigns an interval to every unknown such that all four inequalities hold, and Figure 2.4a shows two. Both contain the collecting semantics ($\{0, ..., 5\}$ at h), and the second is less precise at h and e, so a solution need not be the least one.

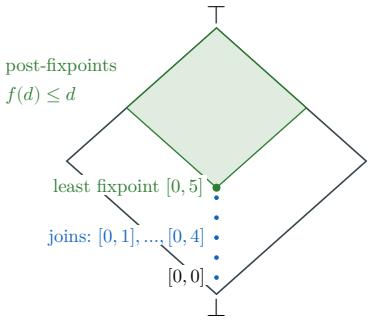
In general, let $\mathcal{X}$ be a set of unknowns and $\sigma : \mathcal{X} \to A$ a *valuation*, which assigns a domain element to every unknown. Valuations are ordered pointwise: $\sigma \leq \sigma'$ if $\sigma(x) \leq \sigma'(x)$ for every unknown x. Each unknown x has a *right-hand side* rhs_x, a function of the valuation, and σ is a *post-solution* if

$$rhs_x(\sigma) \leq \sigma(x) \quad \text{for every unknown } x$$

(`post_solution`), what the constraint-system literature calls a solution [31, §2.4.4]. For the loop head, $rhs_h(\sigma) = [0,0] \sqcup \sigma(t)$. A post-solution is a post-fixpoint of all right-hand sides at once, so the argument of Section 2.3 applies at every program point, and a post-solution bounds the reachable states everywhere. We use the term *constraint system* for the general concept and the term *equation system* for the system that Voblint generates and the solver receives, although soundness only requires inequalities.



(a) two solutions on the graph



(b) post-fixpoints of $h \geq f(h)$

Figure 2.4: The counting loop's inequalities, solved by hand. (a) The least solution at each node of Figure 2.1 and, in orange where it differs, a less precise solution. (b) Post-fixpoints of the loop-head constraint (schematic, shaded) and its least fixpoint $[0, 5]$.

The formulation goes back to data-flow analysis, where Kildall computes one value per node of a program graph by iteration [54, §3].

Programs with procedures can be analyzed with one unknown per program point or separately per calling context, which abstracts how an activation (one execution from call to return) was reached [55, §7.2.1]. Sharir and Pnueli distinguish call strings, which record recent call sites, from the functional approach, which relates abstract entry and return states [56]. Context-sensitive constraint systems use unknowns $[v, c]$ for program point v and context c [43, §3]. For the calls `bump(5)` and `bump(4)` in Chapter 1, entry-argument contexts distinguish return values 6 and 5, while a single unknown must cover both.

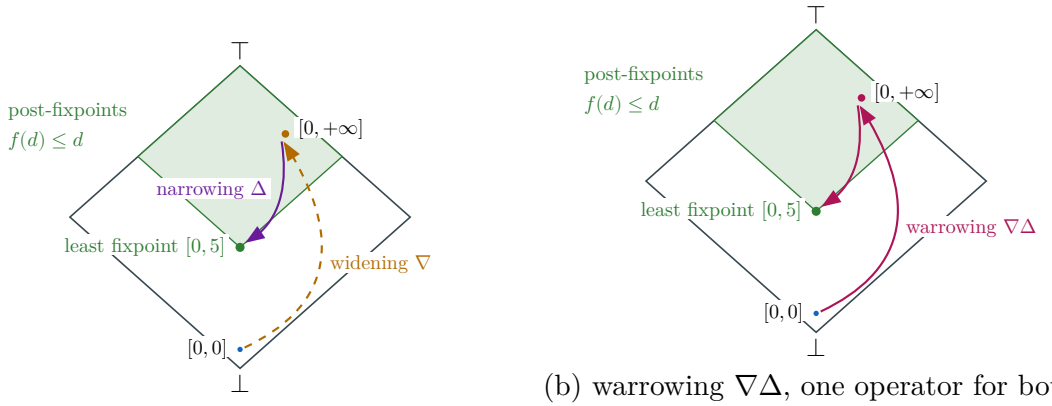
2.5. Widening and narrowing

An analysis should terminate in reasonable time, even on a program whose executions do not terminate. With joins alone, successive abstract iterates can mirror successive loop iterations. At the example's loop head, joins take five increases after $[0, 0]$. For `while (true) { i = i + 1; }`, the iteration over intervals with bounds in $\mathbb{Z}$ never stabilizes, because these intervals have infinite ascending chains such as

$$[0, 0] < [0, 1] < [0, 2] < \ldots.$$

A *widening* $a \nabla b$ replaces the update from the old value a to the new value b by a guess that lies above both, which keeps the result sound (`widening`). A classical widening is also chosen so that repeated widening stabilizes after finitely many steps [45, §9.1.3], [52, §4], which the upper-bound law alone does not ensure. A common strategy widens at loop heads. The standard interval widening replaces a growing bound by $+\infty$ [45, §9.2], [49, §4.5.2], so widening at h turns the first change there, from $[0, 0]$ to $[0, 1]$, into $[0, +\infty]$ at once. Propagated through the loop, this value yields the less precise solution of Figure 2.4a, which loses the bound 5 of the loop condition `i < 5`.

The branch into the body restricts $[0, +\infty]$ to $[0, 4]$, the body yields $[1, 5]$, and evaluating the head again gives $[0, 0] \sqcup [1, 5] = [0, 5]$. A widened iteration only grows and would not take this smaller value. A *narrowing* lets it descend. Both steps appear in Figure 2.5a, and Figure 2.5b shows one update operator that combines them, widening or narrowing depending on the candidate.



(a) widening, then narrowing

(b) warrowing $\nabla\Delta$, one operator for both steps

Figure 2.5: Extrapolation on the lattice of Figure 2.4b. (a) Widening jumps from $[0, 0]$ to the post-fixpoint $[0, +\infty]$, and narrowing then recovers the least fixpoint $[0, 5]$. (b) Warrowing takes the same two steps with one operator, which widens while the candidate grows and narrows once it lies below the current value. Here both reach $[0, 5]$.

A narrowing $a\Delta b$ combines the current value a with a candidate $b \leq a$ and returns a value between the two [45, §9.3.4], [52, §4]:

$$b \leq a\Delta b \leq a.$$

These are the *bracket laws* of the solver's class `narrowing`. A classical narrowing must also make repeated narrowing stabilize. The class omits this law, since termination of the solver is not proved. In the counting loop, the interval narrowing reaches $[0, 5]$, but in general, narrowing need not recover the least fixpoint.

Soundness of the result does not depend on how the solver widens and narrows. The verified TD solver applies both operators in a single update, *warrowing* (`warrow`). It narrows when the candidate lies below the current value and widens otherwise [57], [58, §3], [59], so each update lies above the candidate (`warrowing_properties`). An unknown whose value no longer changes under this update therefore satisfies its inequality. The solver's correctness theorem uses only this and holds whether or not the right-hand sides are monotone (Section 2.7). Monotonicity is thus needed for the classical theory of least fixpoints, but not to certify that a returned valuation is a post-solution. As long as the operators satisfy their laws, the choice of widening and narrowing affects only precision and termination.

2.6. Side-effecting constraint systems

The analyses so far are *flow-sensitive*: they respect the order in which statements execute and therefore compute a separate abstract state for every program point. For some facts, one value for the whole program is enough. A *flow-insensitive* analysis ignores execution order and keeps one abstract value for an aspect of the state across the program, for example, one range for a global variable, and mixed analyses choose between the two per aspect [2].

A flow-insensitive fact needs an unknown of its own that belongs to no program point, and every point that changes the fact has to feed its new value into that unknown. An ordinary right-hand side for the unknown would have to list all these points in advance,

which is not always possible. In a context-sensitive analysis, for example, an unknown that writes a global is a pair of a program point and a calling context, and the contexts are discovered only while the system is solved.

Side-effecting constraint systems avoid that list. Seidl, Vene, and Müller-Olm introduced them for multithreaded programs [42, §3], and Apinis et al. use them for context-sensitive analysis [43]. While the right-hand side of one unknown is evaluated, it may *publish* a *contribution* to another unknown as a *side effect*. We follow the verified solver and let side effects target only *global unknowns* $\mathcal{X}_G$, which have no right-hand side and receive only contributions. From here on, $\mathcal{X}$ denotes the *local* unknowns, which keep their right-hand sides, and a valuation σ assigns values to both kinds. A global unknown need not correspond to a global variable of the program. A right-hand side then returns a pair [31, §2.4.5], $rhs_x : (\mathcal{X} \cup \mathcal{X}_G \to A) \to (\mathcal{X}_G \rightharpoonup A) \times A$, of a partial map ρ of the contributions it publishes, joined per global unknown, and its own value d. If $rhs_x(\sigma) = (\rho, d)$, a post-solution must bound both, $d \leq \sigma(x)$ and $\rho(g) \leq \sigma(g)$ for every $g \in \mathrm{dom}(\rho)$. If the assignments `g = 5` and `g = 4` publish $[5, 5]$ and $[4, 4]$ to the unknown of a flow-insensitive global `g`, that unknown must bound their join $[4, 5]$. This describes what a right-hand side computes. The verified solver represents its evaluation by a strategy tree (Section 2.7), so that it discovers reads and side effects while it solves.

With contexts as indices, a system can have infinitely many unknowns, of which only those that influence the *query*, the unknown whose value is asked for, matter. A *local* solver therefore explores the system from the query and solves only the unknowns that it meets [60]. Its result comes with a *certificate*, the property `part_post_solution`, which requires the bounds above only on a set of unknowns that contains the query and is closed under the local reads of its right-hand sides.

2.7. The verified top-down solver

A straightforward implementation of Kleene iteration recomputes every unknown in every round, although most values no longer change. Practical solvers therefore update one unknown at a time. They work in rounds or with a worklist of unknowns whose inequalities may be violated [51, §§1.5, 1.12], in orders chosen for widening [61], or locally, on demand from a query, as the solver RLD does [62]. Voblint reuses the verified local top-down solver (TD) of [3], [4].

TD starts from the query and evaluates right-hand sides on demand. When the right-hand side of x reads an unknown y, TD first solves y and records that x depends on y. An unknown is *stable* if it has been evaluated and no unknown that it read has changed since then. When the value of y changes later, every unknown that read it is *destabilized* and evaluated again. The unknowns at which a read closes a cycle are the solver's *widening points* [60]. TD detects them while it solves [58, §4], so they need not coincide with the loop heads of the control-flow graph. At a widening point, it combines the old and the new value with warrowing. The playground replays this run on the counting loop step by step. Contributions published to a global unknown are merged into its value by an *update rule* (`update_rule`), which keeps one record per *origin*, the unknown whose right-hand side published the contribution (Section 9.4). The run ends once every unknown the query transitively reads is stable, and it returns this *stable set* V together with the valuation σ.

To solve on demand, TD has to discover which unknowns a right-hand side reads, so that it can solve each of them first. In the counting loop, the reads are fixed, since the right-hand side of h always reads t. In general, a right-hand side may choose the next unknown from a value it has already read. At a procedure call, for instance, the callee context to read is known only once the caller's value has been read. The verified solver therefore receives every right-hand side as a *strategy tree* (`strategy_tree`), a small program that reads one unknown at a time and makes every read and every side effect explicit:

$$\tau \coloneqq \mathsf{Answer}(d) \mid \mathsf{QueryL}(y, k) \mid \mathsf{QueryG}(g, k) \mid \mathsf{Side}(g, d, \tau)$$

with a local unknown $y \in \mathcal{X}$, a global unknown $g \in \mathcal{X}_G$, a domain element $d \in A$ and a continuation $k : A \to \tau$ that receives the value read. $\mathsf{Answer}(d)$ returns d as the value of the right-hand side, QueryL and QueryG read a local or a global unknown, and $\mathsf{Side}(g, d, \tau)$ publishes d to g before continuing with τ. Figure 2.6 shows the trees of the counting loop and of a flow-insensitive assignment.

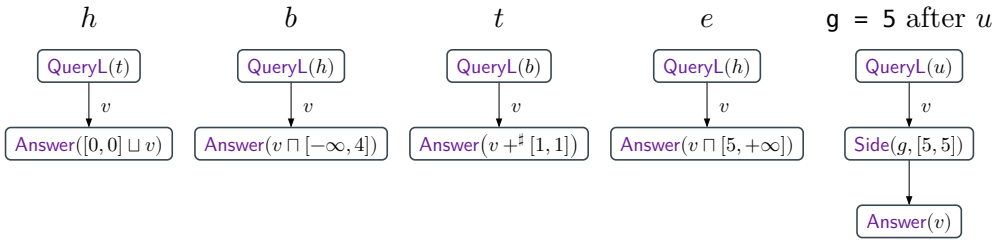


Figure 2.6: Right-hand sides as strategy trees. The first four are the inequalities of the counting loop (Section 2.4): each reads one unknown, passes its value v on, and answers. The last publishes the value of a flow-insensitive global `g` (Section 2.6) before answering.

The solver's main theorem, `partial_post_solution`, establishes the certificate `part_post_solution` of Section 2.6 and states partial correctness. If the run terminates on the query x and returns its stable set V and the valuation σ, then V contains x, is closed under the local reads of its right-hand sides, and on V the valuation is a post-solution, published contributions included. The theorem needs no monotonicity of the right-hand sides and says nothing about unknowns outside V. We call the set on which this certificate holds the *solved set*. Termination of this solver is not proved, and Section 2.8 explains how Isabelle defines such a function.

2.8. Isabelle/HOL mechanisms used in this development

This section explains the mechanisms of Isabelle/HOL [6] needed to read the definitions and theorems that later chapters show, each with the place where the formalization uses it.

HOL is a typed logic of total functions. We write $x :: \tau$ for "x has type τ", $\tau_1 \Rightarrow \tau_2$ for the type of functions from τ_1 to τ_2, `'a` for a type variable, and $\lambda x.t$ for the function that maps x to t. A *datatype* declares a type by its constructors and comes with case distinction and structural induction. VIMP's commands `gcom` are one (Section 3.1). A *record* is a tuple with named fields. The analysis interface `dg_spec` is one (Section 6.3).

Every HOL function is total, so an ordinary recursive definition needs a termination proof. Otherwise a definition such as $f(n) = f(n) + 1$ would yield $0 = 1$ [6, §2.3.4]. Isabelle's function package can also define a recursive function without one. The result is still a total

HOL constant, but its defining equations are proved only on its *domain*, the arguments on which the recursion terminates, and theorems about it assume membership in that domain. The vendored solver is defined this way, with the domain `solve_dom`. Section 9.6 explains why Voblint's main theorem nevertheless needs no termination premise.

A *type class* collects operations and laws that an instance proves once for a carrier [63]. The solver's class `widening` fixes the operator ∇ and assumes that it bounds both operands:

```
(* vendor/td-verification/Warrowing.thy *)                                    Isabelle
class widening =
  fixes widen :: "'a :: order ⇒ 'a ⇒ 'a" (infixl "∇" 65)
  assumes widen_ge1: "a ≤ a ∇ b" and widen_ge2: "b ≤ a ∇ b"
```

A type has at most one instance of each class. The numeric domains state their algebra as classes (Chapter 5).

A *locale* fixes parameters and assumptions, and interpreting it proves the assumptions for an instance and yields its theorems [64]. Unlike a class, a locale can be interpreted several times for one type. Voblint states its soundness obligations as locale assumptions, so an analysis becomes sound by interpreting the locale. `sound_intersection`, for instance, fixes an intersection that keeps every value both operands admit and lies below both:

```
(* src/Abstract_Interpreter/Domain/Eval/Backward_Domain.thy *)                Isabelle
locale sound_intersection =
  fixes intersect :: "'a::numeric_domain => 'a => 'a"
  assumes intersect_sound[intro]:
    "n ∈ γ a ⟹ n ∈ γ b ⟹ n ∈ γ (intersect a b)"
    and intersect_reductive1[intro]: "intersect a b ≤ a"
    and intersect_reductive2[intro]: "intersect a b ≤ b"
```

An *inductive definition* is the least relation closed under its rules, and `inductive_set` defines a set the same way. Its *rule induction* proves a property of every element with one case per rule. The solver's verification defines `reach`, the unknowns on which a query x transitively depends, by two rules:

```
(* vendor/td-verification/Basics.thy *)                                       Isabelle
inductive_set reach for T σ x where
  base: "x ∈ reach T σ x"
| step: "y ∈ reach T σ x ⟹ z ∈ dep T σ y ⟹ z ∈ reach T σ x"
```

Source execution `pstep` (Figure 3.4), graph execution `cstep` (Figure 3.7), the valid activation traces `valid_activation_trace` and the reflexive–transitive closure $\to^*$ of a step relation are defined by rules as well, and a source run is written $\to_p^*$ (`psteps`).

A theorem declares its variables and their types after `fixes`, introduces abbreviations local to the statement after `defines`, lists its premises after `assumes` and states its conclusion after `shows`. The source-level theorem of Section 10.4 uses all four:

```
(* src/Executable_Surface/CLI/Analysis_Certified.thy *)                       Isabelle
theorem run_voblint_source_sound:
  fixes p :: imp_prog and s0 s :: store
  defines G_def: "𝒢 ≡ declared_global p"
      and Pi_def: "Π ≡ prog_table p"
      and g_def: "g ≡ prog_cfg p"
  assumes s0: "s0 ∈ cinit_stores 𝒢"
      and run: "𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)"
      and ans: "run_voblint config p = Analysed res"
  shows "(∃v. p, s0 ⊢ (residual, s, frs) ▷ v)
       ∧ (∀v. p, s0 ⊢ (residual, s, frs) ▷ v ⟶
                s ∈ 𝒞_{𝒢,g,cinit_stores 𝒢} v
              ∧ s ∈ ℛ_res v
              ∧ s ∈ 𝒱_res v)"
```

It fixes a program p :: `imp_prog` and two stores, abbreviates the program's globals classifier, procedure table and graph, and assumes an initial store, a source run $\rightarrow_p^*$ from it and an answer of the analyzer. It shows that the reached configuration is observed at some graph node v (written $\triangleright$, `at_node`, Section 3.6), and that at every such node the reached store lies in the collecting semantics, in the state the report gives there, and in the stores its verdicts admit. Inside a proposition, $A \Longrightarrow B$ is implication and $\exists$, $\wedge$, $\vee$ are the usual connectives. Subscripts, superscripts, and symbols such as $\vdash$ are notation for a constant applied to its arguments, and the text names the constant at first use.

A *quotient type* identifies representations up to an equivalence [65]. The executable abstract states of Chapter 9 are one: two finite representations are identified when they read back to the same function-valued state:

```
(* src/Abstract_Interpreter/Exec/State/Default_St_Base.thy *)                Isabelle
quotient_type 'a default_st =
  "('a::bot) default_st_rep" / "eq_default_st_rep"
  morphisms rep_default_st Abs_default_st
  by (rule equivp_eq_default_st_rep)
```

Its operations are lifted with `lift_definition`.

Code equations determine what code generation emits [47], and one `export_code` declaration emits the analyzer's OCaml from them, so the delivered analyzer runs the definitions the theorems are about. The generated code is *trusted*, not proved: no theorem states that the OCaml program computes what the Isabelle definitions compute, so its correctness rests on the code generator and the OCaml toolchain. The trust has a basis. The generator translates code equations that are proved from the definitions, apart from target-language adaptations such as the mapping of integers to OCaml's arbitrary-precision numbers (`code_printing`), which are trusted. Haftmann and Nipkow give its source and intermediate language a semantics and prove the translation of type classes correct on paper [47], and the regression corpus runs the generated analyzer against expected verdicts (Chapter 12). Theories are grouped into *sessions*, which Isabelle builds and checks as units, and which later chapters cite as the home of a construction. The control-flow graph, for instance, lives in `Voblint_CFG`, which builds on `Voblint_VIMP`.

The *trusted computing base* is the code that must be correct for a proof to be believed, normally only Isabelle's inference kernel. A *proof by evaluation* extends it. The method `eval` compiles a closed proposition to code, runs it, and accepts the result through the code generator's evaluation oracle. In 2025, a defect in normalization by evaluation, which also extends trust beyond the kernel, admitted a proof of `False` [66]. Theorems about fixed programs such as `nv_report` are proved this way.

PART II

What Must Be Over-Approximated

3. Programs and Control-Flow Graphs

Chapter 2 analyzed a single loop through its control-flow graph. The soundness theorem, however, should talk about the programs a user writes, and these have procedures and recursion. This chapter defines two views of such a program and connects them. The source semantics executes a program directly (Section 3.2). An analysis cannot generate its equations from it, because its constraint system needs a fixed, finite set of program points to attach abstract states to. The source semantics locates execution only by the command that still has to run, and under recursion these commands can nest arbitrarily deep, so they form no such set (Section 3.4.1). Voblint therefore compiles a program to a procedure-aware control-flow graph and generates its equations from the graph (Figure 3.1). If soundness were proved only for the graph, the compiler would stay outside the theorem, and a verdict would say nothing about the source program. This chapter therefore proves the first link of the soundness chain: every source run of a well-formed program is matched by a graph run that holds the same store (Section 3.6). The program of Figure 3.1, in which `main` calls `bump(5)` and `bump(4)`, is a running example throughout the remaining chapters.

```
1  fun bump(n) {                    VIMP
2    return n + 1;
3  }
4
5  fun main() {
6    a = bump(5);
7    b = bump(4);
8    __voblint_check(a == 6);
9    __voblint_check(b == 5);
10 }
```

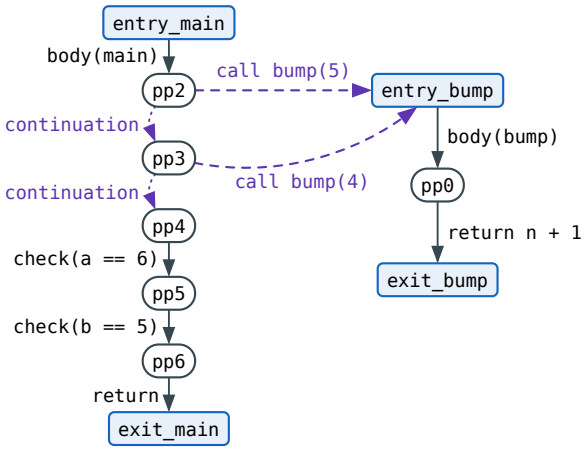


Figure 3.1: The running example and its procedure-aware graph, with node and edge labels as `voblint --dot` prints them. Solid arrows are local edges and dashed arrows are call edges. Dotted connectors are not edges. They show the continuation of a call edge, that is, the node where the caller resumes. Figure 8.1 shows the unknowns that Chapter 8 generates around the first call.

3.1. VIMP, and what it leaves out

VIMP (Figure 3.2) is the small imperative language with C-like syntax that the formalization analyzes. It is kept small because every construct needs a semantics, compiler clauses, and a transfer function proved sound in every abstract domain. It has unbounded integer variables, locals and globals, procedures with value parameters, direct calls with an optional

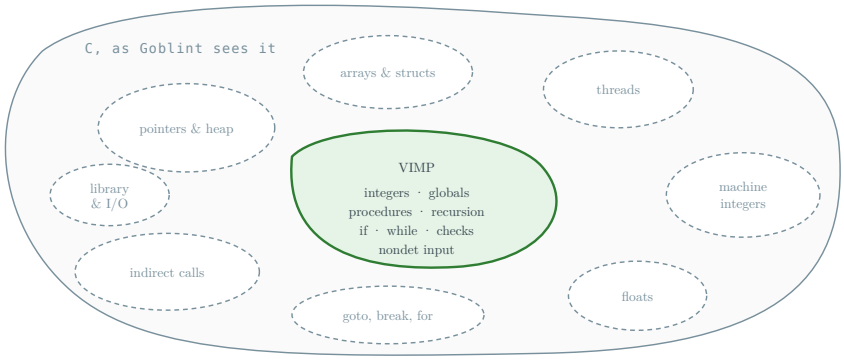


Figure 3.2: Constructs represented in VIMP relative to Goblint's C input, taken from the explainer page. Dashed regions are constructs that VIMP omits. Sizes and positions are schematic. The figure compares only syntax. Section 3.3 lists where shared constructs have different meanings.

result, recursion, assignment, sequencing, conditionals, `while` loops, nondeterministic input, and checks. It omits machine integers, pointers and the heap, arrays and structs, indirect calls, threads, floating point, `goto`, `break`, `continue`, and exceptions. Each of these would need its own preservation argument at every layer (Chapter 16 discusses the heap). Section 3.3 compares the fragment with C and with Goblint's input.

One of these omissions matters in later chapters. Without pointers, concrete memory is a store from variable names to integers. The non-relational analyses of Chapter 5 lift it pointwise to one abstract value per variable. Section 5.2.2 gives a carrier that is not pointwise.

Expressions are integer-valued. As in C11, comparisons and logical operators yield 0 or 1 (§6.5.8p6, §6.5.9p3, §6.5.3.3p5, §6.5.13p3, §6.5.14p3), and a condition holds when it is nonzero (`truthy`, §6.8.4.1p2, §6.8.5p4).

Definition 3.1 (Expressions, `datatype exp`)**.**

$$\begin{aligned} e ::= {} & n \mid x \mid e + e \mid e - e \mid e * e \mid e \,/\, e \mid e \,\%\, e \\ & \mid e < e \mid e <= e \mid e > e \mid e >= e \mid e \,!= e \mid e == e \\ & \mid\, !e \mid e \,\&\&\, e \mid e \,||\, e \end{aligned}$$

Each operator has its own constructor, because an abstract domain chooses its transformer by the operator, and the executable analyzer has to make this choice by inspecting the syntax. The two AFP languages with procedures that we considered, IMP2 and Simpl, do not allow this. IMP2 [67] represents a binary operator by a HOL function of type `int ⇒ int ⇒ int`. Concrete evaluation can apply such a function, but executable code cannot compare it with another one, so it cannot tell `+` from `*`. IMP2 also matches Voblint's programs poorly in other respects. Its semantics is deterministic, so it has no input, and its call command takes no arguments and returns no value, so parameters and results pass through variables set by generated code (Chapter 15). Schirmer's Simpl [68] is more expressive, with nondeterministic specifications and a derived call command that passes parameters, but its basic commands and conditions are likewise HOL functions and sets of states. Both languages target program verification, where semantic objects are convenient,

whereas an analyzer needs syntax that it can inspect. VIMP is therefore defined from scratch, with such a syntax, which code generation can also export.

An expression is evaluated in a *store* s of type `store`, a total function `vname` $\Rightarrow$ `int` from variable names to integers. Evaluation `aval`, written $[\![\texttt{e}]\!]_{\texttt{e}}\ \texttt{s}$, is a function defined by one equation per constructor, for example

$$[\![n]\!]_e\, s = n, \quad [\![x]\!]_e\, s = s(x), \quad [\![e_1 \,/\, e_2]\!]_e\, s = \texttt{c_div}\ \big([\![e_1]\!]_e\, s, [\![e_2]\!]_e\, s\big),$$

$$[\![e_1 < e_2]\!]_e\, s = \begin{cases} 1 \text{ if } [\![e_1]\!]_e\, s < [\![e_2]\!]_e\, s \\ 0 \text{ otherwise.} \end{cases}$$

Every expression has a value, because VIMP fixes the results of division and remainder by zero (`c_div`, `c_mod`, Section 3.3). It also has no effects, so a transfer function may evaluate a guard again, as the backward filtering of Chapter 5 does. Input enters only through the statement `x = __voblint_nondet_int();` and never inside an expression. This keeps `aval` a function.

Commands are built from expressions. Besides the structured statements, there are calls, which may store a result, and two markers that appear only during execution.

Definition 3.2 (Commands, `datatype gcom`)**.**

$$c ::= \textbf{skip} \mid x := e \mid \textbf{check}\ (e) \mid c;\, c \mid \textbf{if}\ (e)\{c\}\ \textbf{else}\ \{c\} \mid \textbf{while}\ (e)\{c\}$$
$$\mid [x :=]\, p(e, ..., e) \mid \textbf{return}\ e? \mid \text{Restore} \mid \text{Unwind}$$

Every constructor except sequencing also carries an *annotation*, whose type is a parameter of `gcom`. The commands of a parsed program carry the unit annotation and have type `com`. The grammar above omits annotations, which Figure 3.4 writes as i.

In concrete syntax, an assignment is written `x = e;` and a check `__voblint_check(e);`. The parser lowers `true`, `false`, and unary minus to the expression grammar above. A check states a condition for the analyzer to decide (Section 10.2) and does not influence the execution. It behaves like **skip** whatever value e has, so it neither stops the run nor refines the store. Its constructor also carries the source position, which only the report uses, to name the check.

Calls come in two kinds. A call of a declared procedure starts an *activation*, one execution of the procedure body from its call to its return. VIMP also has three built-in *library calls*, `__voblint_nondet_int`, `min`, and `max`, which a program calls but does not declare. The fixed table `special_table` recognizes their names, and `special_result` gives the values they may return: any integer for `__voblint_nondet_int`, and the minimum or maximum of the arguments for `min` and `max`. A library call finishes in one step and stores its value, without an activation or a frame (Section 3.2). The remaining constructors, Restore and Unwind, are not source syntax (`source_com` excludes them). They appear only during execution, where Section 3.2 uses them for calls and returns.

A program (`imp_prog`) is a list of procedure declarations (`proc_rep`), each with formal parameters and a body, plus a list of global variables (`declared_global_vars`). Its procedure table (`proc_table`) looks up a name in that list, and the classifier `declared_global` tests membership in the globals. The entry procedure is the declaration named `main`, and `main_body` is its body. Figure 3.3 shows the running example in this form. The formal development starts from this parsed program, and the parser lies outside the theorem (Section 12.5).

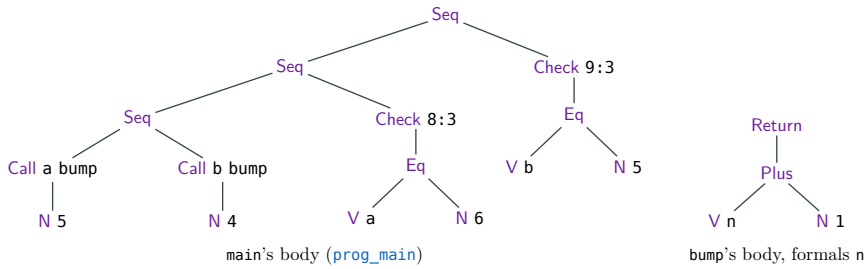


Figure 3.3: The running example as the parser returns it. It consists of the `main` body and the one callee declaration (`proc_rep`) and declares no globals. The playground passes this value to `run_voblint` and shows it in the panel for the call and answer of `run_voblint`. For this program the panel is reached via the `VIMP` link of Figure 3.1. A Call names its destination and callee, and a Check names its source position.

Not every parsed program can run meaningfully. The theorems must exclude programs whose calls cannot step or whose results are undefined, and the *source contract* `wf_source_program` does this. It requires that

- every call names either a declared procedure with matching arity or a library function with suitable arguments and a destination,
- formals are distinct valid locals, and `main` is declared, takes no arguments and contains no **return** (Section 3.2),
- bodies contain no runtime markers Restore or Unwind,
- no declared name is a library name,
- the result variable `ret_var` is reserved, so it is not global, no source expression reads it, and no assignment or call destination writes it,
- a call that stores a result names a *value-providing* callee, one that ends every completed execution of its body with a **return** of a value.

A body *falls through* when execution reaches its end without a **return**. Value-providing is a syntactic condition (`value_providing`). It requires that the body cannot fall through, that it cannot reach a **return** without a value, and that it contains a **return** with a value. So every path through the callee that completes returns a value, although the callee may still diverge. C11 makes using the value of a call that falls through undefined (§6.9.1p12). Without the condition, VIMP would silently store the 0 that the reset on entry left in `ret_var`.

3.2. Executing a source program thy

An *operational semantics* defines the meaning of a program by how it executes. A *small-step* operational semantics does so one step at a time. A step relation leads from one configuration to the next, and a run is a sequence of steps [69, §3.2]. VIMP's semantics `pstep` is of this kind. We call it the *source semantics*, to distinguish it from the graph semantics of Section 3.5.2, which is also a small-step semantics.

Without calls, a configuration is a pair (c, s) of the command that remains to run and the store, as in the semantics of IMP by Nipkow and Klein [6, §7.3]. An assignment updates the store and leaves **skip**, and a check leaves the store unchanged. A sequence steps its first command until it is **skip**, a conditional selects a branch by `truthy`, and a loop unfolds into

a conditional that runs the body and then the loop again. After each step the remaining command describes the remainder of the execution [69, §3.2], which we call a *residual*.

Calls raise two questions. While a callee runs, the caller's remaining work has to wait somewhere, and a **return** has to skip the rest of the callee's body. VIMP keeps the waiting work inside the residual command and saves only data on a stack. A call continues with the callee's body followed by Restore, a marker that exists only at run time and hands control back to the caller once the body falls through. A **return**, possibly before the end of the body, becomes the marker Unwind, which discards the commands after it until it meets the Restore of its call. Each suspended activation keeps its data in a *frame*.

Definition 3.3 (Source execution, `inductive pstep`)**.** A source configuration is a triple (c, s, frs) of the command that remains to run, a store s, and a list frs of `frame` values for the suspended activations. Each frame records the caller's store and the variable, if any, that receives the result. The step relation `pstep` relates a configuration to its successor.

```
(* src/Program_Model/VIMP/VIMP_Proc.thy *)                                    Isabelle
inductive
  pstep :: "(vname ⇒ bool) ⇒ 'a ptable ⇒ 'a acom × store × frame list
                          ⇒ 'a acom × store × frame list ⇒ bool"
    ("(_,_ ⊢/ _ →p/ _)" [51, 51, 51, 51] 50)
  for G :: "vname ⇒ bool" and Π :: "'a ptable"
where
  Assign:  "G, Π ⊢ (Assign i x a, s, frs) →p (SKIP i, s(x := ⟦a⟧e s), frs)"
| Check:   "G, Π ⊢ (Check i l c, s, frs) →p (SKIP i, s, frs)"
| Seq1:    "G, Π ⊢ (Seq (SKIP i) c2, s, frs) →p (c2, s, frs)"
| Seq2:    "G, Π ⊢ (c1, s, frs) →p (c1', s', frs')
             ⟹ G, Π ⊢ (Seq c1 c2, s, frs) →p (Seq c1' c2, s', frs')"
| IfTrue:  "truthy (⟦b⟧e s) ⟹ G, Π ⊢ (If i b c1 c2, s, frs) →p (c1, s, frs)"
| IfFalse: "¬ truthy (⟦b⟧e s) ⟹ G, Π ⊢ (If i b c1 c2, s, frs) →p (c2, s, frs)"
| While:   "G, Π ⊢ (While i b c, s, frs)
                       →p (If i b (Seq c (While i b c)) (SKIP i), s, frs)"
| Call:    "Π p = Some decl
             ⟹ length actuals = length (formals decl)
             ⟹ distinct (formals decl)
             ⟹ vals = map (λe. ⟦e⟧e s) actuals
             ⟹ callee = bind_formals (formals decl) vals (enter_state G s)
             ⟹ G, Π ⊢ (Call i dst p actuals, s, frs)
                 →p (Seq (body decl) (Restore i),
                  callee,
                  Frame s dst # frs)"
| Special: "special_table p = Some desc
             ⟹ classify_special desc actuals = Some sc
             ⟹ special_result sc s v
             ⟹ G, Π ⊢ (Call i (Some x) p actuals, s, frs) →p (SKIP i, s(x := v), frs)"
| RestoreStep:
    "G, Π ⊢ (Restore i, s, Frame fr dst # frs)
       →p (SKIP i, combine_assign dst (s ret_var) (combine_env G fr s), frs)"
| ReturnSome:
    "G, Π ⊢ (Return i (Some e), s, frs)
       →p (Unwind i, s(ret_var := ⟦e⟧e s), frs)"
| ReturnNone:
    "G, Π ⊢ (Return i None, s, frs) →p (Unwind i, s, frs)"
| UnwindDead:
    "¬ is_Restore c2
     ⟹ G, Π ⊢ (Seq (Unwind i) c2, s, frs) →p (Unwind i, s, frs)"
| UnwindAct:
    "G, Π ⊢ (Seq (Unwind j) (Restore i), s, Frame fr dst # frs)
       →p (SKIP i, combine_assign dst (s ret_var) (combine_env G fr s), frs)"
```

Figure 3.4: The declaration of `pstep`, lifted from the theory.

A classifier G `::` `vname` ⇒ `bool`, separate from the store, says which names are global, because a call resets only the locals. For a program it is `declared_global`. The relation is written $G,\ \Pi \vdash \kappa \rightarrow_p \kappa'$ for configurations κ, κ', and a procedure table Π. Its closure

is $\rightarrow_p^*$, and Figure 3.4 lists its rules. No rule inspects an annotation. The rules only copy annotations, from the rewritten command or from the procedure table. The explainer page animates these steps.

A call evaluates the actuals in the caller's store, resets the locals, binds the formals (`enter_state`, `bind_formals`), and pushes a frame. A `return e` writes the value of e to `ret_var`, and popping the frame merges the caller's locals, the callee's globals, and the result. These operations select by name and are polymorphic in the value type (`enter_binding`, `combine_env`, `combine_assign`). Therefore the non-relational analyses reuse the concrete definitions at their abstract state types, with $\top$ in place of the reset value 0.

We model the root activation without a Restore, since `main` has no caller to return to, and a bare Unwind has no step (`pstep_Unwind_stuck`). This is why the source contract rejects **return** in `main` (`no_return`). A source run starts from the configuration (`main_body` $\Pi, s_0, []$), with an initial store s_0 in `cinit_stores` (Section 3.3).

Just as Nipkow and Klein show that well-typed programs do not get stuck [6, §9.1.2], we show that well-formed VIMP programs do not, since claims about the points after a stuck run would hold vacuously. For a program satisfying `wf_source_program`, every reachable configuration has either finished, as **skip** with an empty frame stack, or can step:

```
(* src/Program_Model/Compile/Simulation/Source_Progress.thy *)                Isabelle
theorem source_progress:
  assumes wf: "wf_source_program 𝒢 Π"
      and run: "𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (c, s, frs)"
  shows "(is_SKIP c ∧ frs = []) ∨ (∃cfg'. 𝒢, Π ⊢ (c, s, frs) →ₚ cfg')"
```

The source contract rules out every reason a call could fail to step, and every library call has a result (`special_result_ex`).

3.3. Scope and adequacy of the source semantics

Every source-level guarantee in this thesis is interpreted against the executions of `pstep`, the reference semantics of VIMP, so no guarantee can show that `pstep` describes the intended language. This can only be argued, and this section collects the argument. Table 3.1 compares VIMP with C11 [7] (committee draft N1570) wherever both languages have a construct but model it differently.

	VIMP	C11
integers	unbounded	signed overflow undefined (§6.5p5) unsigned arithmetic wraps (§6.2.5p9)
`a / 0`, `a % 0`	0 and a	undefined (§6.5.5p5)
`/`, `%`, $b \neq 0$	truncated; $(a \;/\; b) \cdot b + a\%b = a$	the same where $a \;/\; b$ is representable (§6.5.5p6)
`&&`, `||`	both operands evaluated	second operand only if needed (§6.5.13p4, §6.5.14p4)
initial values	globals 0, `main`'s locals arbitrary	static 0, automatic indeterminate (§6.7.9p10)
callee locals	0 on every entry	indeterminate (§6.7.9p10)
checks	never stop the run, never refine	`assert` aborts on 0 unless `NDEBUG` (§7.2p1, §7.2.1.1)
`min`, `max`	library calls (`special_table`)	no counterpart
`main`	no explicit `return`	`return` gives the exit status (§5.1.2.2.3)

Table 3.1: Constructs that both languages have but model differently. The VIMP rows restate `c_div`, `c_mod`, `aval`, `cinit_stores`, `enter_state` and `pstep`.

Each row is a place where a statement true of a VIMP program does not necessarily hold for the corresponding C program. Unbounded integers and defined division by zero can flip a verdict. After `z = 5 / 0;` the check `z == 0` holds in VIMP, so a `PROVED` verdict on a program that divides by zero says nothing about C. The analyzer therefore reports a possible zero divisor as a separate diagnostic, and its absence at a reached node proves the divisor nonzero there (`run_voblint_arithmetic_safe`, Section 10.5). Division and remainder truncate toward zero as in C11. Goblint once got the remainder wrong, and issue 1156 [12] reports that it claimed `c % 2 == 1` for $c \in \{-5, -7\}$ (Section 13.2). Evaluating both operands of `&&` is harmless, because expressions are total and have no effects. It gives the short-circuit value. Only the division diagnostic differs. It warns about the divisor in `x != 0 && 10 / x > 1`.

The initial values that VIMP fixes are ones C leaves open. Every source-level theorem assumes an initial store from `cinit_stores`, with globals 0 as for C's static objects and the locals of `main` arbitrary. VIMP does not model C's indeterminate values, some reads of which are undefined (§6.3.2.1p2). Callee locals start at 0 (`enter_state`), but no shipped analysis relies on this, because every abstract entry describes callee locals by $\top$ (`enter_binding` for the non-relational analyses). A check `y == 0` on an unassigned callee local `y` is therefore `UNKNOWN`. Checks never prune executions, so the verdict for one check does not depend on an earlier one. Goblint's `__goblint_check` behaves the same way, and its library table classifies it as checked and not used for refinement (`libraryFunctions.ml`).

A fragment of Goblint's input. Goblint analyzes C after its CIL front end [70] has normalized it, so that expressions have no side effects and a call is a separate instruction with an optional destination (`cil.mli` at the revision Goblint pins). VIMP makes the same choice, and every edge kind of Goblint's CFG, except inline assembly and in-place declarations, has a VIMP counterpart (Table 3.2). No translation from C to VIMP is formalized.

No second semantics. No external semantics cross-checks `pstep`. For C, Isabelle offers the C parser of AutoCorres2 [71], which translates a subset of C into Simpl programs [68], and Coq has the Clight semantics of CompCert [32]. Relating VIMP to either of them, or to IMP2 or Simpl directly, would need a translation and its own adequacy argument. Establishing such a correspondence is left as future work.

3.4. The procedure-aware control-flow graph

3.4.1. Why a graph

An analysis attaches abstract states to program points and transfer functions to transitions. `pstep` offers neither in a usable form. Its only notion of location is the command that remains to run. Without recursion, only finitely many distinct residual commands arise, because a loop may run arbitrarily often, but its unfolding revisits the same command forms. Recursion is different, because a recursive call can nest another Restore wrapper at every call depth. Residual commands therefore do not form the finite, fixed set of locations that Voblint needs. An assignment is also not a transition between fixed program points. Inside a sequence it steps through the rule `pstep.Seq2`, which rebuilds the surrounding residual command. So the same assignment is executed by a different derivation inside every surrounding command. Finally, nothing names the entry of a procedure, its result, or

the point where a caller resumes. The schematic control-flow graph of Figure 2.1 solves the first two problems. It has finitely many nodes and one transfer per edge. The procedure-aware graph then solves the third. It adds an entry node and a result node per procedure, and call edges that record their continuations. Each gives the analysis a place for one fact about a call. The entry node holds the state a callee starts from, the result node the state it returns with, and the continuation names the node where the caller resumes. A call can therefore be analyzed by connecting the call site, the callee's entry and result nodes, and the continuation, instead of by following the callee's body (Chapter 8).

Introducing a graph is a design choice. A *syntax-directed* analyzer, such as that of Nipkow and Klein [6, Ch. 13], computes its result by recursion over the command structure and needs no graph. Voblint instead follows Goblint and is *constraint-based* [43]: it turns the program into a system of inequalities over unknowns indexed by the graph's nodes and leaves the solution to a generic solver that knows nothing about programs. The solver needs no knowledge of calls, because a call contributes right-hand sides and side effects to the same system (Chapter 8). The choice costs the compiler and simulation proof of this chapter and the coverage layer of Chapter 4, which says what the inequalities mean. In return, the argument about programs ends at a solver-independent certificate, a post-solution on the solved set (Section 9.3). Voblint does not implement the solver itself. It uses the verified top-down solver of Tilscher et al. [4], which Voblint includes as a submodule with a few changes (Section 14.3). Chapter 15 returns to the comparison.

3.4.2. Nodes and edges thy

Definition 3.4 (Control-flow graph, `record cfg`)**.** A graph g is a record of type `cfg`:

```
(* src/Program_Model/CFG/CFG_Def.thy *)                                    Isabelle
record cfg =
  intra     :: "(cfg_node × edge_action × cfg_node) set"
  calls     :: "(cfg_node × call_action × cfg_node × cfg_node) set"
  cfg_entry :: cfg_node
  checks    :: "(cfg_node × exp) set"
```

`intra` holds the *local edges* (u, a, v), each labeled by an `edge_action` a. `calls` holds the *call edges* $(u, ca, \mathsf{FunctionEntry}\, q, k)$. Such an edge consists of the call site u, the call information ca, the entry node of the callee (by `wf_cfg`), and the node k where the caller resumes. Nodes (`cfg_node`) are $\mathsf{Statement}\, n$, $\mathsf{FunctionEntry}\, p$ or $\mathsf{FunctionResult}\, p$. `cfg_entry` is the root node. `checks` pairs the node of each check with its condition. For a compiled graph it is computed from the `EA_Check` edges, so it cannot drift from them, and the report classifies the same edges (`classify_checks`).

The call information $\mathsf{CallEdge}\, dst\ formals\ args$ (`call_action`) copies the callee's formals onto the edge, so the graph semantics binds actuals to formals without looking up the procedure table. VIMP has no function pointers (Section 3.1), so every call names its callee, and the compiler can fix the target of each call edge. Supporting indirect calls would require discovering call targets during the analysis, which is left for future work. The node kinds match Goblint's (`node0.ml`), which also has statement and function-entry nodes and calls our FunctionResult node `Function`.

The graph keeps local edges and call edges apart, in its fields `intra` and `calls`, because they execute differently. A local edge changes the store within one activation, while a call edge starts a new one. No edge action denotes a call, so a call can never be taken as a

VIMP	transition	Goblint
`EA_Assign`	assignment	`Assign`
`EA_Assume`	guard holds	`Test`
`EA_AssumeNot`	guard fails	`Test`
`EA_Special`	`min`, `max`, nondeterministic input	`Proc`
`EA_Check`	check, identity on the store	`Proc`
`EA_Body`	leave a procedure entry, identity	`Entry`
`EA_Ret`	return, writes `ret_var`	`Ret`
`EA_Nop`	**skip**, runtime markers, unusable library calls	`Skip`
call edge	call of a declared procedure	`Proc`
none	inline assembly, in-place declaration	`ASM`, `VDecl`

Table 3.2: Transitions of the graph and their counterparts among the edges of Goblint's CFG at the pinned revision (`edge.ml`). All rows except the last two are `edge_action` constructors on `intra` edges. Goblint routes library calls and `__goblint_check` through `Proc`. It skips empty statements and uses `Skip` only for the self-loop of an empty loop.

local step, and each rule of the trace semantics in Chapter 4 reads at most one of the two relations.

Returns, in contrast, are ordinary local edges. A **return** in p leads to the result node FunctionResult p, which all callers of p share, so recursion needs no copies of nodes. A shared result node cannot know where to continue, and in a compiled graph no edge leaves it. The call edge records that place instead, as its continuation k. After a return, execution resumes at the continuation saved in the frame, and an analysis connects the callee's result to the correct caller through the call edge. The matching caller is explicit in the graph, so no separate return-to-call matching is needed. A completed program ends at FunctionResult `main` (`cfg_exit`), so there is no separate global exit node. Figure 3.5 shows these properties on a recursive program.

```
1  fun dec(x) {                    VIMP ↗
2    return x - 1;
3  }
4
5  fun sum(n) {
6    if (n < 1) {
7      return 0;
8    }
9    m = dec(n);
10   r = sum(m);
11   return r + n;
12 }
13
14 fun main() {
15   x = sum(2);
16   __voblint_check(x == 3);
17 }
```

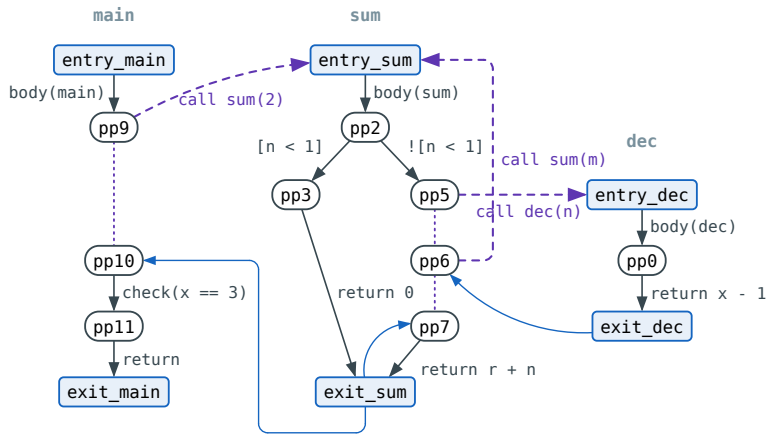


Figure 3.5: A recursive program and its compiled graph, labeled as `voblint --dot` prints it. Here pp n is Statement n, and entry_p, exit_p are FunctionEntry p, FunctionResult p. As in Figure 3.1, solid gray arrows are `intra` edges, and dashed purple arrows are `calls` tuples. Dotted connectors and thin blue arrows are not edges. Dotted connectors show the continuation of a call tuple. Thin blue arrows show where execution continues after a result node. Each procedure has one copy of its nodes, whatever the recursion depth. Reserved node numbers without edges, such as pp 4, are not nodes of the graph.

3.5. Compilation

The compiler turns a VIMP program into a procedure-aware graph. Its design decides which program points exist, and the simulation of Section 3.6 relies on how it lays out the code of each command.

3.5.1. Compiling a command thy

To emit the edges of a command, a compiler has to know where control goes after it. Compilers commonly pass this target down, so that each statement receives the label of the code that follows it, the inherited attribute S.next of the dragon book [55, §6.6.3]. Voblint's compiler works the same way and is therefore called *continuation-passing*. Besides the command, it takes the node k at which control continues afterward, and it emits the command's edges so that they end at k (`compile` for commands, `compile_proc` and `compile_prog` for procedures and programs). Schematically, writing $\mathrm{compile}(c, k)$ for the edges of c against the continuation k, and n and h for fresh nodes,

$$\begin{aligned}
\mathrm{compile}(c_1;\ c_2, k) &= \mathrm{compile}(c_1, \mathrm{entry}(c_2)) \cup \mathrm{compile}(c_2, k),\\
\mathrm{compile}(\textbf{if}\ (e)\{c_1\}\ \textbf{else}\ \{c_2\}, k) &= \left\{ n \xrightarrow{e} \mathrm{entry}(c_1), n \xrightarrow{\neg e} \mathrm{entry}(c_2) \right\}\\
&\quad \cup \mathrm{compile}(c_1, k) \cup \mathrm{compile}(c_2, k),\\
\mathrm{compile}(\textbf{while}\ (e)\{c\}, k) &= \left\{ h \xrightarrow{e} \mathrm{entry}(c), h \xrightarrow{\neg e} k \right\} \cup \mathrm{compile}(c, h),\\
\mathrm{compile}(\textbf{return}\ e, k) &= \left\{ n \xrightarrow{\textbf{return}\ e} \mathsf{FunctionResult}\, p \right\}.
\end{aligned}$$

The alternative would give every fragment an exit node of its own. Sequencing two fragments would then need a no-op edge from the exit of the first to the entry of the second, so a sequence of m commands would produce $m - 1$ such edges between nodes that denote the same program point. With continuations, a node is a program point between two transfers, as in Goblint's CFG, and a **return** simply ignores its continuation. A conditional needs no join node, because both branches end at the same k. A loop compiles its body against its own head, so the compiled counting loop of Figure 2.1 has no node t and no edge $t \to h$. A **skip** branch adds no node, and its guard edge leads directly to k. To compile c_1 against the entry of c_2, the compiler must know how many nodes c_1 allocates (`csize`, `compile_next_id`).

Calls and returns leave the current fragment. A call to a declared procedure becomes a call edge from the call site to the callee's entry, with k as its continuation, and adds no local edge. A **return** becomes an `EA_Ret` edge to the result node of its procedure. A body that can reach its end without a **return** needs such an edge as well. `compile_proc` compiles the body against one extra node, from which a return edge without a value leads to the result node (Figure 3.6). It emits this edge for every body, so in a body that always returns, the extra node and its edge are dead code. Section 9.3.2 explains why the compiler keeps them. Every edge into FunctionResult p is therefore a return edge.

These rules give every compiled graph a fixed shape. Call edges always enter a procedure entry, no local edge enters one, and every return edge ends at the result node of its own procedure. The structural contract `wf_cfg` collects these properties, and every compiled graph satisfies it without premises (`compile_prog_wf`). The graph's entry is FunctionEntry `main` (`cfg_entry_compile_prog`), and its edge sets are finite (`compile_prog_finite`). Later

```
(* src/Program_Model/Compile/VIMP_Proc_to_CFG.thy *)                    Isabelle
definition compile_proc ::
  "'a ptable ⇒ pname ⇒ 'a gproc_decl ⇒ nat
   ⇒ nat × (cfg_node × edge_action × cfg_node) set
       × (cfg_node × call_action × cfg_node × cfg_node) set"
where
  "compile_proc Π p decl n =
     (let r = n + csize (body decl);
          (n', ben, E, K) = compile Π p (body decl) (Statement r) n
      in (Suc r,
          insert (FunctionEntry p, EA_Body p, ben)
            (insert (Statement r, EA_Ret None p, FunctionResult p) E),
          K))"
```

Figure 3.6: The declaration of `compile_proc`, lifted from the theory.

chapters rely on this shape to route calls (Chapter 8) and to enumerate edges in the executable analyzer.

The compiler reads the program as a table Π and a list ps of the declared procedures other than `main`. Its input contract `wf_compile_input` $\mathcal{G}$ Π ps is `wf_source_program` plus the condition that ps lists exactly those procedures, without repetition.

3.5.2. Graph execution thy

Definition 3.5 (Graph execution, `inductive cstep`)**.** A graph configuration (`cconf`) is a triple of a node, a store, and a stack of suspended callers (`cframe`). Each entry records where to resume, which variable receives the result, and the caller's store. The step relation `cstep` is written $\mathcal{G}, g \vdash \kappa \to_c \kappa'$, and $\to_c^*$ is its closure.

```
(* src/Program_Model/CFG/CFG_Exec.thy *)                                Isabelle
inductive cstep :: "(vname ⇒ bool) ⇒ cfg ⇒ cconf ⇒ cconf ⇒ bool"
    ("(_,_ ⊢/ _ →c/ _)" [51, 51, 51, 51] 50) for 𝒢 and g where
  Intra [intro]:
    "(u, a, v) ∈ intra g ⟹ s' ∈ edge_step a s ⟹
     𝒢, g ⊢ (u, s, stk) →c (v, s', stk)"
| Call [intro]:
    "(u, CallEdge dst pars actuals, FunctionEntry q, cont) ∈ calls g ⟹
     𝒢, g ⊢ (u, s, stk)
       →c (FunctionEntry q, call_enter 𝒢 (CallEdge dst pars actuals) s, (cont, dst, s) # stk)"
| Return [intro]:
    "𝒢, g ⊢ (FunctionResult q, t, (cont, dst, caller) # stk)
       →c (cont, combine_collect 𝒢 dst caller t, stk)"
```

Figure 3.7: The declaration of `cstep`, lifted from the theory.

Figure 3.7 lists the rules, one per kind of step. A local edge applies its transfer `edge_step`, a call edge enters the callee and pushes a frame, and a result node pops the top frame. In the running example (Figure 3.1), a run of `main` reaches the call site of `bump(5)`, takes the call edge to the entry of `bump` with $n = 5$, and pushes a frame that names the continuation. It then follows the local edges of `bump` to its result node, where the frame is popped. Execution resumes at the continuation, and `a` holds 6. The entry and exit transfers are built from the same functions that `pstep` uses (Figure 3.4). `call_enter` binds the formals in `enter_state` of the caller's store, and `combine_collect` applies `combine_assign` to `combine_env`. For this reason both executions can hold the same store. `cstep`, the trace semantics and the coverage contract of Chapter 4 are defined for an arbitrary graph, not only for compiled ones.

3.6. Relating the two executions

So far `pstep` and `cstep` are two separate definitions. One describes source programs, the other arbitrary graphs, and nothing yet says that a compiled graph behaves like its program. The soundness theorem is stated over `pstep`, but the analysis solves equations generated from the graph, so every source run must be matched by a run of the compiled graph that holds the same store at corresponding points. We prove this with a *forward simulation*, a relation between source and graph configurations that every source step preserves.

Two differences make the relation nontrivial. The source stores the remaining work of a suspended caller inside the running command, behind a Restore marker. The graph, in contrast, stores it on the frame stack as the continuation node k. The two also step at different places. Unfolding a loop into a conditional, discarding a finished **skip**, and propagating Unwind past the commands it skips have no corresponding graph step. As a result, the graph answers one source step with zero or more steps. We prove only this direction, because soundness needs no converse. The analysis is proved to cover every run of the graph from its entry (Chapter 4), and every source run is matched by such a graph run.

Definition 3.6 (Simulation relation, `inductive csim`)**.** `csim`, written `Π, g, p ⊢ (c, s, frs) ≈ (v, s, stk)`, relates a source configuration to a graph configuration holding the same store, where p is the procedure of the outermost activation, `main` for a run of the program.

```
(* src/Program_Model/Compile/Simulation/Simulation_Relation.thy *)                    Isabelle
inductive csim :: "'a ptable ⇒ cfg ⇒ pname ⇒ 'a acom × store × frame list
                   ⇒ cfg_node × store × cframe list ⇒ bool"
    ("(_,_,_ ⊢/ _ ≈/ _)" [51, 51, 51, 51, 51] 50) for Π g where
  Base:
    "control_at Π p body_p k n c v ⟹ compiled_at Π g p body_p k n ⟹
     Π, g, p ⊢ (c, s, []) ≈ (v, s, [])"
| Nested:
    "Π, g, q ⊢ (inner, s, frs) ≈ (v, s, stk) ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after (Seq inner (Restore i)) afters, s, frs @ [Frame caller dst])
       ≈ (v, s, stk @ [(cont, dst, caller)])"
| Returning:
    "pop_ready w ⟹ pop_ann w = Some i ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after w afters, callee, [Frame caller dst])
       ≈ (FunctionResult q, callee, [(cont, dst, caller)])"
```

Figure 3.8: The declaration of `csim`, lifted from the theory.

Figure 3.8 lists the three rules. Each rule locates one activation with two predicates. `control_at` $\Pi\ p\ b\ k\ n\ r\ v$ says that the residual r of the body b, compiled at offset n with continuation node k, is at node v. `compiled_at` $\Pi\ g\ p\ b\ k\ n$ says that b is the body of procedure p and that its compilation at that offset lies in g. The rules differ in the stacks. Base relates a single activation with both stacks empty. Nested adds one suspended caller beneath both stacks. On the source side the caller is the wrapper Restore around the running command, followed by the caller's pending commands *afters*, which `seq_after` sequences after it. On the graph side it is a frame, and the caller's remaining work `seq_after` **skip** *afters* lies at the frame's continuation node *cont*. Both suspended-caller rules also locate the call that opened the activation. A second `control_at` premise

places `seq_after` (Call q...) *afters* in the caller's body. This premise names the callee q, whose body the inner activation runs. In Nested, the index changes from q to the caller's procedure. Returning covers the moment after a callee finishes. The graph has reached FunctionResult q of that callee, while the source still holds the Restore of the activation, possibly behind an Unwind that is propagating toward it (`pop_ready`). The Restore carries the annotation of its call (`pop_ann`), which is the annotation that the call-site premise refers to.

Equal commands can occur at different places, so on a program as written the relation does not determine the node. In `if (b) { x = 1; } else { x = 1; }`, the residual `x = 1` is related to both branch nodes. The proof therefore runs on the program's *elaboration* (`elab_pt`), which annotates each statement with the nodes the compiler emits for it and compiles to the same graph (`compile_prog_elab_pt`). On it, `point` reads the related node off the residual's annotation (`csim_point_compile_prog`). Every run of a well-formed program is the *erasure* (`erase`) of a run of its elaboration (`psteps_lift`). For a program p, we write $p, s_0 \vdash (c, s, \mathrm{frs}) \rhd v$ (`at_node`) and say the configuration is *observed* at v when such a lifted run from s_0 reaches it at a command that names v and the simulation relates it to v. A configuration reachable along two paths can be observed at two nodes.

Theorem 3.7 (One step is matched, `csim_step`)**.** *If a source configuration and a graph configuration are related by `csim` and the source takes one step, the graph takes zero or more steps to a configuration related to the new source configuration.*

```
(* src/Program_Model/Compile/Simulation/Simulation_Preservation.thy *)
theorem csim_step:
  assumes SIM: "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)"
      and PC: "procs_embedded Π g"
      and WF: "return_safe c"
      and STEP: "𝒢, Π ⊢ (c, s, frs) →ₚ src'"
  shows "∃cfg'. 𝒢, g ⊢ (v, t, stk) →c* cfg' ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
```

Isabelle

The theorem has two premises beyond the related pair and the source step. `procs_embedded` Π g says that every declared body is source syntax under a non-library name and lies in g with its entry and return edges, so no source call enters a body missing from the graph. `return_safe` c says that the running command returns only inside an activation that a call opened, which is the runtime form of the rule that `main` contains no **return** (Section 3.2). Both premises hold for every well-formed program (`procs_embedded_compile_prog`, `wf_compile_input_return_safe`), and every source step preserves the second (`return_safe_pstep`). By induction on the run, `csim_star` extends the theorem to finite source runs, as in CompCert's simulation diagrams [32, §4.3].

The initial source configuration is related to the node after the `EA_Body` edge of `main` (`compile_prog_main_base`). Every finite source run of a well-formed program is therefore matched by a graph run from the entry that reaches corresponding nodes with the same store. This gives the first inclusion of the soundness chain of Figure 1.3, so an analysis that covers all graph runs covers all source runs. The executable analyzer rejects ill-formed programs (`wf_program_compile_input_exec`), so the end-to-end theorem needs no such premise. A context-sensitive analysis needs more structure than these flat runs offer. Chapter 4 therefore regroups graph runs into activation traces (`source_run_has_activation_trace`) and defines on them the coverage obligations used to establish soundness.

4. Traces and the Coverage Contract

Chapter 3 matched every source run with a run of the compiled graph. For an analysis that computes one abstract state per node, this suffices. A context-sensitive analysis, however, computes a separate state for each node and calling context, so to justify its result we need to know which executions each context represents.

The collecting semantics of Chapter 2, which gathers the stores reached at each node, does not record this. In the running example, both calls of `bump` reach the same result node, but a context-sensitive analysis may distinguish the activation entered with 5 from the one entered with 4. This chapter gives that distinction a concrete meaning.

Each call of a procedure starts a new *activation* of it, which lasts until the call returns [55, §7.2.1]. We represent executions by *activation traces*. An activation trace records the path of one activation through the graph. It also records the trace of the caller that created the activation, because the contexts of a call are admitted from the caller's context and store. After a call returns, it also records the finished callee, because the activation continues with the callee's result. The calling contexts of an activation are then derived from its trace.

On this semantics we build a local *coverage contract*. Its obligations are the concrete interface that the abstract analyses and the equation system of Chapter 5 to Chapter 9 must satisfy. Once they hold, the claim for a node and a context covers the stores that executions in that context reach there, and the claims of all contexts together cover every store that a run reaches at the node.

4.1. Why contexts need traces

The collecting semantics of Chapter 2 assigns each node v of a control-flow graph the set of stores that some run holds on reaching v [45, §4]. Chapter 2 over-approximated it with one unknown per node, which we now write $[v]$ [43, §2]. A context-sensitive analysis instead splits $[v]$ into unknowns $[v, c]$, one per calling context c, following the value tables of Sharir and Pnueli [56], as Apinis et al. formulate them as a constraint system [43, §3]. Seidl et al. state that the invariant for $[u, c]$ "only need[s] to take into account executions reaching u that satisfy the restriction imposed by context c" [2, §4]. The claim for $[v, c]$ therefore has to describe only a subset of the stores that the claim for $[v]$ describes, and it can be correspondingly more precise.

In the running example (Figure 3.1), `main` calls `bump(5)` and then `bump(4)`, and `bump` returns its argument plus one. A context-insensitive analysis has one claim for the result node of `bump`, which must describe both results, 6 and 5. An analysis that distinguishes the two calls by their argument can instead claim the result 6 for the call with argument 5 and the result 5 for the call with argument 4. To justify these claims, we must know which store at the result node belongs to which call. The collecting semantics, however, is too weak to tell. It holds both stores at the result node but does not record which activation reached each of them, so we need a richer semantics that keeps this information.

The graph execution (Section 3.5.2) has a frame stack, which records the callers that wait for the running activation. The stack, however, holds only what execution needs to resume these callers, and it forgets a call once the call has returned. A context policy needs more. It admits the contexts of a call from the caller's context and the caller's store at the call, and after the return the caller must continue in the contexts it had before. Activation traces keep this information. A callee trace records the caller that created it, and a resumed trace records both the caller and the finished callee.

Contexts could in principle be derived from the graph run itself. A flat record of the run, namely the list of node-store pairs (v, s) that the run passes through (Figure 4.1, top), interleaves the steps of all activations and leaves their nesting implicit. In the program of Figure 3.5, `sum(2)` calls `dec` and then itself, so one run visits the nodes of `sum` in three activations. To find the contexts of a step, we would therefore have to recover from the list which call created its activation and which return resumes which caller.

Grouping the same run by activation, in contrast, makes these relationships explicit (Figure 4.1, bottom). Each activation has its own activation path. A call creates a new activation attached to its caller, and a return composes the finished callee back into that caller. The construction is inspired by the *local traces* of Schwarz et al. [5], which record one thread's view of a concurrent execution. An activation trace applies the same idea to a sequential program and records one activation's view of the run (Section 15.5).

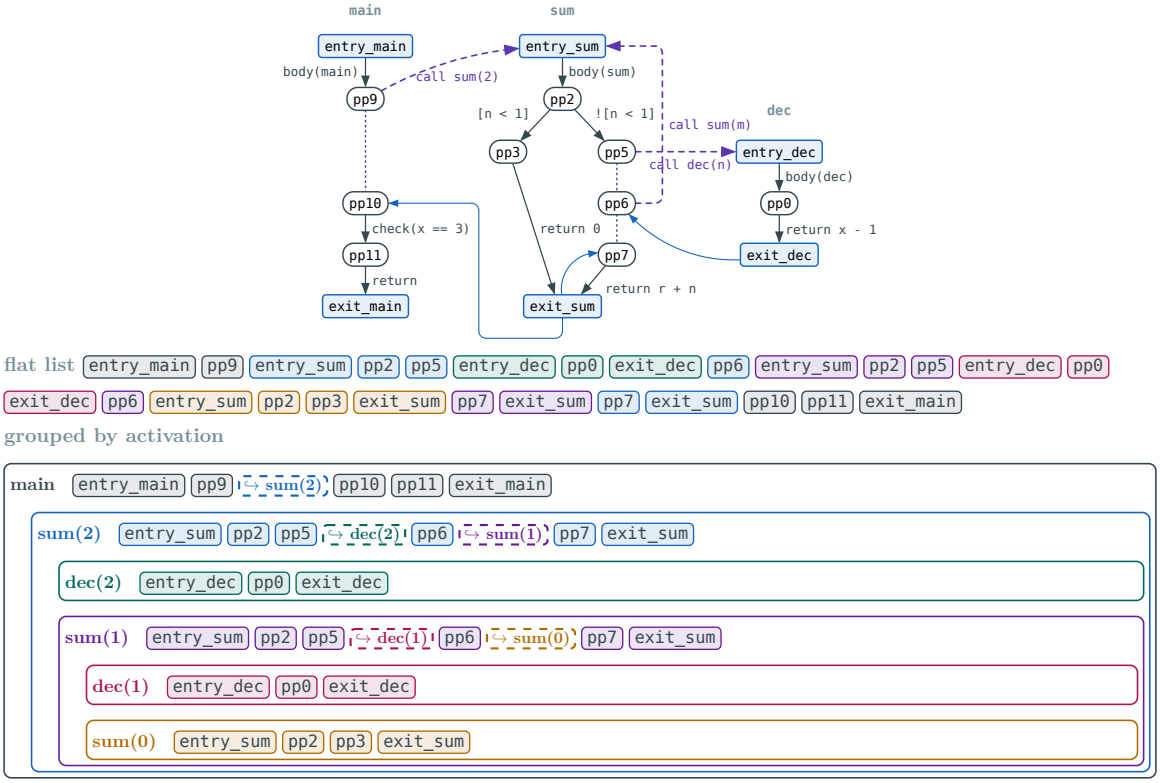


Figure 4.1: The compiled graph of the program in Figure 3.5 (top) and one run on it, as the flat list of its steps and grouped by activation. Colors mark each step's activation, which the flat list itself does not record. A dashed tag ↪ marks a call of the named callee. The run is written out by hand, and stores are omitted.

4.2. Traces

4.2.1. Activation traces thy

We turn the grouping of Figure 4.1 into a datatype.

Definition 4.1 (Activation trace, `datatype activation_trace`)**.** A trace has one constructor per way an activation begins or continues (Figure 4.2).

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)
type_synonym activation_path = "(cfg_node × store) list"
```

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)
datatype activation_trace =
    Root activation_path
  | Call (activation_trace_caller: activation_trace) activation_path
  | Resume (activation_trace_current: activation_trace) (activation_trace_callee: activation_trace)
    activation_path
```

Figure 4.2: The declarations of `activation_path` and `activation_trace`, lifted from the theory. An `activation_path` is the path of one activation, a list of pairs of a CFG node and a store.

Root π is the initial activation of the program with activation path π. A run starts with Root $[(entry, s_0)]$, the entry node of the graph, which is the entry of `main`, paired with an initial store s_0. Call $t'\ \pi$ is a callee created by the caller t' (`activation_trace_caller`). Its activation path begins at the callee's entered store. Resume $t'\ t''\ \pi$ is the activation t' (`activation_trace_current`) continued past the call that produced the finished callee t'' (`activation_trace_callee`). The observer `path_of` t returns the activation path, `sink_node` t the node of its last entry, `sink_store` t that entry's store, and `caller_of` t the caller that created the activation, which a Root does not have. The explainer page shows these observers interactively on a run of the program of Figure 4.1.

Each trace represents the history of one activation up to a program point, so on well-formed compiled programs its activation path stays inside the nodes of one procedure (`valid_activation_trace_frag_callers`). A callee is a new activation and gets a trace of its own, which holds its caller as a field. When the callee finishes, a Resume creates a new trace for the continued caller, which holds both the frozen caller and the finished callee. A whole run is therefore a family of traces linked by these fields.

Because a Call stores its caller exactly, frozen at the call node, a finished callee composes back into the activation that created it, and no search for a compatible stack frame is needed. This is what makes context-specific return flow sound. A return continues exactly the caller that created the callee, in each context that caller carries, so the result of a callee entered in context c' flows back only to the callers whose calls admitted c'. A claim for a context therefore needs to cover only the executions in that context.

The datatype stores no calling context. Which information a context records depends on the analysis. A call-string analysis distinguishes activations by their recent call sites, and an entry-state analysis by abstract entry states that it computes itself. If a trace stored its context, the concrete semantics would change with every analysis, and for entry-state contexts it would even depend on the analysis's own result (Section 8.2). Instead, the concrete semantics is fixed once. Each context policy supplies, for every concrete call, the set of callee contexts it admits, and contexts of traces are derived from these sets (Section 4.3). The soundness theorems are proved once for all policies, and an analysis only has to show that its claim meets the coverage contract of Section 4.4 for its policy.

4.2.2. Valid traces thy

The datatype also admits terms that no execution produces. A path may contain a step that follows no edge of the graph, for instance a jump from the entry of `sum` directly to its result node. A Resume may pair a caller with a callee that another activation created, for instance continue `main` after its call `sum(2)` with the finished inner activation `sum(1)`, whose caller is `sum(2)`. Validity rules out such terms.

Definition 4.2 (Valid traces, `inductive_set valid_activation_trace`)**.** For a global-variable classifier $\mathcal{G}$, a graph g, and a set S of initial stores, $\mathcal{T}_{\mathcal{G},g,S}$ (`valid_activation_trace`) is the least set of traces closed under the four rules of Figure 4.3.

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)                Isabelle
inductive_set valid_activation_trace ::
    "(vname ⇒ bool) ⇒ cfg ⇒ store set ⇒ activation_trace set"
    ("𝒯_,_,_")
  for 𝒢 and g and S where
  root:
    "s ∈ S
     ⟹ Root [(cfg_entry g, s)] ∈ 𝒯_,g,S"
| intra:
    "t ∈ 𝒯_,g,S
     ⟹ (sink_node t, a, v) ∈ intra g
     ⟹ s' ∈ edge_step a (sink_store t)
     ⟹ extend t (v, s') ∈ 𝒯_,g,S"
| call:
    "caller ∈ 𝒯_,g,S
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Call caller
          [(FunctionEntry p,
            call_enter 𝒢 (CallEdge dst pars args) (sink_store caller))]
        ∈ 𝒯_,g,S"
| ret:
    "callee ∈ 𝒯_,g,S
     ⟹ caller_of callee = Some caller
     ⟹ sink_node callee = FunctionResult p
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Resume caller callee
          (path_of caller
             @ [(cont, combine_collect 𝒢 dst
                         (sink_store caller) (sink_store callee))])
        ∈ 𝒯_,g,S"
```

Figure 4.3: The four rules of `valid_activation_trace`: root, intra, call, and ret. Each reads the graph relation for its own kind of step: `intra` for local flow, and `calls` both for entering a callee and for recovering the continuation *cont* at a return.

The root rule corresponds to the initial graph configuration, and the other three rules correspond to the three rules of `cstep`. Intra appends a step along a local edge. Call starts a new trace that holds the caller, where `cstep` pushes a frame. Ret builds a third trace that holds caller and callee, where `cstep` pops the frame. The explainer page animates

the source execution, the graph execution, and the activation trace side by side for one program.

The ret rule applies when a callee trace has reached a result node `FunctionResult p`. In a compiled graph no edge leaves it, since one result node serves every caller of p, so the rule finds the caller through `caller_of callee = Some caller` and takes a `calls` edge from its call node into p, which on compiled programs is the only such edge. That edge names the continuation *cont*, where the caller continues with the store that `combine_collect` builds from both final stores.

4.2.3. Graph runs are valid traces thy

The traces describe the runs of the compiled graph, and through Section 3.6 those of VIMP programs, keeping the history that the frame stack discards.

Definition 4.3 (Trace representation, `definition activation_trace_repr`)**.** A valid trace represents a graph configuration if it ends at the configuration's node and store, and its chain of creating callers matches the runtime frame stack (Figure 4.4).

```
(* src/Program_Model/Compile/Source_To_Trace.thy *)                                    Isabelle
inductive stack_repr :: "cfg ⇒ cframe list ⇒ activation_trace ⇒ bool" for g where
  empty [intro]: "caller_of t = None ⟹ stack_repr g [] t"
| frame [intro]: "caller_of t = Some caller ⟹ sink_store caller = s
          ⟹ fst (hd (path_of t)) = FunctionEntry p
          ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
          ⟹ stack_repr g stk caller
          ⟹ stack_repr g ((cont, dst, s) # stk) t"
```

```
(* src/Program_Model/Compile/Source_To_Trace.thy *)                                    Isabelle
definition activation_trace_repr ::
  "(vname ⇒ bool) ⇒ cfg ⇒ store set ⇒ cconf ⇒ activation_trace ⇒ bool" where
  "activation_trace_repr 𝒢 g S cf t = (case cf of (v, s, stk) ⇒
     t ∈ 𝒯_{𝒢,g,S} ∧ sink_node t = v ∧ sink_store t = s ∧ stack_repr g stk t)"
```

Figure 4.4: The declarations of `stack_repr` and `activation_trace_repr`, lifted from the theory.

For a well-formed compiled program (`wf_compile_input`), the correspondence is lock-step. A local graph step extends the current trace, a call creates a Call trace, and a return creates a Resume trace (`cstep_preserves_activation_trace_repr`). The initial configuration has a representing trace, a Root trace (`located_activation_trace_entry`). Hence every configuration that a graph run of such a program reaches has a representing valid trace (`csteps_preserve_located_activation_trace`). The converse is not proved, and soundness does not need it. With the simulation of Chapter 3, this result extends to VIMP programs.

Theorem 4.4 (Source runs are traces, `source_run_has_activation_trace`)**.** *For a well-formed compiled program, every finite source execution from an initial store in S has a matching graph configuration and a valid trace ending at the same node with the same store, whose caller chain represents the graph's frame stack.*

```
(* src/Program_Model/Compile/Source_To_Trace.thy *)                                    Isabelle
theorem source_run_has_activation_trace:
  assumes wf: "wf_compile_input 𝒢 Π ps"
    and s0: "s0 ∈ S"
    and run: "𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)"
  shows "∃v stk t. Π, compile_prog Π ps, prog_main_name ⊢ (residual, s, frs) ≈ (v,
  s, stk)
                    ∧ activation_trace_repr 𝒢 (compile_prog Π ps) S (v, s, stk) t"
```

4.2.4. The node collecting semantics thy

Collecting the final stores of the valid traces per node gives the *node collecting semantics*. It plays the role of the collecting semantics of Chapter 2 for programs with procedures, with valid traces in place of runs.

Definition 4.5 (Node collecting semantics, `definition node_collect`)**.**

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Collect.thy *)                Isabelle
definition node_collect ::
    "(vname ⇒ bool) ⇒ cfg ⇒ store set ⇒ cfg_node ⇒ store set"
    ("𝒞_,_,_") where
  "𝒞_,g,S v =
     {sink_store t | t. t ∈ 𝒯_,g,S ∧ sink_node t = v}"
```

A store is in $\mathcal{C}_{\mathcal{G},\mathtt{g},\mathtt{S}}\ \mathtt{v}$ exactly when some valid trace ends at v with this store. The classifier, graph, and initial stores are those of the program at hand, written $\mathcal{G}$, `g`, and `S` from here on. A procedure's result is an ordinary collected node, and the stores in which a run of the whole program completes are those collected at FunctionResult *main*. Every store that a finite source execution of a well-formed program reaches therefore lies in the node collecting semantics at a corresponding node (`source_reaches_node_collect`).

The node collecting semantics forgets the traces, so a claim stated over it alone is context-insensitive. The valid traces themselves, however, keep their structure. The same set of traces thus supports two views: collected per node, as here, and split by context, as in Section 4.3.

4.3. Calling contexts thy

A *context* is an index under which an activation is analyzed, and one activation may carry several. At a fixed node, activations with the same context contribute to the same abstract unknown $[v, c]$. Contexts change only at calls. Within one activation, a step along a local edge keeps the contexts of the trace (`activation_context_rel_extend`), and after a return the caller continues in the contexts it had before the call (`activation_context_rel_Resume_iff`).

Which information a context records is a design decision of the analysis, and the literature describes many such *context policies* [16, §8.4.1], [51, §2.6], [51, §2.9], [56]. The call-string approach records the call history, and practical variants bound it to the k most recent call sites, since a recursive procedure otherwise has infinitely many contexts. The functional approach, in contrast, computes a procedure summary independent of callers, which each call site instantiates. Goblint uses partial tabulation, a variant of the functional approach that tabulates summaries only for the calling contexts the analysis meets [31, §3.3], [35, §5, p. 110]. Voblint's entry-state policy is of this kind. Its context is the abstract value of the callee's formal parameters in the entry state (Chapter 8), and nothing bounds the number of such contexts (Section 9.6). Goblint obtains the context of a callee by applying each analysis's `context` function to the callee's abstract entry state (Section 15.5), and Erhard et al. treat full entry states, their projections, and call strings in one framework [72, §4]. Voblint offers the context-insensitive policy, bounded call strings, and entry-state contexts (`context_mode`, Chapter 8).

These context selectors are functions, each choosing one callee context per call from the call site, the caller's context, or the entry state. Goblint's entry operation, however, may

return several alternatives for one call, each a pair of a caller state and a callee entry state, and each alternative gets its own context. Several alternatives let an analysis split a call into cases. Suppose a caller passes an argument that is either negative or positive, and the caller's state keeps the two cases apart, for instance as two path-sensitive states. With a single entry state, the callee is analyzed once for the join of both cases, which a sign analysis can only describe by the greatest element $\top$, any sign. With one alternative per case, the callee is analyzed once in a context for negative arguments and once in a context for positive ones, so the analysis preserves the argument's sign separately in the two contexts. If the cases overlap, one concrete call lies in several alternatives and belongs to several contexts at once. A context function of the call cannot express this, so a Voblint policy returns, for each concrete call, a set of callee contexts. The analyses shipped with Voblint return one alternative per call, so for them the set has at most one element for a fixed caller context. A separate example returns two overlapping alternatives and admits one concrete call in two contexts (`ov_two_contexts_admitted`). The set may also be empty. Under entry-state contexts it is empty for every caller store that the analysis's state at the call site does not describe, for instance at a call that the analysis considers unreachable. No alternative describes such a store, so no context is admitted for it. Emptiness is harmless there, because the claim contains no such store. It is harmful at a call that the program actually makes. In one example, an entry operation that returns no alternative leaves the callee unanalyzed and the continuation at $\bot$ (`ov_empty_no_callee_context`, `ov_empty_continuation_bot`), although the program, which has no branches, reaches the continuation. The coverage contract rules this out through TOTAL (Section 4.4).

Definition 4.6 (Context policy, `type_synonym context_policy`)**.** A context policy maps a concrete call to the set of callee contexts it admits (Figure 4.5).

```
1 (* src/Program_Model/CFG/Collecting/Activation_Trace_Context.thy *)                    Isabelle
2 type_synonym 'c context_policy =
3   "cfg_node ⇒ 'c ⇒ call_info ⇒ store ⇒ store ⇒ 'c set"
```

Figure 4.5: The declaration of `context_policy`, lifted from the theory.

A context policy is a mathematical object over concrete stores and is not executed. The analyzer instead chooses callee contexts from the abstract entry states it computes, and Section 8.2 gives the two conditions, totality and adequacy, under which these choices agree with the policy.

A context policy *adm* maps a concrete call to the set of callee contexts c' it admits. Each of its arguments serves some policy. The call site lets call strings record where a call happened: with call strings of length one, `bump(5)` enters the context that consists of its call site in `main`. The caller's context lets call strings extend the caller's history and lets entry-state routing read the abstract state that the analysis computed for the caller in that context. The call information, which names the call's callee, formals, arguments, and result variable, is what the analysis's entry operation needs to compute an abstract entry state. The two stores, finally, serve entry-state contexts. The context is read off the values that an abstract entry state gives the formals, and that entry state must describe the entered store, the store at the callee's entry after the formals have been bound: `bump(5)` enters with $n = 5$. Each alternative also describes the caller, and the return later combines this description with the callee's result. A context is therefore admitted only for a caller store that its alternative describes (Section 8.2).

`admits_call_context` applies *adm* to an actual call edge of the graph, with the entered store that the edge's entry transfer `call_enter` computes from the caller's store.

The relation `activation_context_rel` assigns contexts to whole traces. It does not pick one context per activation. It states which contexts an activation may carry, and there can be none, one, or several. The root activation carries the initial context c_0. A called activation carries a context c' if its caller carries some context c and c' lies in the set that *adm* admits for the call from c. A resumed activation carries the contexts that the caller had before the call. The explainer page draws, for each policy, the contexts that the analyzer creates for one program, with every copy of a procedure labeled by its context.

Definition 4.7 (Context of a trace, `inductive activation_context_rel`)**.** The contexts that a trace may carry (Figure 4.6).

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Context.thy *)                    Isabelle
inductive activation_context_rel ::
  "(vname ⇒ bool) ⇒ 'c context_policy ⇒ 'c ⇒ cfg ⇒ activation_trace ⇒ 'c ⇒ bool"
  for 𝒢 :: "vname ⇒ bool" and adm :: "'c context_policy" and c₀ :: 'c and g :: cfg
where
  Root [intro]: "activation_context_rel 𝒢 adm c₀ g (Root xs) c₀"
| Call [intro]:
    "activation_context_rel 𝒢 adm c₀ g caller c
     ⟹ admits_call_context 𝒢 g adm (sink_node caller) c p (sink_store caller) es c'
     ⟹ activation_context_rel 𝒢 adm c₀ g (Call caller ((FunctionEntry p, es) # xs)) c'"
| Resume [intro]:
    "activation_context_rel 𝒢 adm c₀ g current c
     ⟹ activation_context_rel 𝒢 adm c₀ g (Resume current callee xs) c"
```

Figure 4.6: The declaration of `activation_context_rel`, lifted from the theory.

The contexts of an activation are thus fixed when it is created.

The node collecting semantics can then be split by context. A store belongs to context c at node v if some valid trace ending at v with that store carries c.

Definition 4.8 (Activation collecting semantics, `definition activation_collect`)**.**

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Context.thy *)                    Isabelle
definition activation_collect ::
  "(vname ⇒ bool) ⇒ 'c context_policy ⇒ 'c ⇒ cfg
     ⇒ store set ⇒ cfg_node ⇒ 'c ⇒ store set"
    ("𝒜_,_,_,_,_") where
  "𝒜𝒢,adm,c₀,g,s v c =
     {sink_store t | t. t ∈ 𝒯𝒢,g,s ∧ sink_node t = v
                      ∧ activation_context_rel 𝒢 adm c₀ g t c}"
```

We call $\mathcal{A}_{\mathcal{G},\mathrm{adm},c_0,g,s}\ v\ c$ the *activation collecting semantics* at v and c. It contains the final stores of the valid traces that end at v and carry c, and the claim for the unknown $[v, c]$ must contain it. In the running example under interval entry-state contexts, `bump(5)` gets the context $[[5, 5]]$ and `bump(4)` the context $[[4, 4]]$ (Section 8.2). The final store of `bump(5)`, with result 6, lies in the activation collecting semantics of context $[[5, 5]]$ at `bump`'s result node, and the final store of `bump(4)`, with result 5, in that of context $[[4, 4]]$. The node collecting semantics there contains both stores. Stores reached in two contexts lie in both sets.

The two sets show what the split by context buys. A single claim for the result node must contain both stores, so an interval analysis can at best bound the result by $[5, 6]$. A claim per context only has to contain the stores of its own context, 6 in $[[5, 5]]$ and 5 in $[[4, 4]]$. No store is lost by the split as long as every valid trace carries at least one context, because then every store of the node collecting semantics lies in the activation collecting semantics of some context. The coverage contract of the next section guarantees this.

4.4. The coverage contract thy

The goal is a claim `cover` v c, a set of stores for each node and context, that contains $\mathcal{A}_{\mathcal{G},\mathtt{adm},\mathtt{c_0},\mathtt{g},\mathtt{S}}$ `v c`. A claim is a mathematical object, and its sets of stores may be infinite. The analyzer instead computes abstract values, and Chapter 8 takes `cover v c` to be the concretization of the value computed for $[v, c]$. The coverage contract reduces the goal to local conditions on `cover` that mention only stores. It is therefore the interface between the analyzer and the concrete semantics.

Valid traces grow by four rules, so it is enough to check the claim locally against the same four cases. There is one obligation per rule: INIT for the root activation, INTRA for a local edge, CALL for the entry into a callee, and RETURN for the continuation after a callee finishes. Each obligation other than INIT looks at one step of a run and requires the claim to contain the store after the step whenever it contains the stores before it. Figure 4.7 shows these obligations at one call, together with a fifth, TOTAL.

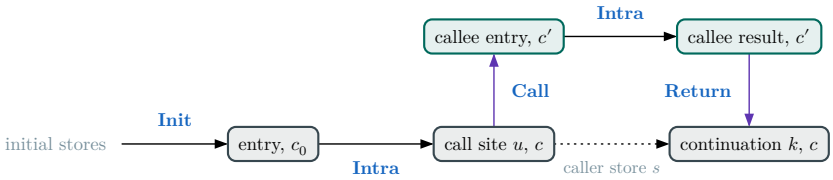


Figure 4.7: The obligations at one call (schematic). Boxes are (node, context) pairs. Solid arrows show where each obligation applies; INTRA may be applied repeatedly along an activation path. The dotted arrow is no obligation. It shows the caller store s that RETURN reads. RETURN combines the caller's store s with the callee's result read in the admitted context c'. TOTAL demands that at least one c' exists.

The fifth obligation, TOTAL, concerns a call at a call site u in a caller context c from a caller store s. If $s \in$ `cover u c`, TOTAL requires the policy to admit at least one callee context for the call from s (`call_context_total_on`).

The other four obligations do not suffice without it (`total_dropped_unsound`). At a call that gets no context, CALL and RETURN hold vacuously, so nothing forces the claim at the continuation, while the resumed caller keeps its contexts (`resume_keeps_context`). A store that the caller reaches after the return can therefore lie outside the claim.

Asking for contexts at every call would be too strong for entry-state policies, whose sets are empty for the caller stores that the analysis excludes (Section 4.3). The context-insensitive policy and call strings admit a context for every call and meet TOTAL for every claim (`call_context_total_on_of_fun`). For entry-state contexts it follows from the coverage of the analysis's entry (Section 8.2). Isabelle states the five obligations as the assumptions of the locale `activation_coverage` (Figure 4.8).

Theorem 4.9 (Context-indexed soundness, `activation_collect_sound`)**.** *If the claim meets the five obligations of `activation_coverage`, then for every node v and context c, the activation collecting semantics at v and c is contained in the claim `cover v c`.*

```
(* src/Abstract_Interpreter/Framework/Activation/Activation_Backbone.thy *)   Isabelle
theorem activation_collect_sound:
  assumes "activation_coverage g S cover adm c₀ 𝒢"
  shows "𝒜𝒢,adm,c₀,g,S v c ⊆ cover v c"
```

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)                    Isabelle
locale activation_coverage =
  fixes g :: cfg and S :: "store set"
    and cover :: "cfg_node ⇒ 'c ⇒ store set"
    and adm :: "'c context_policy"
    and c₀ :: 'c
    and 𝒢 :: "vname ⇒ bool"
  assumes INIT[intro]: "⋀s. s ∈ S ⟹ s ∈ cover (cfg_entry g) c₀"
    and INTRA[intro]: "⋀u a v c s s'. (u, a, v) ∈ intra g
        ⟹ s ∈ cover u c ⟹ s' ∈ edge_step a s ⟹ s' ∈ cover v c"
    and CALL[intro]: "⋀u dst pars args p cont c c' s.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ c' ∈ adm u c (call_info_of (CallEdge dst pars args) p) s
             (call_enter 𝒢 (CallEdge dst pars args) s)
        ⟹ call_enter 𝒢 (CallEdge dst pars args) s ∈ cover (FunctionEntry p) c'"
    and RETURN[intro]: "⋀u dst pars args p cont c c' p' s t es.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ admits_call_context 𝒢 g adm u c p' s es c'
        ⟹ t ∈ cover (FunctionResult p) c'
        ⟹ combine_collect 𝒢 dst s t ∈ cover cont c"
    and TOTAL: "call_context_total_on cover adm 𝒢 g"
```

Figure 4.8: The declaration of `activation_coverage`, lifted from the theory. In RETURN, p' and `es` range over every call edge out of `u`. On compiled programs a call site has one call edge (`compile_prog_calls_source_unique`), so $p' = p$.

The same obligations give every valid trace at least one context, so the activation collecting semantics of all contexts together is exactly the node collecting semantics.

Theorem 4.10 (Exhaustive contexts, `node_collect_eq_Union_activation_collect`)**.** *Under the five coverage obligations, including* TOTAL*, the node collecting semantics at every node v is the union of the activation collecting semantics at v over all contexts.*

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)                    Isabelle
theorem node_collect_eq_Union_activation_collect:
  "𝒞 v = (⋃c. 𝒜 v c)"
```

Corollary 4.11 (Claims cover the collection, `node_collect_covered`)**.** *Under the five coverage obligations, every store of the node collecting semantics at v lies in the claim `cover v c` of some context c.*

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)                    Isabelle
corollary node_collect_covered: "𝒞 v ⊆ (⋃c. cover v c)"
```

Apinis et al. recall the corresponding observation for the abstract side [43, §3]: after local solving, the abstract values that reach a program point are bounded by the join of the solution's values at that point over the contexts the solver encountered.

This chapter has built the concrete half of the soundness argument. Every finite source execution of a well-formed program is matched by a graph run (Chapter 3), which a valid activation trace represents (Section 4.2.3). Its contexts place its final store in their activation collecting semantics (Section 4.3), which every claim meeting the coverage contract contains. The following chapters construct such claims by abstract interpretation, with abstract domains describing sets of stores (Chapter 5) and equation systems computing the abstract values (Chapter 8, Chapter 9).

PART III

Analyses

5. Abstract Domains

Chapter 4 reduced soundness to five obligations on a claim. A claim assigns each node and context a set of stores. The obligations are local conditions, and together they guarantee that the claim contains every store that an execution in that context reaches at the node (`activation_collect_sound`). These sets are arbitrary and usually infinite. An analyzer, however, computes with finite abstract values, and its solver (Section 2.7) compares, joins, widens, and narrows them without knowing what they mean. Recall from Section 2.3 that an abstract value a denotes a set $\gamma(a)$ of concrete values. A value of a numeric domain such as Sign or Interval denotes a set of integers, $\gamma(a) \subseteq \mathbb{Z}$. An abstract state d, in contrast, denotes a set of stores, $\gamma(d) \subseteq (\texttt{vname} \to \mathbb{Z})$ (Section 5.2). We write γ for both, as the theories do. The type of the argument determines which concretization is meant. An arbitrary lattice of abstract values is not enough, for two reasons. First, the order must agree with the meaning. The solver only proves inequalities $a \leq b$ in the abstract order, while the obligations of Chapter 4 are inclusions between sets. The law

$$a \leq b \Longrightarrow \gamma(a) \subseteq \gamma(b)$$

turns each such inequality into an inclusion (`gamma_mono`, Figure 5.1). Second, a value can denote no integer without being the lattice's bottom element. The interval pair $[5, 3]$, whose lower bound exceeds its upper bound, is such a value. An analyzer that recognizes only the bottom element as empty misses it. This is still sound, since the analyzer then assumes more stores than exist, but it costs precision. After the next join, the information that a program point is dead is gone (Section 5.2.1).

This chapter therefore asks what an abstract domain must provide so that the analyzer can compute with it, every result still contains the stores it stands for, and no precision is lost merely through the abstract representation, such as the fact that a program point is unreachable.

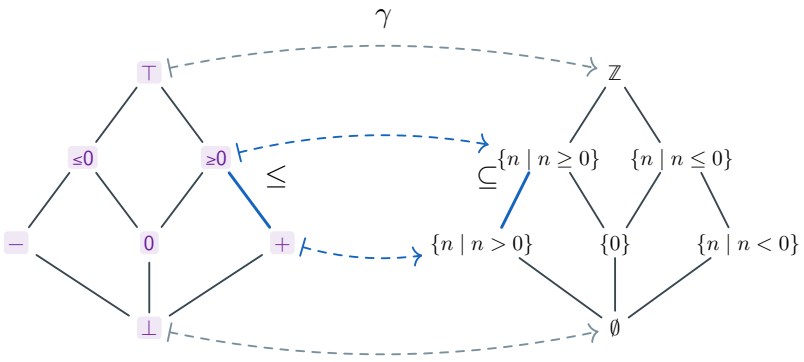


Figure 5.1: The Sign domain (left) and the sets of integers its values denote (right, mirrored), both ordered bottom to top. The dashed arrows show four instances of γ. Since γ is monotone, the blue step $+ \leq \geq 0$ becomes the inclusion $\gamma(+) \subseteq \gamma(\geq 0)$. Adapted from [6, Fig. 13.5].

5.1. What an abstract value means thy

The two requirements of the chapter opening become laws on the domain's carrier. A domain is a type A of abstract values, its *carrier*. The solver compares its values, joins them where control flow merges, starts from a least value $\bot$, and extrapolates with the widening ∇ and the narrowing Δ of Section 2.5. The analysis also needs a greatest value $\top$ for unknown values. A domain further supplies an emptiness test, a printer, and a concretization $\gamma : A \to \mathcal{P}(\mathbb{Z})$ with four laws. Such a type is a *numeric domain*:

```
(* src/Abstract_Interpreter/Domain/Lattice/Abstract_Domain.thy *)                Isabelle
class numeric_domain = executable_domain +
  fixes gamma :: "'a::{bounded_semilattice_sup_bot, order_top} ⇒ int set" ("γ")
  assumes gamma_bot[simp]: "γ ⊥ = {}"
  assumes gamma_top[simp]: "γ ⊤ = UNIV"
  assumes gamma_mono: "a ≤ b ⟹ γ a ⊆ γ b"
  assumes is_empty_correct: "is_empty a ⟷ γ a = {}"
```

The third and fourth laws, `gamma_mono` and `is_empty_correct`, are the two requirements of the chapter opening. The class extends `executable_domain`, the part the analyzer runs, by the concretization and its laws.

Intervals (Section 2.3) are this chapter's running example thy . As in Nipkow and Klein [6, §13.8.1], bounds are extended integers `eint`, and an interval is a pair of bounds with $\gamma([l, u]) = \{n \in \mathbb{Z} \mid l \le n \wedge n \le u\}$. On nonempty pairs, the order and join (`less_eq_ivl`, `sup_ivl`) are those of Section 2.3. Nipkow and Klein identify all empty pairs by a quotient type. Voblint's `ivl` is a plain datatype of bound pairs, so it must handle them itself. Every pair with $l > u$ is empty, and so are $[+\infty, +\infty]$ and $[-\infty, -\infty]$, since no integer equals an infinite bound. The bottom value $\bot = [+\infty, -\infty]$ is only one of these empty pairs. An empty pair such as $[5, 3]$ denotes the same set as $\bot$, yet it is neither $\bot$ nor below it.

The arithmetic operations build their results through `normalize_ivl`, which replaces every empty pair by $\bot = [+\infty, -\infty]$ and leaves nonempty pairs unchanged. Otherwise $[1, 0] + [1, 1] = [2, 1]$ would keep creating new empty values, and the solver, which tests whether a value changed (Section 2.7), could fail to stabilize. Normalizing never changes the denoted set (`normalize_ivl_gamma`), so it is sound wherever it is applied.

The meet intersects the bounds, so $[0, 3] \sqcap [5, 9] = [5, 3]$, an empty pair. In the order on pairs, $[5, 3]$ is the greatest lower bound of the two operands, as a meet must be. A normalizing meet would return $\bot$ instead. However, $[5, 3]$ lies below both operands but not below $\bot$, since its lower bound 5 is smaller than the lower bound $+\infty$ of $\bot$. So $\bot$ would not be the greatest lower bound (`meet_ivl_normalized_breaks_greatest`). Because an empty pair is not necessarily $\bot$, the emptiness test inspects the bounds (`is_bottom_ivl`). A pair is empty if $l > u$, if its lower bound is $+\infty$, or if its upper bound is $-\infty$. The printer uses the same test and shows every empty pair as $\bot$. It lies outside the verified boundary (Section 12.5).

The widening is the standard one of Section 2.5 (`widen_ivl_core`), with one exception: widening with $\bot$ returns the other operand. The standard rule pushes a bound to infinity whenever the new value reaches beyond it. Both bounds of $\bot = [+\infty, -\infty]$ are exceeded by every nonempty interval, so the rule would widen $\bot \nabla [0, 0]$ all the way to $\top$. The narrowing `narrow_ivl_td` refines only infinite bounds and keeps finite ones, so $[0, +\infty] \Delta [0, 10] = [0, 10]$. Each bound leaves infinity at most once, so a narrowing sequence is finite.

5.2. From values to stores

A numeric domain describes the possible values of one variable. The obligations of Chapter 4, however, are stated over sets of stores. A store maps every variable name to an integer, `store` $=$ `vname` $\to \mathbb{Z}$, so at a program point the analysis must describe whole stores. This thesis uses the two forms introduced in Section 2.4, non-relational and relational states. A non-relational description gives each variable its own abstract value, and a relational one records how variables relate to each other. The two differ in what they can state. Take the stores with $0 \le x \le y \le 5$ (Figure 5.2). The best non-relational description over intervals is $\{x \mapsto [0,5], y \mapsto [0,5]\}$. It shows $x \ge 0$ and $y \le 5$, but not $x \le y$, since it also admits $x = 5, y = 0$. A relational description that records $x \le y$ shows exactly that fact, but it bounds neither variable, so it also admits $x = y = -7$. Therefore neither description is stronger than the other. Together, however, they describe exactly the given set.

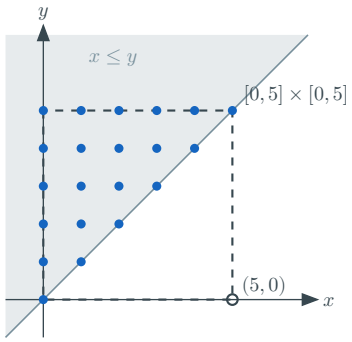


Figure 5.2: Stores over two variables as points (x, y). The stores with $0 \le x \le y \le 5$ are the dots. The best non-relational state denotes the square (dashed), which also admits $(5,0)$. The relational state $\{x \le y\}$ denotes the half-plane above the diagonal (shaded), which is unbounded. Their intersection is exactly the dots. Illustrative.

An *abstract state* describes a set of stores. It is a type D with the order and join the solver needs, and a monotone concretization that maps each state d to the set $\gamma(d)$ of stores it describes. Each kind of state defines its own concretization, and Isabelle overloads one constant, `gamma_S`, written $\gamma(d)$, for all of them. The numeric analyses use non-relational states. An analysis that records relations $x \le y$ between variables uses a relational state.

5.2.1. A non-relational state thy

A non-relational state is represented pointwise. It replaces the integer of a store by an abstract value: `abs_state` $=$ `vname` $\Rightarrow$ `'a`, so a state d maps every variable name x to $d(x)$. Its concretization admits a store if every variable's value lies in the concretization of its abstract value:

```
1 (* src/Abstract_Interpreter/Domain/State/Nonrelational_State.thy *)                    Isabelle
2 definition gamma_state :: "('a::numeric_domain) abs_state ⇒ store set" where
3   "gamma_state d = {s. ∀x. s x ∈ γ (d x)}"
```

Order and join work variable by variable, as for any function type in HOL. A state d_1 lies below d_2 if $d_1(x) \le d_2(x)$ for every variable x, and the join is $(d_1 \sqcup d_2)(x) = d_1(x) \sqcup d_2(x)$ (Figure 5.3). Because γ is monotone, so is this concretization, and a larger state describes more stores.

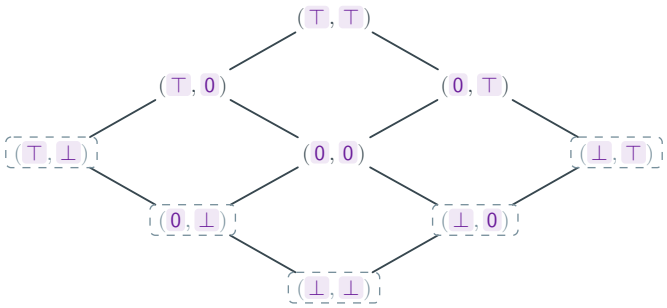


Figure 5.3: Pointwise states $(d(x), d(y))$ over two variables, with values from the fragment $\bot \leq 0 \leq \top$ of Sign, ordered variable by variable. The dashed states have a $\bot$ component and denote no store, although only the lowest one is the bottom state.

Some program points are reached by no execution, for example code behind a condition that never holds. The analysis should recognize them from their state. A pointwise state makes this hard, because it can say "unreachable" in many ways. Every state with one empty variable describes no store, and only one of them is the bottom state (Figure 5.3).

Neither simple test works. Treating only the all-bottom state as unreachable misses most cases, because refining a branch condition (Section 5.5) usually empties a single variable, as in $\{x \mapsto \bot, y \mapsto \top\}$. Treating every empty state as unreachable is correct at the moment of the test, but the information does not survive the next step. Joining two differently empty states can give a nonempty one (Figure 5.4). This is sound, since it only adds stores, but it loses the fact that the point is unreachable and is thus imprecise.

```
1 fun main() {                                                    VIMP ↗
2   x = __voblint_nondet_int();
3   y = __voblint_nondet_int();
4   if ((x == 0 && x == 1) || (y == 0 && y == 1)) {
5     __voblint_check(false);
6   }
7 }
```

Figure 5.4: A branch no execution enters, since no integer equals both 0 and 1. Sign refines the left disjunct to $\{x \mapsto \bot, y \mapsto \top\}$ and the right one to $\{x \mapsto \top, y \mapsto \bot\}$. Both are empty, but their pointwise join is $\{x \mapsto \top, y \mapsto \top\}$. The branch turns each empty arm into Bot before the join, and the analyzer reports the check in the branch as unreachable (claim `dom-disjunct-sign`).

Voblint therefore makes unreachability a value of its own, as Nipkow and Klein do with an option type [6, §13.7] and as Goblint does. Goblint's framework lifts the local state by an outer bottom element for dead code (`Analyses.Dom`), and a transfer function that finds its path unreachable raises the `Deadcode` exception instead of returning a state. In Voblint, a lifted state (`lifted`) is either Bot, meaning unreachable, or Lifted(d) for an ordinary state d, with Bot below every other value and $\gamma(\text{Bot}) = \emptyset$.

After every transfer, the analysis tests whether the result is empty and, if so, replaces it with Bot (`normalize_lift`). Comparing with the bottom state cannot implement this test, for the reason above. Instead, the test relies on the domain's emptiness test, which `is_empty_correct` makes exact for every value. A pointwise state is empty exactly when some variable's value is (`is_empty_state`, `is_empty_state_iff_gamma_state_empty`). The variable names form an infinite set, so the executable analyzer decides this on a

finite representation of the state (Section 9.5). Replacing an empty state by Bot only exchanges one abstract representation of the empty set for another, equivalent one (`gamma_state_normalize_lift`), so it cannot make the analysis unsound.

From then on Bot stays Bot. A transfer applied to Bot is skipped and returns Bot, and a join with Bot returns the other operand. An unreachable program point thus keeps the value Bot, and the analysis can report it as unreachable. In the program of Figure 5.4, both arms are joined inside one branch transfer, so the test after the transfer would come too late. The branch refinement of Section 5.5 therefore also replaces each empty arm of a disjunction with Bot before the join, and the two arms join to Bot.

5.2.2. A relational state thy

A relational state keeps what the pointwise form forgets. Its concretization therefore maps a whole state to a set of stores directly, instead of a value to a set of integers. The type `relc` records a set P of variable pairs, where the pair (x, y) stands for the constraint $x \leq y$. Its concretization is `gamma_relc`:

$$\gamma(\mathsf{RelC}(P)) = \{s \mid \forall (x, y) \in P. s(x) \leq s(y)\}, \quad \gamma(\mathsf{RelBot}) = \emptyset.$$

Below, the pair (x, y) is written $x \leq y$.

More pairs describe fewer stores, so the order is reverse inclusion: $\mathsf{RelC}(P) \leq \mathsf{RelC}(Q)$ exactly when $Q \subseteq P$ (`less_eq_relc`), with RelBot below every value and $\top = \mathsf{RelC}(\emptyset)$, which constrains nothing. The join keeps the pairs both operands share, $\mathsf{RelC}(P) \sqcup \mathsf{RelC}(Q) = \mathsf{RelC}(P \cap Q)$ (`sup_relc`). The widening is simply the join. The solver's widening contract (`widening`) only asks for a value above both operands, and the join is one, although the contract alone does not force the iteration to terminate. Here an ascending chain can only drop pairs from a finite set, since every constructed P is finite (below), so it is finite anyway. The narrowing returns its second operand, the newly computed value. This is sound for the same reason, since the narrowing law only asks for a value between the two operands.

The carrier does not close its pairs under transitivity (Figure 5.5). The values $\{x \leq y, y \leq z\}$ and $\{x \leq y, y \leq z, x \leq z\}$ therefore denote the same stores but are different values. The join can also forget an implied relation. The join of $\{x \leq y, y \leq z\}$ and $\{x \leq y, x \leq z\}$ keeps only their common pair and is $\{x \leq y\}$, although both operands imply $x \leq z$.

Pairs enter only at guards, and an assignment to x removes every pair that mentions x, so P is finite in every value the analysis constructs. This matters for the executable analyzer. Generated code stores a finite set as the list of its elements, while an infinite set, such as the set of all pairs, needs a complement representation for which not every set operation has executable code.

All constraints in P can be satisfied simultaneously by assigning the same value to every variable. In particular, the store that maps every variable to 0 satisfies every pair in P. Therefore every $\mathsf{RelC}(P)$ denotes at least one store, regardless of which constraints P contains. Only RelBot denotes the empty set of stores. The emptiness test only compares with RelBot (`is_empty_relc_gamma`). Unlike the pointwise form, the carrier has a single representation of unreachability, so it needs no lifting.

A relational state stores no value per variable, so it cannot evaluate an expression to an abstract value. Its transfers update the constraints directly instead. An assignment to x forgets every pair that mentions x, and a comparison between two variables adds the pairs

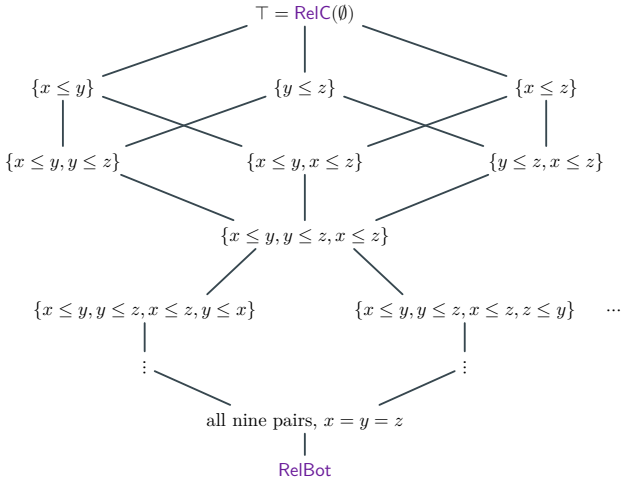


Figure 5.5: The relational states over the variables x, y, and z, ordered bottom to top by reverse inclusion. Each RelC value is drawn as its set of pairs. The upper part shows all values built from $x \leq y$, $y \leq z$ and $x \leq z$. Values with other pairs, such as $y \leq x$, are drawn only below the lowest of these, and the dots stand for the rest. Nine pairs exist, including $x \leq x$, so the values are finitely many, and the lowest RelC value holds all nine and denotes the stores with $x = y = z$.

it implies (Section 5.5). The type instantiates the solver's classes and `executable_domain`, but it is not a numeric domain. The next three sections introduce operations on single numeric values. They serve non-relational states, and Section 5.5 returns to the relational state at its end.

5.3. Evaluating expressions thy

The carrier operations serve the solver. The analysis additionally needs operations that mirror the program's own arithmetic, comparisons, and branch conditions. We call them the domain's *primitives*. This section and the next two introduce them, one use at a time.

An assignment `x = e` needs the abstract value of e. Forward evaluation computes it from an abstract state d, as in the generic abstract interpreter of Nipkow and Klein [6, §13.5.3]. The abstract result $[\![e]\!]^\sharp\, d$ must contain every concrete result (`sound_evaluator`):

$$s \in \gamma(d) \Longrightarrow [\![e]\!]_e\, s \in \gamma\big([\![e]\!]^\sharp\, d\big).$$

Interval evaluates by interval arithmetic on the bounds (`aval_ivl`). For example, if $d(x) = [0, 9]$, Interval evaluates `x + 1` to $[1, 10]$.

A condition is an expression too, and VIMP reads its value as a truth value, with nonzero meaning true. If the abstract value of a condition admits only one truth value, a branch can drop the arm that needs the other. A domain therefore also supplies a truth test (`sound_truth_test`). The test tobool(a) has three possible answers. It returns Some(b) if every integer in $\gamma(a)$ has truth value b, and None otherwise:

$$\text{tobool}(a) = \text{Some}(b) \wedge i \in \gamma(a) \Longrightarrow (i \neq 0) = b.$$

Interval's test `interval_tobool` returns Some(true) for $[l, u]$ if the interval excludes 0, that is $u < 0$ or $l > 0$, and Some(false) if $l = u = 0$. Otherwise it returns None. So $[1, 5]$ is true, $[0, 0]$ is false, and $[0, 5]$ is undecided.

5.4. Answering queries

An assertion `__voblint_check(c)` asks whether c holds in every store that reaches its program point. The analysis must answer this from the abstract state there alone. Voblint answers it through queries, as does Goblint's `assert` analysis.

A domain answers two comparisons on abstract values, $\mathrm{less}(a, b)$ for $a < b$ and $\mathrm{eq}(a, b)$ for $a = b$. Like the truth test, each has three possible answers. A definite answer must hold for every pair of integers the two values denote, and None, meaning unknown, is always allowed:

```
(* src/Abstract_Interpreter/Domain/Eval/Numeric_Queries.thy *)                Isabelle
locale sound_numeric_queries =
  fixes less :: "'a::numeric_domain ⇒ 'a ⇒ bool option"
    and eq :: "'a ⇒ 'a ⇒ bool option"
  assumes less_sound[intro]:
      "less a b = Some r ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ (i < j) = r"
    and eq_sound[intro]:
      "eq a b = Some r ⟹ i ∈ γ a ⟹ j ∈ γ b ⟹ (i = j) = r"
```

Domains differ in which comparisons they can decide. Interval decides $a < b$ when one interval lies entirely below the other, where a shared endpoint suffices for the answer false, and $a = b$ when the intervals are disjoint or the same single integer. Sign decides $a < b$ whenever the signs separate the values, for example for zero and a positive value. Parity and Congruence, the domains of Chapter 11, are weaker here. Parity can only refute equality between an even and an odd value and never decides $<$. Congruence answers only when both values are single integers.

Definition 5.1 (Query, `fun check_query`)**.** A query evaluates a condition c on an abstract state d to one of three answers, *true*, *false*, or *unknown*, by recursion on c, using only the domain's two comparisons.

In `check_query`, a comparison first evaluates its operands forward to abstract values $\hat{a}$ and $\hat{b}$ (Section 5.3) and then asks one of the two comparisons, with the operands swapped or the answer negated where needed:

$$a < b \rightsquigarrow \mathrm{less}(\hat{a}, \hat{b}), \qquad a > b \rightsquigarrow \mathrm{less}(\hat{b}, \hat{a}), \qquad a == b \rightsquigarrow \mathrm{eq}(\hat{a}, \hat{b}),$$
$$a \leq b \rightsquigarrow \neg\, \mathrm{less}(\hat{b}, \hat{a}), \quad a \geq b \rightsquigarrow \neg\, \mathrm{less}(\hat{a}, \hat{b}), \qquad a \neq b \rightsquigarrow \neg\, \mathrm{eq}(\hat{a}, \hat{b}).$$

Negation keeps *unknown*. The connectives `!`, `&&` and `||` combine the answers of their operands in three-valued logic. A definite *false* decides `&&`, and a definite *true* decides `||`, regardless of the other operand's answer. Both operands are evaluated in the same state, so for x in $[1, 2]$, `x == 1 || x == 2` stays unknown, although every store satisfies it (claim `dom-disjunction-one-state`). Goblint is more precise here. CIL compiles `&&` and `||` into branches (`cabs2cil.ml`), and where `x == 1` fails, Goblint's interval refinement narrows $[1, 2]$ to $[2, 2]$ (`baseInvariant.ml`, [31, §4.5.1, Rem. 8]). Voblint's interval inverse of a failed equality changes nothing (`inv_eq_ivl`). Any other expression e is read as $e \neq 0$, its truth value in VIMP. A new domain therefore gets checks on arbitrary conditions by providing these two comparisons. One proof covers every domain. The answer *true* certifies that c holds in every store the state describes, *false* that it holds in none, and *unknown* makes no claim (`check_query_sound`). `sound_check_query` combines the queries with forward evaluation, and Section 10.2 turns the answers into the analyzer's verdicts.

The condition `x <= y && y != 0` is answered from $\text{less}(\hat{y}, \hat{x})$ and $\text{eq}\big(\hat{y}, \hat{0}\big)$ alone. In Sign, with x non-positive and y positive, both comparisons give a definite *false*, both negations give *true*, and so does the conjunction (Figure 5.6).

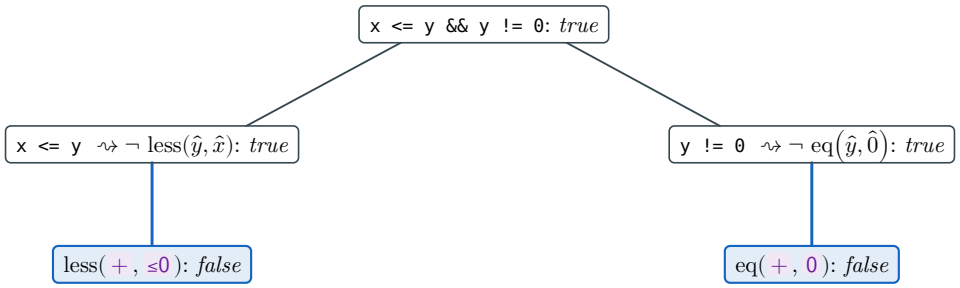


Figure 5.6: How `check_query` answers a compound condition, in Sign with `sign: x=≤0, y=+` (claim `dom-query-tree-sign`). Each comparison becomes one of the domain's two comparisons (blue), as ⇝ marks; the answer follows the colon. Negation flips a definite answer, and `&&` is true because both operands are, so the check is PROVED.

5.5. Learning from a guard thy

When the analysis enters a branch, it knows whether the condition held. On the true arm of `if (0 < x)`, for example, x is positive. Ignoring this knowledge is sound, since the state before the branch already contains every store that takes either arm. It costs precision, however, as the following program shows.

```
1 x = __voblint_nondet_int();                                    VIMP ↗
2 if (0 < x) {
3   y = x;
4 } else {
5   y = - x;
6 }
7 __voblint_check(y >= 0);
```

In this program y ends up as $|x|$, so the check always holds. Without refinement, both arms keep $x \mapsto \top$, y becomes $\top$, and the state cannot show that the check holds. Backward refinement runs the condition in reverse, as in the backward abstract interpretation of tests by Cousot [73, §§8.5, 10.1] and the backward analysis of Nipkow and Klein [6, §13.7]. Given abstract operands a_1, a_2, and the result an operation must produce, an inverse operator returns refined operands a_1', a_2' that keep every concrete pair producing that result. For a comparison that must yield r:

$$n_1 \in \gamma(a_1) \wedge n_2 \in \gamma(a_2) \wedge (n_1 < n_2) = r \implies n_1 \in \gamma(a_1') \wedge n_2 \in \gamma(a_2'),$$

and likewise for equality, addition, subtraction, and multiplication (`sound_inverse_ops`).

The refined operand is then combined with the value known before. Nipkow and Klein use the lattice meet for this and require it to be exact on the denoted sets. Voblint asks for less, namely an intersection that keeps every integer both operands share and lies below both (`sound_intersection`, shown in Section 2.8). For Interval it is the normalized meet `intersect_ivl`, which agrees with the meet on the operands below. In the program above, Interval refines x to $\top \sqcap [1, +\infty] = [1, +\infty]$ on the true arm and to $[-\infty, 0]$ on the false arm. Both arms then give y a non-negative value, and the join yields `interval: y=[0,+∞]`.

The guard filter `bfilter` takes a condition, the required truth value and a state, and applies the inverse operators along the structure of the condition. For the arithmetic operands of a comparison it calls `afilter`, which refines a state so that an expression

evaluates within a required value [6, §13.7.1]. Its soundness is exactly what the branch needs, since it never removes a store that satisfies the guard. The filter refines a conjunction from right to left. Each conjunct is applied once, to the state the conjuncts on its right have refined, and the filter does not iterate to a local fixpoint. The bounds that reach each variable can therefore depend on the order of the conjuncts. With Interval, `0 < i && i < j && j < 10` bounds i to $[1, 8]$ but not j from below, and the reversed order bounds j to $[2, 9]$ but not i from above (claim dom-conjunct-order).

Lemma 5.2 (Sound guard filter, bfilter_sound)**.** *If a store s lies in $\gamma(d)$ and the guard e has truth value* `res` *at s, then s lies in the meaning of the filtered state.*

```
(* src/Abstract_Interpreter/Domain/Eval/Backward_Domain.thy *)                Isabelle
lemma bfilter_sound [intro]:
  assumes "s ∈ γ d" "truthy (⟦e⟧ₑ s) = res"
  shows "s ∈ γ (bfilter e res d)"
```

The shipped inverse operators are sound but not the most precise possible. Interval and Sign, for instance, do not invert arithmetic and cannot learn $x < 4$ from `x + 1 < 5`, as the interval analysis of Nipkow and Klein does [6, §13.8.3].

The branch transfer sound_refinement.branch first drops an arm that the truth test rules out, and otherwise applies the filter. Where it joins the arms of a disjunction, it uses bfilter_lifted, which turns each empty arm into Bot first (Section 5.2.1). The branch inherits the filter's soundness (sound_refinement.branch_sound).

A relational state learns from a guard in the same way, but what it learns is a relation. On the true arm of `if (x < y)`, the relational state of Section 5.2.2 adds the pair $x \leq y$ (assume_step), the meet of its current value with RelC$\{x \leq y\}$. The carrier stores only weak orders, so the strictness of $x < y$ is lost, which is sound but less precise. The relational refinement relc_branch_step satisfies the same statement as the filter (relc_branch_step_sound).

5.6. The complete domain interface thy

The operations of this chapter fall into two groups. The *carrier operations* serve the solver: the order, the join, bottom and top, widening and narrowing, the emptiness test, and the printer (Section 5.1). The *primitives* mirror the program's operations: evaluation and the truth test (Section 5.3), the two comparisons (Section 5.4), and the inverse operators and the intersection (Section 5.5). Every non-relational domain hands its primitives over as one record of type nonrelational_ops, for Interval ivl_ops, and proves them sound once by interpreting sound_nonrelational_ops (Figure 5.7).

The record contains no transfer function, because the primitives determine every edge. An assignment `x = e` overwrites x with the abstract value of e, and a skip leaves the state unchanged. A branch is the backward refinement of Section 5.5, built from the evaluator, the truth test, and the inverse operators. Procedure entry evaluates the arguments in the caller's state, sets every variable of the callee to $\top$, and binds the formal parameters. A new non-relational domain therefore supplies values and primitives only, and its interpretation of sound_nonrelational_ops proves all of these transfers sound at once.

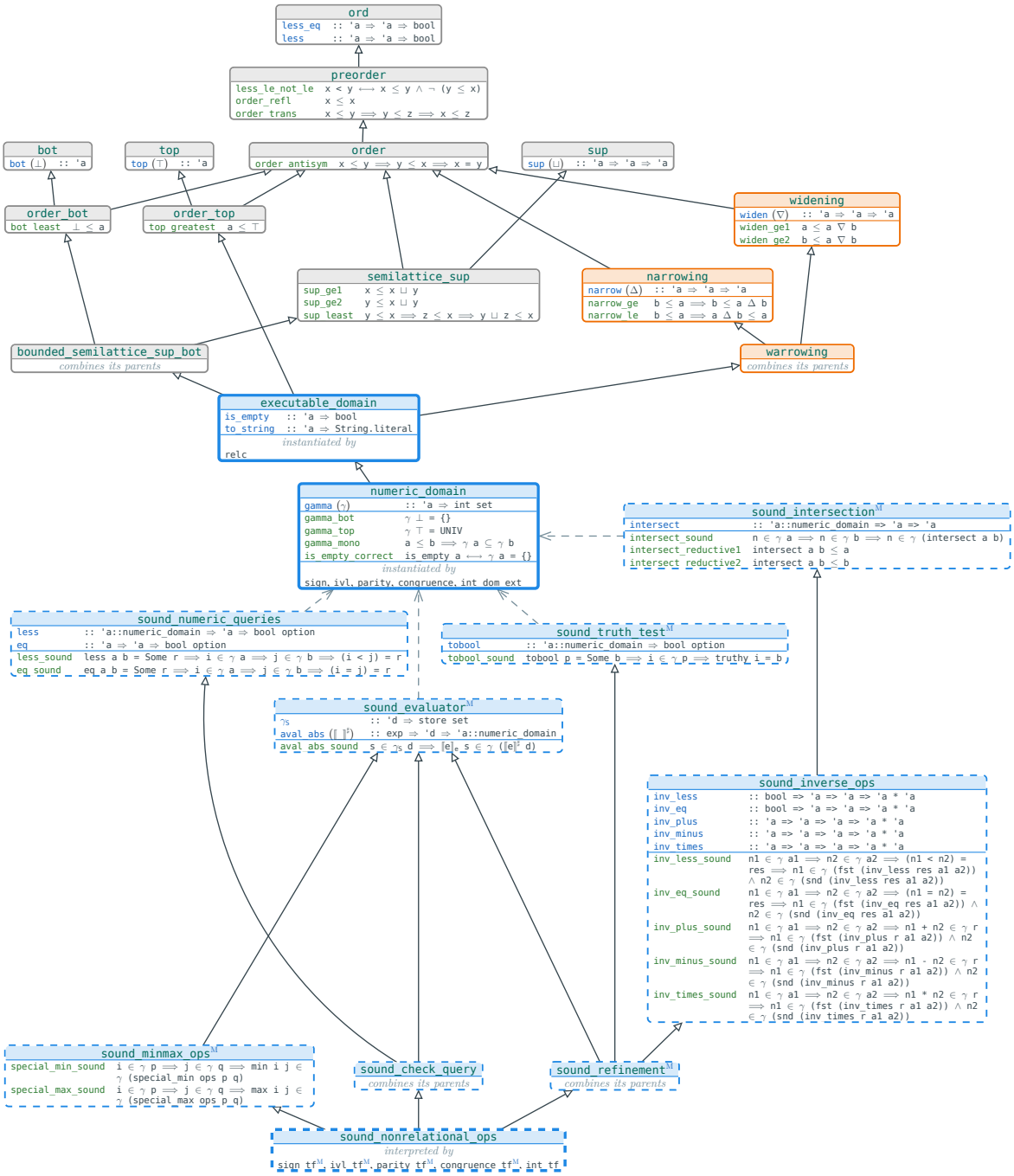


Figure 5.7: What a non-relational domain supplies over its carrier type `'a`, down to the soundness locale `sound_nonrelational_ops` an analysis interprets. Each class (solid) or locale (dashed) lists the operations and laws it declares, which the declarations below it inherit. Solid arrows point to what a declaration extends, dashed ones to the class its type variable is constrained to. An interface an analysis implements has a heavier border and lists its instances. A superscript ^{M} marks an interface with a monotone strengthening, and an interpretation that proves it (`mono_nonrelational_ops`). Color gives where a node is declared: ☐ HOL, ☐ the vendored solver, ☐ Voblint.

5.7. A carrier the interface excludes

Voblint cannot host every domain Goblint offers. The verified solver requires its values to form a `bounded_semilattice_sup_bot` (`TD_side_upd_rule`), and the generic framework inherits the requirement. That class asks for the *least* upper bound (`semilattice_sup`), and the proofs use it. Reductive refinement bounds the join of two disjunction arms by the state it started from (`sound_refinement.bfilter_reductive`). This requirement excludes every carrier in which two values have upper bounds but no least one.

Goblint's exclusion-set domain `DefExc` is such a carrier once its bit range is removed. A non-bottom value is `Definite n`, denoting $\{n\}$, or `Excluded (S, r)`, denoting the integers in the bit range r outside the finite set S. Mathematical integers have no bit range, so `Excluded S` denotes $\mathbb{Z} \setminus S$ (Figure 5.8). Then `Definite 1` and `Definite 2` have no least upper bound. Every `Excluded {p}` with $p \notin \{1, 2\}$ lies above both. A least upper bound u would lie below each of them, so by monotonicity of γ

$$\{1, 2\} \subseteq \gamma(u) \subseteq \bigcap_{p \notin \{1,2\}} (\mathbb{Z} \setminus \{p\}) = \{1, 2\},$$

but no value denotes exactly $\{1, 2\}$, since `Definite n` denotes a single integer and `Excluded S` all but finitely many.

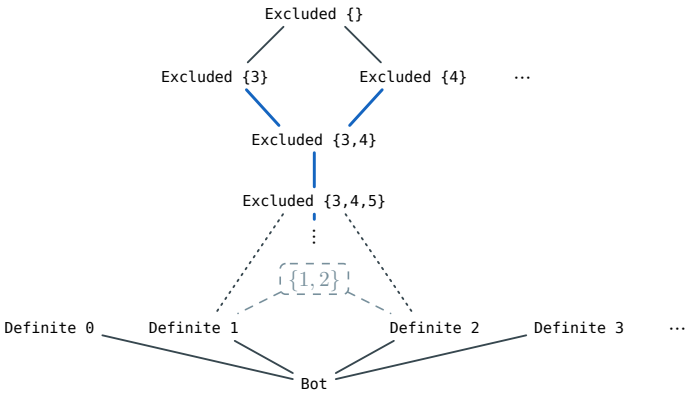


Figure 5.8: Goblint's exclusion sets without a bit range, ordered bottom to top: `Definite n` denotes $\{n\}$ and `Excluded S` denotes $\mathbb{Z} \setminus S$. Every `Excluded S` with $1, 2 \notin S$ lies above `Definite 1` and `Definite 2`. The blue chain of such upper bounds descends forever, and each of its elements lies above both values (dotted). A least upper bound would denote $\{1, 2\}$ (dashed), which no value does. Illustrative, not machine-checked.

Goblint's domain escapes this argument through its bit range. With a range r, the value `Excluded (`$r \setminus \{1, 2\}$`, r)` denotes exactly $\{1, 2\}$. Its join does not return this value anyway. Joining two distinct `Definite` values yields `Excluded ({0}, r)`, or excludes nothing if one of them is 0. Voblint could admit this carrier only by bounding its integers, or by weakening `semilattice_sup` to an upper-bound law throughout the solver and the framework.

The domain interface is now complete. Its laws make every operation keep the stores its inputs admit, but each law speaks only about one value or one state. A program moves its state along edges and through calls, and Chapter 6 states what an analysis must prove there.

6. What an Analysis Supplies

Chapter 5 ended with laws about one abstract state. Its order agrees with its meaning, and its operations over-approximate the concrete ones. The obligations of Chapter 4, however, speak about edges, calls, and contexts. Most of what connects the two is the same for every analysis, namely the equations built from the control-flow graph, the calling contexts, and the solver. Sign and Interval differ only in the abstract effect of each program construct. Goblint therefore separates an analysis from the framework that runs it [2, §6], and Voblint makes the same split in its program and in its proof.

This chapter describes what an analysis must supply, and what it must prove about it, so that the framework can discharge the obligations of Chapter 4 once for every analysis. An analysis supplies a record of abstract operations, and the analysis contract states what they preserve.

6.1. Transfers at edges and calls

An ordinary edge of the control-flow graph carries an action a, such as an assignment or a guard, whose concrete meaning `edge_step` is fixed by Chapter 3. For a store s, `edge_step a s` is the set of stores the edge can produce. An analysis describes the stores at a program point by an abstract state d of its *local domain* D_L (Section 6.2) and supplies, for each action a, an abstract transfer $[\![a]\!]^\sharp$. The transfer is sound if it loses no successor:

$$s \in \gamma_D(d) \land s' \in \texttt{edge_step a s} \Longrightarrow s' \in \gamma_D([\![a]\!]^\sharp\, d).$$

This is the abstract counterpart of the obligation INTRA of Chapter 4 for one edge. Unlike Saan's abstract operators [31, §2.4.2], the obligation does not require strictness, $[\![a]\!]^\sharp \bot = \bot$. The interface has one transfer per kind of action, named after the corresponding method of Goblint's `Spec` (Figure 6.2).

A call cannot be handled by one such transfer. After it returns, the caller's own locals are the ones from before the call, while the globals and the result come from the callee (`combine_collect`, RETURN). The abstract return must therefore see both sides. As in Goblint, an analysis supplies two operations at a call. *Enter* (`dgs_enter`) gives the abstract state the callee starts from, and *combine* (`dgs_combine_env`, `dgs_combine_assign`) gives the caller's state after the call. Take one call:

```
1 global g;                                                    VIMP
2 fun inc(a) { g = g + a; return a - 1; }
3 fun main() { g = 1; x = 5; y = inc(x); }
```

Enter describes two stores associated with the call `y = inc(x)`. The *entry state* e describes the store the callee starts with (`g = 1`, `a = 5`). The *resume state* q describes the caller's store (`x = 5`, `g = 1`), which combine later merges with the callee's exit state r (the theories call q the continuation). Figure 6.1 shows q, e, r, and the state combine computes.

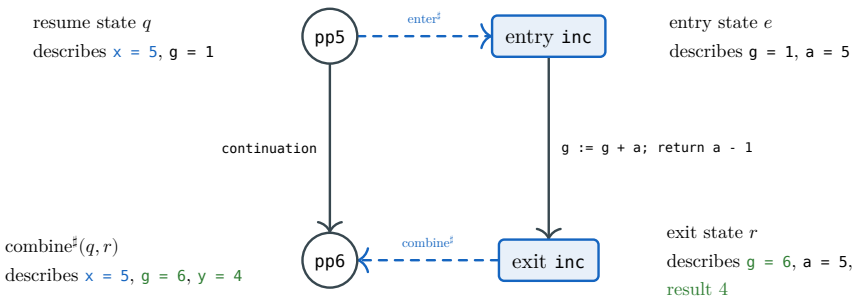


Figure 6.1: Enter and combine at the call `y = inc(x)`, with the concrete store each abstract state must describe (schematic, with the body of `inc` drawn as one edge). The combined state must describe the caller's locals (blue) together with the callee's globals and result (green), as `combine_collect` does.

For soundness, enter must return a pair of a resume state that covers the caller's store and an entry state that covers the entered store (`entry_pairs_cover`). Like Goblint's `enter`, it may return several such pairs. Goblint's path-sensitivity lifter uses this to return one pair per path [74, §5.7]. Every analysis shipped with Voblint returns a single pair, and only a test example of Chapter 4 returns two, so path-sensitive analyses remain future work.

6.2. Facts without a program point thy

Not every fact belongs to one program point. An analysis may keep, for example, one range for a global variable `g` that holds throughout the run. Seidl et al. describe a Goblint analysis with two domains [2, §6]. The local domain D_L holds facts per program point and context, and D_G holds the values of *analysis globals*, which are unknowns without a program point. An assignment to `g` publishes its value to the analysis global of `g` as a side effect, and an edge that reads `g` depends on that unknown. Goblint's `Spec` accordingly declares a local lattice `D` and a global lattice `G`.

The word *global* thus has two meanings. A *program global* is a variable of the VIMP program. An analysis global is a fact the analysis keeps flow-insensitively. The two are independent. An analysis may track a program global in its local state, and an analysis global does not have to stand for any program variable. By default, the numeric analyses keep program globals in the local state and use no analysis global (Section 6.4).

A transfer that uses analysis globals still returns the next local state, but it may also read and publish them on the way. In Goblint, it does this through a *manager* argument. The manager gives the transfer the current local state and operations to read and publish analysis globals. For example, `man.sideg g d` publishes `d` to the analysis global `g`, and the framework passes such publications to the solver [2, §6]. Voblint adopts this design. Its transfer (`man_transfer`) receives a manager of type `man` instead of a local state. It reads an analysis global v with `man_global` v and publishes d to it with `man_sideg` v d. As a result, the equations of Chapter 8 depend on an analysis global only where a transfer actually reads it. A transfer that is a pure function of the local state reads only the state at the edge's source and publishes nothing (`sp_compile_transfer_program_local_transfer`). By default, the numeric analyses are of this kind.

6.3. The analysis contract thy

The framework takes an analysis as one parameter, the record dg_spec with one field per operation (Figure 6.2): the edge transfers of Section 6.1, enter, the two combine stages, and a query handler that Chapter 7 uses to let analyses exchange facts.

```
(* src/Abstract_Interpreter/Framework/Spec/DG_Spec.thy *)                                   Isabelle
record ('x,'k,'v,'dl,'dg) dg_spec =
  dgs_skip          :: "('x,'k,'v,'dl,'dg) man_transfer" ("skip♯")
  dgs_assign        :: "vname ⇒ exp ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("assign♯")
  dgs_special       :: "special_call ⇒ vname ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("special♯")
  dgs_branch        :: "exp ⇒ bool ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("branch♯")
  dgs_body          :: "pname ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("body♯")
  dgs_return        :: "exp option ⇒ pname ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("return♯")
  dgs_enter         :: "call_info ⇒ ('x,'k,'v,'dl,'dg) man_enter_transfer" ("enter♯")
  dgs_event         :: "analysis_event ⇒ ('x,'k,'v,'dl,'dg) man_transfer" ("event♯")
  dgs_combine_env   :: "call_info ⇒ ('x,'k,'v,'dl,'dg) man_combine_transfer" ("combine'_env♯")
  dgs_combine_assign :: "call_info ⇒ ('x,'k,'v,'dl,'dg) man_combine_transfer" ("combine'_assign♯")
  dgs_query         :: "('x,'k,'v,'dl,'dg) man_query"
```

Figure 6.2: The declaration of dg_spec, lifted from the theory. Every field except the query handler carries, marked ♯, the name of the corresponding method of Goblint's Spec. A call_info names the callee, its formals, the arguments, and the destination of a call.

The goal is that Chapter 8 can derive the obligations of Chapter 4 for every record that satisfies a fixed set of laws. Most of them form the *analysis contract* analysis_contract. Since a transfer may read analysis globals, a local state alone no longer fixes which stores it describes. The contract therefore uses a concretization $\gamma_{DG}(d, e)$ of a local state d under an *environment* e, which maps every analysis global to its value. The locale states three semantic laws and one technical premise:

```
(* src/Abstract_Interpreter/Framework/Spec/DG_Spec_Sound.thy *)                            Isabelle
locale analysis_contract =
  fixes S :: "('x,'k,'v,'D::bounded_semilattice_sup_bot,
               'G::bounded_semilattice_sup_bot) dg_spec"
    and γDG :: "'D ⇒ ('v ⇒ 'G) ⇒ store set"
    and 𝒢 :: "vname ⇒ bool"
  assumes spec_wf: "dg_spec_wf S"
    and gammaDG_mono:
      "⟦d ≤ d'; e ≤ e'⟧ ⟹ γDG d e ⊆ γDG d' e'"
    and step_sound:
      "edge_collect a (γDG (dg_local (τ src)) (genv key τ))
         ⊆ γDG (edge_out S a src key τ) (genv key τ ⊔ edge_pub S a src key τ)"
    and combine_sound:
      "⟦s ∈ γDG q (genv key τ); t ∈ γDG r (genv key τ)⟧ ⟹
        combine_collect 𝒢 (ci_dst ci) s t
          ∈ γDG (combine_out S ci key q r τ) (genv key τ ⊔ combine_pub S ci key q r τ)"
```

First, τ is an arbitrary valuation of the solver's unknowns, valued in pairs dg_state (Section 9.1.3), and key maps each analysis global to its unknown. Thus d is the local half dg_local of τ at the edge's source, and e is genv key τ, the global halves at those unknowns. Second, edge_out and edge_pub are the local state a transfer returns and the environment of what it publishes, which is $\bot$ at every unknown it does not publish to. combine_out and combine_pub are the same for combine, whose resume state is q and whose exit state is r. Third, $\mathcal{G}$ marks the program globals, which combine_collect takes from the callee.

analysis_contract.gammaDG_mono requires that a larger state or environment describe more stores. This turns the solver's inequalities between values into inclusions between sets of stores, as γ does for a domain (Chapter 5). analysis_contract.step_sound is the abstract counterpart of INTRA. Every store the edge produces from a store described under

e is described by the returned state under e joined with the publications. A transfer that publishes nothing thus meets the condition of Section 6.1 with e as a fixed parameter. `analysis_contract.combine_sound` is the counterpart of RETURN in the same shape. Finally, `dg_spec_wf` requires that the program of every operation run its continuation exactly once, so what it publishes is well defined. Beyond these laws, the contract asks the domains only for a join semilattice with a least element, and never for a map from variables to abstract values (Section 11.4).

The contract supplies the context-independent core of INTRA and RETURN. CALL requires that the callee's entry covers the entered store *in every context the policy admits for the call*. Entry coverage (`entry_pairs_cover`) provides the covering pair, but the routing policy chooses the context in which it is analyzed, so Chapter 8 combines the two. TOTAL, that every covered call reaches some context, follows from the policy, and for entry-state contexts also from entry coverage (Section 8.2). INIT requires the analysis's initial state d_0 to cover the initial stores, which the analysis discharges when it is registered (`dg_analysis.init_sound`).

6.4. Program globals: flow-sensitive or flow-insensitive thy

The contract leaves open where program globals are kept. By default, program globals are part of the local state, as in Goblint's default base analysis for single-threaded programs (base.ml, [31, §5.6.1]). The other placement keeps each program global at an analysis global of its own, written by side effects and read where it is used. The analysis is then *mixed flow-sensitive*, flow-sensitive in the locals and flow-insensitive in the globals.

```
1 global Gx;                                                    VIMP ↗
2 fun set() { Gx = 1; }
3 fun get() { return Gx; }
4 fun main() { x = 1; set(); y = get(); }
```

The flow-insensitive value of `Gx` must cover the initial `Gx = 0` as well as the 1 written by `set`. Sign therefore claims only ≥0 for `y` after `y = get()` (claim `mixed-flow-sign-shared`), although every run ends with `y = 1`. Flow-sensitively, Sign carries + from `set`'s exit into `get`'s entry and derives + (claim `mixed-flow-sign`). In sequential VIMP the flow-insensitive placement can thus cost precision. A guard on such a global refines nothing, since the guard writes no global, as in Goblint's non-relational thread-modular analyses [31, §5.6.2]. The placement suits multithreaded programs, where the simplest thread-modular analysis keeps shared globals flow-insensitively [2, §1, Ex. 4], [31, §5.3.1]. VIMP has no threads, but an extension with threads could build on it (Section 16.3.1).

Like Goblint and Apinis et al. [2, §3], [43, §5], the analyzer keeps one unknown per program global, so a write that changes `g` destabilizes only the right-hand sides that read `g` and those that depend on them. `keyed_split_contract` shows that this placement meets the analysis contract whenever the wrapped analysis is sound and the domain satisfies monotonicity laws and a frame law for the globals a transfer does not assign, which the analyzer's combined state does (`mcp_keyed_dg_analysis`), so `run_voblint_source_sound` covers it.

The interface is modular, since an analysis proves facts about its own operations only, while context handling, equation construction, and solving are implemented and proved once by the framework. Chapter 7 combines several analyses.

7. How Analyses Cooperate

Chapter 6 fixed what one analysis supplies, a record of operations whose soundness is proved operation by operation. Its transfers, however, see only the analysis's own state, while a check may need a fact that one analysis knows and another must use. In the program below both guards hold at the assignment, so $x = y$ and `z` is 1.

```
fun main() {                                                    VIMP ↗
  x = __voblint_nondet_int();
  y = __voblint_nondet_int();
  if (x <= y) {
    if (y <= x) {
      z = (x == y);
      __voblint_check(z == 1);
    }
  }
}
```

Interval keeps one range per variable. After the guards both `x` and `y` are still unbounded, so `x == y` evaluates to $[0, 1]$ and the check stays `UNKNOWN` with `interval: z=[0,1]`. The order analysis is relational. Its state is a set of pairs (x, y), each meaning that the value of x is at most the value of y. After the guards it holds (x, y) and (y, x), so it knows that $x = y$, but it tracks no value of `z` and reports `UNKNOWN` too. Neither analysis can decide the check. Together they can, if Interval's assignment can learn what the order analysis knows at that point (Figure 7.1).

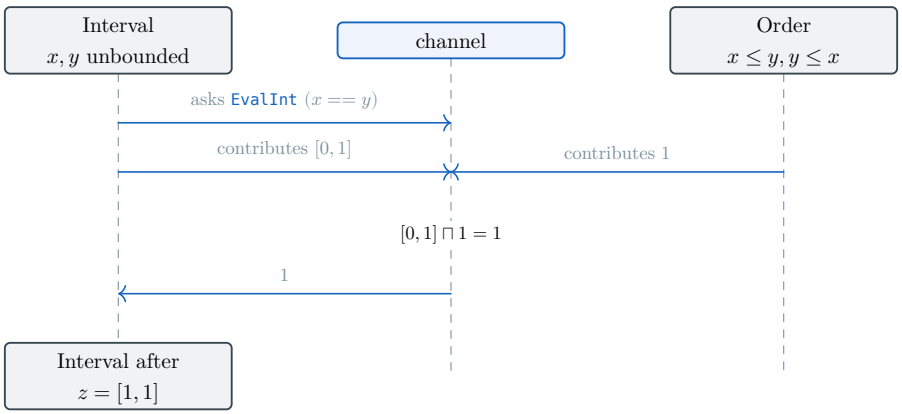


Figure 7.1: One step of Interval and the order analysis at `z = (x == y)` in the program above (schematic). Interval's assignment asks `EvalInt` $(x == y)$. The channel asks the handler of every analysis and gives Interval the meet of their contributions. Each analysis updates only its own part of the state. The final states match the analyzer run `PROVED` with `interval: z=[1,1]; order: {x≤y ∧ y≤x}`.

Goblint solves this with its *master control program* (MCP). The activated analyses run side by side, each on its own part of one combined state, and a transfer may ask all activated analyses a question through the manager's `ask` function. Goblint has no machine-checked argument that these exchanges are sound. A proof, however, has a modularity problem. If Interval's assignment is proved sound using the order analysis's invariant, the two proofs are coupled, every new pair of analyses needs a new proof, and adding an analysis reopens old ones. This chapter describes how each analysis instead gets one proof obligation that does not mention its partners, and why analyses meeting it combine into one analysis that meets the analysis contract of Section 6.3.

7.1. Questions and answers thy

The analyses must agree on what a question means and on what an answer claims. A *query* asks for a fact about the stores a state describes. Voblint has one kind, named after Goblint's. The query `EvalInt` e asks which integers the expression e may evaluate to. Its answer is a value of the reduced product Int (`int_dom`, Section 11.3), lifted by a bottom and by a top element that claims nothing (`query_lift`). A comparison evaluates to 0 or 1, so the exact answer 1 to `EvalInt` $(x == y)$ says that $x = y$.

Definition 7.1 (Truth of an answer, `fun eval_holds`)**.** An answer a to `EvalInt` e *holds* at a store s if $[\![e]\!]_e s \in \gamma(a)$:

Isabelle
```
(* src/Abstract_Interpreter/Framework/Cooperation/Analysis_Query.thy *)
fun eval_holds :: "query ⇒ answer ⇒ store ⇒ bool" where
  "eval_holds (EvalInt e) a s ⟷ ⟦e⟧e s ∈ gamma_answer a"
```

Two laws, stated by the locale `query_algebra`, let answers from several analyses work together:

Isabelle
```
(* src/Abstract_Interpreter/Framework/Cooperation/Analysis_Query.thy *)
locale query_algebra =
  fixes answer_holds :: "'q ⇒ 'r::{semilattice_inf, order_top} ⇒ store ⇒ bool"
  assumes top_sound: "answer_holds q ⊤ s"
      and inf_sound: "⟦answer_holds q a s; answer_holds q b s⟧ ⟹ answer_holds q (a ⊓ b) s"
```

The top answer $\top$ holds at every store, so an analysis that cannot answer a question answers $\top$ and claims nothing. If two answers hold at a store, so does their meet, so a question asked of all analyses can be answered by the meet of their answers. In the example, when asked `EvalInt` $(x == y)$ after both guards, Interval answers $[0, 1]$, which holds at every store, and the order analysis answers 1. Their meet is $[0, 1] \sqcap 1 = 1$, so $x = y$. Goblint's `MCP.query'` combines answers the same way, as formalized by Apinis [74, §5.5].

A transfer may ask several questions. The framework therefore hands it all answers at once, as a *channel* ch of type `channel`, a function from queries to answers. A channel *holds* at a store s if every answer it gives holds at s:

Isabelle
```
(* src/Abstract_Interpreter/Framework/Cooperation/Analysis_Query.thy *)
definition channel_holds :: "('q ⇒ 'r) ⇒ store ⇒ bool" where
  "channel_holds ch s ⟷ (∀q. answer_holds q (ch q) s)"
```

By the two laws, the channel answering $\top$ everywhere holds at every store, and the pointwise meet of two channels that hold at s holds at s (`query_algebra.channel_holds_inf`).

7.2. Proving an analysis against any channel thy

Interval's assignment in Figure 7.1 uses the wrapper `ask_assign`, which any analysis can put around its assignment $x := e$. The wrapper asks `EvalInt` e. When the answer is one integer n, it runs the analysis's own assignment of the literal n, and otherwise the assignment of e. At $z := (x == y)$ Interval receives 1 and sets z to $[1, 1]$, which is right only if the answer is true. Proving that from the order analysis's invariant would couple the two proofs.

Voblint instead makes the channel an argument of the transfer and the truth of its answers a premise of the proof. The transfer `ls_step` $c\,ch\,a\,x$ of an analysis c receives a channel ch besides the action a and the value x before the edge. Soundness is the condition of Section 6.1 with one extra premise. For every channel ch, every $s \in \gamma(x)$ *at which* ch *holds*, and every $s' \in$ `edge_step a s`, we require $s' \in \gamma($`ls_step` $c\,ch\,a\,x)$, and `ls_step_sound_iff` splits this into one law per kind of edge. In the example, the premise excludes the stores where $x \neq y$, at which the answer 1 is false. At the others e evaluates to 1, so the analysis's soundness for the literal 1 gives soundness for e. One theorem covers every analysis (`ask_assign_sound`). Its proof uses only that the answer holds, so it works with every partner whose answers hold.

An analysis without analysis globals is written as a *local specification* (`local_spec`, Figure 7.2), whose operations each take a channel, and whose handler `ls_query` answers queries about a state.

Soundness of a local specification (`sound_local_spec`) asks for a monotone concretization, which turns the solver's inequalities into inclusions as in Section 6.3, and one law per operation, with one law for the two combine stages together (`sound_local_spec_iff`). The edge and combine laws are the counterparts of the step and combine laws of the analysis contract (Section 6.3), and the enter law matches entry coverage, each assuming that its channels hold. The new handler law requires the handler's answers about a state x to hold at every store of $\gamma(x)$ at which the channel it receives holds (`sound_query`). It is the analysis's promise to its partners, and the only one they rely on. Beyond the analysis contract, a sound local specification thus also contains entry coverage and the handler law.

The idea is not new. In Astrée an abstract transfer only has to cover concrete arguments and results that satisfy the constraints of its channels [19, §§5.3, 6]. Verasco, whose channels are inspired by Astrée's and whose term we adopt, proves each transfer function under the hypothesis that the channels it receives are correct [33, §7]. Voblint uses the same form for Goblint's queries.

```
(* src/Abstract_Interpreter/Framework/Cooperation/MCP_Spec.thy *)                Isabelle
record 's local_spec =
  ls_query :: "channel ⇒ 's ⇒ channel"
  ls_skip :: "channel ⇒ 's ⇒ 's"
  ls_assign :: "channel ⇒ vname ⇒ exp ⇒ 's ⇒ 's"
  ls_special :: "channel ⇒ special_call ⇒ vname ⇒ 's ⇒ 's"
  ls_branch :: "channel ⇒ exp ⇒ bool ⇒ 's ⇒ 's"
  ls_body :: "channel ⇒ pname ⇒ 's ⇒ 's"
  ls_return :: "channel ⇒ exp option ⇒ pname ⇒ 's ⇒ 's"
  ls_event :: "channel ⇒ analysis_event ⇒ 's ⇒ 's"
  ls_enter :: "channel ⇒ call_info ⇒ 's × 's ⇒ ('s × 's) list"
  ls_combine_env :: "channel ⇒ channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
  ls_combine_assign :: "channel ⇒ call_info ⇒ 's ⇒ 's ⇒ 's"
```

Figure 7.2: The declaration of `local_spec`, lifted from the theory. Every operation receives a channel of type `channel`, and the first combine stage receives two.

7.3. Many analyses over one state thy

The analyses cannot be solved one after another. In the opening example Interval needs the order analysis's facts at `z = (x == y)`, and in the example of Section 13.3 the order analysis needs Interval's, at the same program point and context. The framework therefore runs all selected analyses as a single analysis whose abstract state is a tuple with one field per analysis. At `z = (x == y)` this state is the pair of Interval's state, in which x and y are unbounded, and the order analysis's state $\{x \le y, y \le x\}$. The solver sees one value per node and context and never looks inside it. The tuple is ordered, joined, and widened field by field (`analysis_product`), so each field evolves as it would alone, except that its transfers may ask the other fields. If one selected field shows that a node is unreachable, the whole tuple does (`mcp_norm`).

The combined state `mcp_st` has one field per analysis the analyzer ships, and a run uses the fields of the analyses the user selects (Figure 7.3). The tuple describes a store if every selected field does:

```
(* src/Abstract_Interpreter/Framework/Cooperation/MCP_Spec.thy *)                Isabelle
definition mcp_gamma :: "('s ⇒ store set) list ⇒ 's ⇒ store set" where
  "mcp_gamma Γ x = (⋂γi ∈ set Γ. γi x)"
```

Here Γ is the list of concretizations of the selected fields. Each $\gamma_i \in \Gamma$ reads its own field of x, so $\gamma_i(x)$ is the meaning that field contributes.

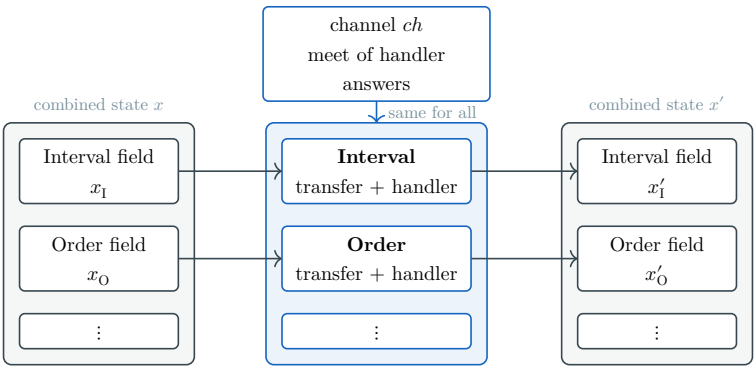


Figure 7.3: One step of the combination (schematic). Each component reads its own field of the combined state x and writes its own field of x', and the components run in turn. All components receive the same channel ch (blue), which the framework builds from x as the meet of the answers of all handlers.

We call a local specification that runs on one field a *component*. Each component reads and writes only its own field, through a *lens* (`lens_of`) that takes the field out of the combined state and puts a new value back, as Goblint's MCP hands each analysis its own part of the combined state. A component therefore leaves the concretizations of all other fields unchanged (`field_frame`), which makes the components *independent* (`mcp_independent`). `mcp_combine` runs the components in turn, gives every one the same channel, and meets the answers of their handlers. If the list of components is nonempty, the components are independent, and each is a sound local specification, then the combination is a sound local specification for the intersection of their concretizations (`mcp_combine_sound`). `mcp_comp_sound` applies this to every nonempty selection of distinct analyses. The components cannot invalidate one another's guarantees, since each guarantee is about a field no

other component writes, and it holds for any channel that holds, whichever component answered.

The premise that the channel holds leaves one task to the framework. For every state x it must build a channel that holds at every store of $\gamma(x)$. It asks the combined handler, whose components may ask in turn, and answers with $\top$ any query that is asked again while it is being answered, as Goblint's `MCP.query'` does. Every answer of this channel holds (`ls_channel_sound`). `dg_spec_of` passes this channel to the operations and turns a sound local specification into a record that meets the analysis contract (`dg_spec_of_contract`). The combination of any nonempty selection of distinct analyses therefore meets the analysis contract, and Chapter 8 treats it like a single analysis.

Checks are decided from the same kind of answers. At a check with condition b, every active analysis answers the question `EvalInt` about the truth value of b from its own part of the solved state, and the analyzer meets these answers (`mcp_answer`). An exact 1 proves the check, and an exact 0 refutes it (`mcp_classify`). Several analyses together can thus decide a check that each alone leaves unknown, as the run `PROVED` of the opening example shows. The exchange also works in the other direction, with the order analysis asking Interval (Section 13.3).

7.4. What a new analysis has to prove

A new analysis joins the combined state by proving the laws of its local specification, which mention no other analysis. Its field, the lens, and the code that reads its answers are generated from its entry in the analysis manifest (Chapter 14), so only its operations and their proofs are written by hand. There are two routes to these proofs. A numeric domain proves its primitive operations sound once (`sound_nonrelational_ops`, Section 11.1), and the framework derives the laws from that proof (`sound_nonrelational_ops.dg_analysis_execI`). Any other analysis starts from conservative defaults, a handler that answers $\top$ and operations that keep the state at skips, guards, procedure bodies, events, and the first combine stage. These defaults are sound for every concretization (`sound_conservative_local_spec`). The analysis then supplies and proves the assignment, the special calls, the return, enter, and the second combine stage, and proves each operation that replaces a default. The order analysis follows this route. It answers comparisons between variables it has ordered, asks at assignments, and enters and returns with no facts (`order_spec_sound`).

Two limits remain. Components cannot read or publish analysis globals (Section 6.2), which Goblint's MCP allows by tagging each analysis's globals with its index. No theorem guarantees that the combination is more precise than its components. The opening program and the one in Section 13.3 only show concrete cases where it is.

Each analysis is proved sound against every channel that holds, and the framework supplies such a channel and runs each analysis on its own field. The proofs of the analyses therefore stay independent, and adding an analysis needs one new proof and reopens none.

Chapter 5 gave the laws of one abstract state, Chapter 6 the contract of one analysis, and this chapter showed that a selection of analyses meets the same contract as one. None of these chapters mentions contexts, equations, or a solver. Chapter 8 builds an equation system over nodes and contexts from the operations of the combined analysis and discharges the obligations of Chapter 4 for it.

PART IV

From Analyses to Verdicts

8. Equations, Contexts, and Routing

Chapter 4 reduced soundness to a *claim*, which assigns to every node v and context c a set of stores `cover v c`, and stated five local obligations under which the claim contains every store that an execution brings to v in context c. Chapter 6 and Chapter 7 stated what an analysis supplies and proves. What is still missing is a way to compute such a claim. This chapter turns the procedure-aware graph of Chapter 3 into a system of equations and shows that every post-solution induces a sound claim.

Inside a procedure the construction is direct, while calls need more care. A call enters a context of the callee. Under some context policies, that context depends on the abstract entry state computed from the caller, which becomes known only while the equations are solved. Formally, the goal is that every post-solution σ satisfies

$$\mathcal{A}_{\mathcal{G},\mathtt{adm},\mathtt{c_0},\mathtt{g},\mathtt{S}}\ \mathtt{v}\ \mathtt{c} \subseteq \gamma_M(\sigma(v,c))$$

at every node v and context c. The left side is the activation collecting semantics of Section 4.3, and $\gamma_M(\sigma(v,c))$ is the set of stores that the solved abstract state at (v,c) describes under the solved values of the analysis globals (Section 6.3). The running example is the program of Figure 3.1.

8.1. Unknowns and right-hand sides thy

The running example calls `bump` first with 5 and then with 4. If both activations shared one abstract state at the entry of `bump`, that state would have to contain both values, the return would merge the two results, and the analysis could no longer show that the first result is exactly 6. Contexts avoid this merge by giving the two activations separate copies of `bump`.

Voblint therefore indexes the unknowns of the equations by contexts. For every node v and context c there is a *local* unknown (v,c), whose value $\sigma(v,c)$ is the abstract state for the activations of context c at v. These are the indices of the claim. Goblint's local unknowns have the same shape [74, §5.2]. Besides the local unknowns, the equations have *global* unknowns without a program point (Section 2.7). They hold the analysis globals of Section 6.2 and the entry states that calls publish to their callees.

The equation of (v,c) has to collect every abstract state that can reach v in context c, because a post-solution must cover every store that reaches v there. Such states come from four sources: the initial state at the program entry, the transfer along each local edge into v, the result of each call that returns to v, and, at a procedure entry, the entry states that calls publish to it. An entry state is the abstract state in which the callee starts, for example with its formal parameters bound to the abstract values of the call's arguments (Section 6.1). Along an edge inside a procedure, the activation keeps its context (Section 4.3), so the second source applies the transfer to the predecessor's value in the same context c. A callee, however, runs in a context of its own.

The generated right-hand side `routed_node_rhs` computes exactly the join of these four sources under a valuation τ:

```
(* src/Abstract_Interpreter/Framework/Constraints/DG_Indexed_Generator.thy *)
lemma routed_node_rhs_parts:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                       extra g c w). sp_wf p"
  shows
  "eq (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, c) τ =
   DG (rhs_init g bot0 d₀ v
       ⊔ (⨆(u, a)←pred_sel g v c. rhs_edge it τ c u a)
       ⊔ (⨆(u, ca)←site_sel g v. rhs_call cmb route τ c u ca v)
       ⊔ rhs_seed extra route τ c v) bot"
```

Here $\bigsqcup_{x \leftarrow xs} f(x)$ joins $f(x)$ over the list xs, and the empty join is $\bot$. The lists range over the local edges into v and over the calls that return to v. The parts `rhs_init`, `rhs_edge`, `rhs_call`, and `rhs_seed` are the four sources in this order. Publications to analysis globals (Section 6.2) are not part of this value. A post-solution lies above it, and γ_M is monotone, so $\gamma_{M(\sigma(v,c))}$ contains what each source describes.

At a call, the analyzer chooses the callee's context with a *routing* function $\text{context}^\sharp(u, c, e)$ of the call node u, the caller context c, and the entry state e. Under entry-state contexts the choice depends on e, which is known only while the equations are solved. The set of calls that contribute to a particular callee context is therefore not known in advance, and the callee's entry cannot list them in its equation. Each call instead publishes its entry state to the callee as a side effect (Section 2.6).

Figure 8.1 follows the first call of the running example. The equation of its continuation, $(pp3, c_0)$, connects the caller and the callee. It reads the caller's value at $(pp2, c_0)$ and applies enter (Section 6.1), which yields an *entry pair* of a resume state q_1 and an entry state e_1 that binds n to $[5, 5]$. The entry state selects the callee context $c_1 = [[5, 5]]$. The equation publishes e_1 to that context, reads the result r_1 of `bump` in c_1, and joins $\text{combine}^\sharp(q_1, r_1)$ into its own value. The equation of the second call's continuation, $(pp4, c_0)$, does the same

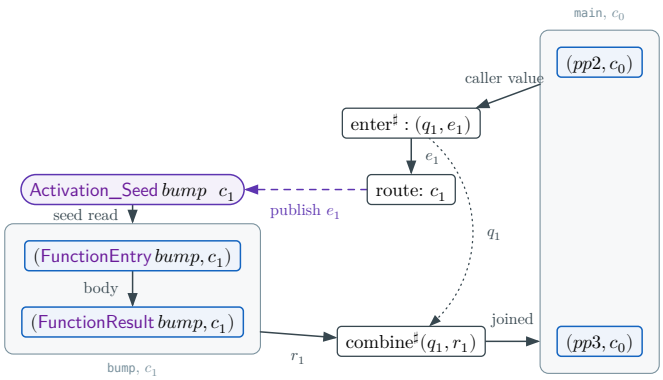


Figure 8.1: The first call of the running example under entry-state contexts. The right-hand side of the continuation $(pp3, c_0)$ reads the caller's value, applies enter, routes the entry state e_1 to c_1, publishes it to the seed of `bump` in c_1 (purple, dashed), reads the result in the same context and joins the combined state into its own value. Blue boxes are local unknowns, the purple box is a global unknown, white boxes are steps of the right-hand side, and gray frames group the unknowns of one procedure in one context.

with $c_2 = [[4, 4]]$. The two calls thus use separate copies of `bump`, and the first result stays exactly 6.

A caller's equation publishes to the callee's seed and reads the callee's result, and it reads no other unknown of the callee. A recursive call is therefore an ordinary call. Its equation reads the result of the callee in some context, which creates a cycle of dependencies between unknowns. The solver treats this cycle like the cycle of a loop, and the unknown at which a read closes it becomes a widening point (Section 2.7). The explainer's solver replay animates this protocol.

Formally, a call publishes its entry state to a *seed* Activation_Seed $p\ c'$, one global unknown (`global_unknown`) per callee entry and context. Every call routed to (p, c') publishes to this seed, and the equation of the entry (FunctionEntry p, c') reads it. In general, a call may enter several callees, and enter may return several entry pairs. Each of them is routed separately.

If the entry state is bottom, no store enters the callee, so the call publishes nothing and does not read the callee's result. Without this test, the read would make the solver evaluate the callee's body from an entry state that describes no store. Goblint's normal-call transfer skips bottom entry states in the same way. The optimization is sound because a state classified as bottom is required to have an empty concretization (`is_bot_sound`).

8.2. Routing and context policies thy

The analyzer chooses a callee context from abstract information. The concrete semantics of Chapter 4 also assigns each concrete activation a set of contexts, through the context policy *adm* of Section 4.3, and this set may depend on concrete stores that the analyzer never sees. For the equations to be sound, the two choices must fit together. Whenever the concrete semantics assigns an activation to a context c', some call must have published to the seed of c' an abstract entry state that covers the activation's entered store. Otherwise the unknowns of c' would have to cover a store that no equation put there.

Suppose a concrete call from a caller store s, which the caller's solved value covers, enters the callee in store s', and *adm* admits c' for it. Then enter, applied to the caller's solved value, must return some entry pair (q, e) that the equations route to c', with q covering s and e covering s' (Figure 8.2):

$$\textit{adm} \text{ admits } c' \text{ for } (s, s') \implies \text{some entry pair } (q, e) \text{ is routed to } c' \text{ with } s \in \gamma(q) \text{ and } s' \in \gamma(e).$$

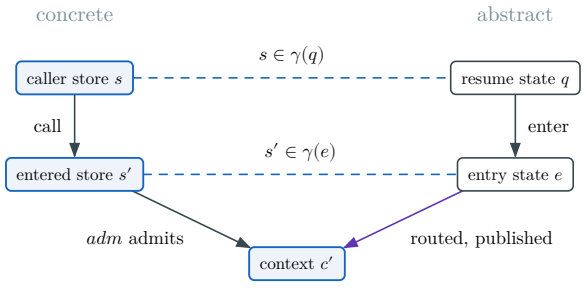


Figure 8.2: Adequacy. If the concrete semantics assigns a call to c' (left), the equations must route a covering entry pair to the same c' (right).

We call this condition *adequacy* (`routing_adequate`). It also requires the callee's entry unknown in c' to be in the solved set. CALL and RETURN need adequacy, because the call publishes e and its continuation reads the callee's result in the same c'. A second condition, *totality* (`routing_total`), requires *adm* to admit at least one context for every concrete call from a store that the caller's solved value covers. Without totality, a concrete call could be assigned no context at all and would therefore disappear from the context-indexed collecting semantics. The obligation TOTAL of Section 4.4 rules this out, and totality is exactly what it asks of the generated equations. Both conditions are premises of the soundness theorem and are proved once for each context policy.

policy	*adm* **admits** c' **iff**	**routed to**	**why they agree**
unit	$c' = ()$	$()$	both constant
call strings	$c' = \mathrm{take}_k(u \cdot c)$	$\mathrm{take}_k(u \cdot c)$	the same term
entry states	$c' = e\|_{\mathrm{formals}}$, (q, e) covers (s, s')	$e\|_{\mathrm{formals}}$	*adm* reads the routing

Table 8.1: The contexts *adm* admits for a call at site u from caller context c with caller store s and entered store s', where the equations route the entry state e, and why the two agree. $u \cdot c$ prepends the site, take_k keeps the first k entries, $e|_{\mathrm{formals}}$ lists the formals' values in e, and an entry pair (q, e) of the caller's solved value covers (s, s') when $s \in \gamma(q)$ and $s' \in \gamma(e)$. Unit and call strings are *functional* policies, since they choose the context without reading abstract states.

For unit and call strings the correspondence is direct, as the last column of Table 8.1 shows. *adm* admits exactly the context that the routing function computes for the call, which does not depend on e (`context_policy_of_fun`), so totality is immediate. Adequacy then reduces to the entry coverage of Section 6.1 and a routed entry unknown in the solved set.

Figure 8.3 shows how the policies differ on a call chain. A call string of length one separates the two calls of `wrap` but merges them again in `scale`, where both arrive from the same call site. Length two and entry-state routing keep them apart.

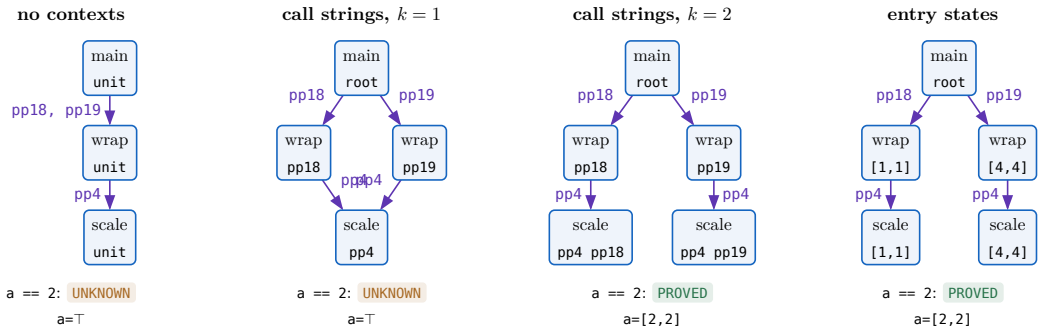


Figure 8.3: Procedure copies under four context policies, on a nested-call program: `scale(v)` returns `2 * v`, `wrap(w)` returns `scale(w)` from `pp4`, and `main` calls `a = wrap(1)` at `pp18` and `b = wrap(4)` at `pp19`, then checks `a == 2`. Boxes are copies labeled with their contexts (`root` is that of `main`), and arrows are calls labeled with their sites (Interval, claims `ctx-demo-*`).

An entry-state context consists of the abstract values that the entry state e gives the callee's formal parameters, written $e|_{\mathrm{formals}}$. For `bump(n)` and an e that binds n to $[4, 5]$, the context

is $[[4,5]]$, a list with one entry per formal. Unlike a call string, it depends on the abstract entry state, so the concrete call alone does not determine it.

If the caller's solved state says $x \in [4,5]$ at `bump(x)`, the equations publish $n \mapsto [4,5]$ to the context $[[4,5]]$. An execution with $x = 4$ must then be admitted in $[[4,5]]$, where its covering entry state was published, and not in $[[4,4]]$.

The entry-state policy (`routed_entry_context_rel`) does exactly this. For a call from s with entered store s', it admits c' only when $c' = e|_{\text{formals}}$ for an entry pair (q, e) with $s \in \gamma(q)$ and $s' \in \gamma(e)$. The equations publish that same e to c', so every admitted context is backed by a covering entry state, which is adequacy. Entry coverage (Section 6.1) gives every covered call such a pair, which is totality.

This policy depends on the solved analysis (Section 4.3) without circularity: the proof fixes σ, defines adm from it, and shows that σ covers the induced collecting semantics.

8.3. Soundness of the generated system thy

obligation	what the generated equations provide
INIT	the initial state at the program entry covers the initial stores
INTRA	the transfer along each local edge is sound
CALL	the routed entry state covers the entered store (adequacy)
RETURN	the callee's result is combined soundly in the same context (adequacy)
TOTAL	every covered call is admitted in some context (totality)

Table 8.2: How the generated equations meet the five obligations of Chapter 4, for an arbitrary program.

The argument holds for every domain and context policy. Its locale `routed_context` assumes a post-solution σ on a solved set closed under local edges and call continuations, the `analysis_contract` of Chapter 6, a sound bottom test, a sound callee resolution, and adequate and total routing. The theorem adds that the program entry is solved in the initial context and that the initial stores S_0 lie in $\gamma_{DG}(d_0, e_0)$: the initial state d_0 describes them under an environment e_0 of analysis-global values below the solved ones.

Theorem 8.1 (Routed collecting soundness, `activation_collect_dg_sound`)**.**

```
(* src/Abstract_Interpreter/Framework/Context/Routed_Context.thy *)                Isabelle
theorem activation_collect_dg_sound:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
  shows "A_{G,adm,c₀,g,S0} v c ⊆ cover v c"
```

Here `cover v c` is $\gamma_M(\sigma(v, c))$ inside the solved set and empty outside it. The proof turns the bound on each part, and on each seed, into its row of Table 8.2. `fun_route_activation_collect_sound` and `entry_state_activation_collect_sound` instantiate the theorem for the analyzer, and Chapter 9 computes such a post-solution.

9. Solving the Equations

Chapter 8 showed that every post-solution of the generated equations covers the context-indexed collecting semantics, provided the solved set contains the program entry and every unknown an execution from the entry reaches (`activation_collect_dg_sound`). This chapter shows that the analyzer obtains such a post-solution. Voblint solves the equations with the verified top-down solver of Tilscher et al. [4], which is proved partially correct. Chapter 8 described each right-hand side by the value it computes from a valuation, together with what it publishes to global unknowns. The solver needs each right-hand side as a strategy tree (Section 2.7): a small program that reads one unknown at a time and makes every read and every publication explicit, so that the solver can record dependencies while it evaluates. The equations first have to be brought into this form. Then the solver's correctness theorem has to be connected to the premises of the soundness theorem. Both steps assume that the solver returns. Whether it does is the last question of this chapter.

9.1. Adapting the equations to the solver

The solver accepts a right-hand side only as a strategy tree, uses one value type for all unknowns, and allows side effects only to global unknowns. A direct encoding of the equations, along the lines of Apinis et al. and Goblint, needs four things this interface does not offer (Table 9.1). Chapter 8 already stated the equations in the adapted form, and this section explains why each adaptation is needed. Only buffering rewrites a right-hand side after the fact, and it leaves the value, the publications, and the reads unchanged (`traverse_rhs_buffer_sides`, `sides_of_rhs_buffer_sides`, `dep_aux_buffer_sides`).

the direct encoding needs	**the solver offers**	**adapter**
read a result chosen from the caller's value	one read at a time	strategy tree (Section 9.1.1)
write the callee's entry, a local unknown	side effects to globals only	seeds (Section 9.1.2)
different local and global values	one value type	product (Section 9.1.3)
publish twice to one target	update rule at every side effect	buffering (Section 9.1.4)

Table 9.1: Where the direct encoding of the equations and the interface of the vendored solver differ, and the adapter that bridges each.

9.1.1. Dynamic reads thy

A local edge always reads the same unknown, its predecessor, whereas a call chooses what to read from values it has already read. In the running example the continuation $(pp3, c_0)$ first reads the caller $(pp2, c_0)$. Only from that value does it compute the entry state $n = [5, 5]$, which determines the context c_1, and only then does it know which result to read, $(\mathsf{FunctionResult}\, bump, c_1)$. A strategy tree allows this, because each continuation receives the value just read and may choose the next read from it. In the notation of Section 2.7, the contribution of a call at u in context c (`rhs_call`) is the value of the tree t_{call},

$$t_{\text{call}} = \mathsf{QueryL}\ \big((u, c), \lambda d.\ \mathsf{Side}\ \big(\mathsf{Activation_Seed}\, p\ c', e,\ \mathsf{QueryL}\ \big((\mathsf{FunctionResult}\, p, c'), \lambda r.\ \mathsf{Answer}\ \big(\text{combine}^\sharp(q, r)\big)\big)\big)\big),$$

where d is the caller's value, $(q, e) = \text{enter}^\sharp(d)$, $c' = \text{context}^\sharp(u, c, e)$, and r is the callee's result. The Side step publishes the entry state to the callee's seed (Section 9.1.2). `routed_call_program` implements this tree for a call and additionally handles bottom entry states and several callees or entry pairs.

9.1.2. Local entries

A call has to pass its entry state e to the callee's entry $(\mathsf{FunctionEntry}\, p, c')$, which is a local unknown. Apinis et al. publish to the entry directly [43, §6], and Goblint does the same with its local side effect `sidel` (`constraints.ml`). The vendored solver's Side accepts only global unknowns, and allowing local targets would mean changing the solver and redoing its correctness proof.

Voblint therefore routes the entry state through the seed $\mathsf{Activation_Seed}\, p\ c'$ of Section 8.1, a global unknown: the call publishes e to it with Side, and the equation of the entry reads it back with QueryG. In the running example `bump(5)` publishes $\{n \mapsto [5, 5]\}$ to $\mathsf{Activation_Seed}\, bump\ \ c_1$, and $(\mathsf{FunctionEntry}\, bump, c_1)$ reads it. Publishing alone does not make the solver evaluate `bump`, since it solves only unknowns that some tree reads. The call, however, reads the callee's result, and solving the result reaches the entry, which reads the seed.

Because seeds are global unknowns, the update rule for globals merges their contributions (Section 9.4). Without contexts, both calls publish to the one seed of `bump`, and warrowing widens it. The analyzer then reports `UNKNOWN` for `a == 6` with `interval: a=[-∞,6]`, whose lower bound $-\infty$ comes from this widening.

9.1.3. One value type thy

A local unknown holds an abstract state of the analysis's local domain, and an analysis global holds a value of its global domain (Section 6.2). The vendored solver has a single value type $'d$ for all unknowns, so Voblint stores both kinds in their product: every unknown holds a pair `dg_state` of a local and a global half, `dg_local` and `dg_global`, uses one half and leaves the other at $\bot$ (Table 9.2). Order and join on pairs work componentwise, so the solver's requirements on the value type follow from those on the two halves. Whether an unknown is global is a matter of the solver's interface and says nothing about the half

it uses: a seed is a global unknown that carries a local value, the entry state it passes to the callee.

solver unknown	role	value
(v, c), local	the state at v in context c	$(d, \bot)$
`Activation_Seed`, global	an entry state published by a call	$(e, \bot)$
`Analysis_Global` v, global	the analysis global v	$(\bot, g)$

Table 9.2: Which half of `dg_state` each kind of unknown uses. The global unknowns are named by `global_unknown`.

Transfers never see the pair or the seeds: the manager of Section 6.2 gives each transfer the local state and access to the analysis globals. Goblint combines the two kinds of value in a lifted sum instead, in which each unknown holds a value of exactly one of the two kinds. Voblint uses the product because its componentwise lattice structure keeps the Isabelle proofs simple, at the price of an unused $\bot$ half in every unknown.

9.1.4. Repeated publications thy

One right-hand side can publish to the same global unknown twice. Two calls that resume at the same node may be routed to the same callee context and then write the same seed. Declaratively two writes to one target g mean one bound:

$$\mathsf{Side}\ (g, a); \mathsf{Side}\ (g, b) \quad \text{means} \quad \sigma(g) \geq a \sqcup b.$$

The vendored solver applies the update rule at every Side, first to a and then to $a \sqcup b$. The per-origin update rules keep one record per origin (Section 2.7), so in every re-evaluation the recorded contribution first shrinks to a and then grows back to $a \sqcup b$. Under warrowing the shrinking step narrows, the growing step widens again, and the solver may not stabilize. The update rules of Stemmler et al. assume one joined contribution per target and evaluation [44, §3], and Goblint's per-origin narrowing rule for globals, when `narrow-globs` is enabled, joins the side effects of an evaluation before updating (`td3UpdateRule.ml`).

`buffer_sides` collects the publications of a right-hand side and joins those to the same target until a flush point. Where several calls return at a node, it flushes only when the right-hand side answers, so each seed receives one Side per evaluation. Elsewhere it flushes before every local read, so that an entry state reaches the callee before the callee's result is demanded, in the order of Goblint's normal-call transfer (`constraints.ml`). A flow-insensitive global (Section 6.4) assigned on two edges into one node therefore still receives one Side per edge. Where the buffer waits for the answer, the solver does extra work: a newly routed callee is solved from the seed's old value first, and once the joined seed is published, the callee and then the caller are evaluated again.

Buffering changes only when publications are issued. The buffered generator `routed_node_rhs_buffered`, which the analyzer runs, answers the same join of the four parts as the direct generator of Section 8.1. Under widening update rules the timing can change the computed result, though not the equations it solves. Section 16.3.1 discusses a solver extension that would make seeds and buffering unnecessary.

9.2. A demand-driven solve

The generator defines a right-hand side for every pair of a node and a context (`compiled_routed_eqs_for`). Under entry-state contexts over intervals there are infinitely many such pairs, and a solve never enumerates them all. It starts from one query, the result of `main` in the initial context c_0 (`root_query`, as in [43, §3]), and evaluates an unknown only when a right-hand side it is solving reads it [4], [60]. The solve thus runs backward from the result and discovers contexts as it goes: at a call it computes the callee's context from the caller's value and demands the callee's result there. Table 9.3 and Figure 9.1 follow this solve on the running example, generated from the analyzer's solver trace (`--trace`, Interval, warrowing; claim `pg-contexts-trace`). The entry states $e_1 = \{n \mapsto [5,5]\}$ and $e_2 = \{n \mapsto [4,4]\}$ route to the contexts $c_1 = [[5,5]]$ and $c_2 = [[4,4]]$.

phase	evaluating	what happens
1	`exit_main` to `pp2`	the root query at `exit_main` queries back through `pp6`, `pp5`, `pp4`, `pp3` and `pp2` to `entry_main`
2	`entry_main`	QueryG reads Activation_Seed *main* c_0 $= \bot$, joins d_0
3	`pp3`	computes e_1 and routes to the new context c_1, publishes it to Activation_Seed *bump* c_1, queries (`exit_bump`, c_1), which queries back to the entry, which reads Activation_Seed *bump* $c_1 = e_1$: the result is `interval: a=⊤, b=⊤, #ret=[6,6], n=[5,5]`
4	`pp4`	computes e_2 and routes to the new context c_2, publishes it to Activation_Seed *bump* c_2, queries (`exit_bump`, c_2), which queries back to the entry, which reads Activation_Seed *bump* $c_2 = e_2$: the result is `interval: a=⊤, b=⊤, #ret=[5,5], n=[4,4]`. The answers then propagate up through `pp5` and `pp6` to `exit_main`

Table 9.3: The solve of the running example, condensed into phases. The second column names the unknown, or chain of unknowns, whose right-hand side the phase evaluates.

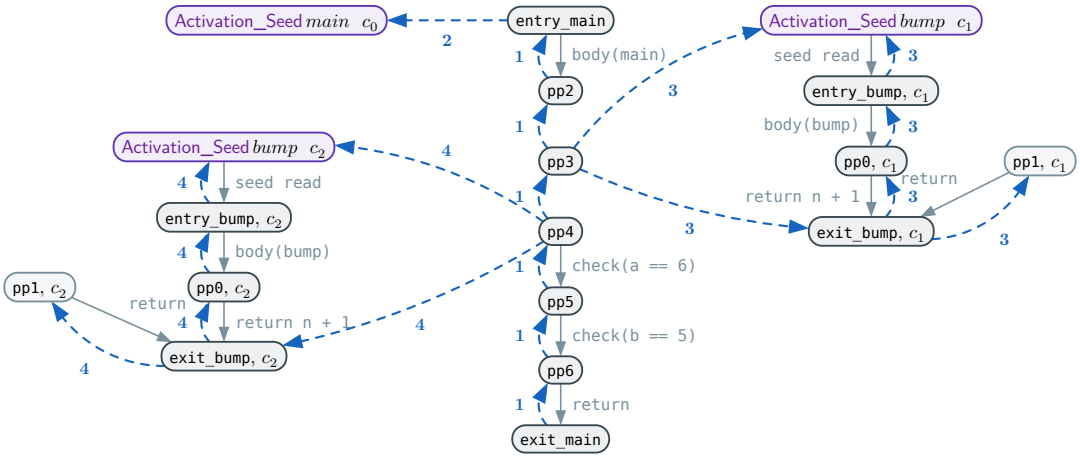


Figure 9.1: The solve of Table 9.3 on the unknowns. Gray arrows are graph edges, including a seed feeding its entry; blue dashed arrows are the solver's queries and publications, labeled with their phases. The solve runs backward from the result of `main` and demands the copies of `bump` for c_1 (right) and c_2 (left) as it discovers those contexts. Gray `pp1` is the dead end of the body of `bump`, which always returns (Section 9.3.2); its value stays $\bot$.

The checks at `pp4` and `pp5` read the solved values `interval: a=[6,6]` and `interval: b=[5,5]`, and the analyzer proves both. Without contexts, $c_1 = c_2$, both calls publish to one seed, and `bump` is solved once for both (Section 9.1.2).

A terminating solve thus reaches finitely many unknowns, in the contexts it discovers.

9.3. What a terminating solve guarantees

A solve that returns gives a valuation σ and the set V of local unknowns it stabilized. The soundness theorem of Section 8.3 needs the bounds of a post-solution on a set that contains the program entry and every unknown an execution from there can move to. The solver instead bounds σ on V, and V is closed under the reads of its right-hand sides, starting from the query: everything the result of `main` depends on is solved. Section 9.3.1 states this certificate, and Section 9.3.2 derives from it the set the theorem needs.

9.3.1. The certificate thy

The verified solver states its guarantee as the certificate `part_post_solution` of Section 2.6, over the equation system T, the valuation σ and the solved set `vars`:

```
(* vendor/td-verification/Basics_side.thy *)                    Isabelle
abbreviation part_post_solution ::
    "('x,'g,'d::bounded_semilattice_sup_bot) eqsT ⇒ 'x ⇒ ('x + 'g ⇒ 'd) ⇒ 'x set
    ⇒ bool" where
  "part_post_solution T x sigma vars ≡
    x ∈ vars ∧ (∀u ∈ vars.
    dep_L T sigma u ⊆ vars ∧
    eq T u sigma ≤ sigma (Inl u) ∧
    sides_of_rhs (T u) sigma ≤ sigma)"
```

T (`eqsT`) maps each unknown u to its strategy tree $T\,u$. Evaluating a tree under the final valuation σ determines three things: the answer it returns, the local unknowns it reads, and the contributions it publishes. In Isabelle these are `eq` $T\,u\,\sigma$ (an abbreviation of `traverse_rhs`), $\mathtt{dep_L}\,T\,\sigma\,u$, and `sides_of_rhs`, which joins the published values per target and is $\bot$ where the tree publishes nothing. They are taken under the final valuation because the unknowns a tree reads depend on the values it has already read (Section 9.1.1). With them the certificate states four facts about the query x and the solved set V:

$$x \in V \tag{C1}$$

$$\forall u \in V.\ \mathtt{dep_L}\,T\,\sigma\,u \subseteq V \tag{C2}$$

$$\forall u \in V.\ \mathtt{eq}\,T\,u\,\sigma \leq \sigma(u) \tag{C3}$$

$$\forall u \in V.\ \mathtt{sides_of_rhs}\,(T\,u)\,\sigma \leq \sigma \tag{C4}$$

(C1) puts the query into the solved set, and (C2) closes the solved set under reading. (C3) is the post-solution bound for local unknowns, and (C4) bounds every published contribution at once, since it compares two valuations pointwise (`part_post_solution_query`, `part_post_solution_closed`, `part_post_solution_local_bound`, `part_post_solution_side_bound`). The returned σ may lie above the least solution, because the solver widens and narrows at its widening points (Section 2.7), and (C1) to (C4) are all the solver guarantees about σ.

(C3) and (C4) together are the pair condition of Section 2.6. As a pair, the tree of u evaluates to `rhs_pair`: its answer and `rhs_sides`, which restricts `sides_of_rhs` to the targets the evaluation publishes to. `part_post_solution_iff_rhs_pair` proves the certificate equivalent to (C1), (C2), and that pair bound at every $u \in V$. Both components come from the evaluators above, so the solver and its strategy trees are unchanged.

In the running example the query is $(\mathsf{FunctionResult}\, main, c_0)$, and V holds the 15 local unknowns the solve of Section 9.2 reached. The tree of $(pp3, c_0)$ reads $(pp2, c_0)$ and, in the context c_1 it computes from that value, $(\mathsf{FunctionResult}\, bump, c_1)$; by (C2) both lie in V. (C3) at $(pp3, c_0)$ requires $\mathrm{combine}^\sharp(q_1, \sigma(\mathsf{FunctionResult}\, bump, c_1)) \leq \sigma(pp3, c_0)$, where q_1 is the resume state of the first call's entry pair, and (C4) there requires $e_1 \leq \sigma(\mathsf{Activation_Seed}\, bump\ \ c_1)$.

9.3.2. Solved backward, used forward

The solver discovers unknowns backward from the result of `main`. The soundness proof follows executions forward from the entry, and each step uses the bound (C3) at the node the execution moves to, so it needs every node an execution visits to be solved. The argument is the same in every context, so below we speak of nodes rather than unknowns.

The two directions meet through (C2). A node's equation reads its predecessors, and the node after a call reads the call site, so every node that reaches a solved result along such steps is solved as well. It therefore suffices that every node of a procedure reaches the procedure's result. Code after a `return` is the case to check, since no execution reaches it, but the compiler still compiles it. In Figure 9.2, line 3 reaches the result through `return 1`. Line 6 continues to the end of the body, and the compiler ends every body with a return edge without a value (Section 3.5.1), so line 6 reaches the result as well. Once the result of `f` is solved, (C2) therefore makes every node of `f` solved, dead code included.

```
1 fun f(b) {                                  VIMP ↗
2   return b;
3   if (b < 0) {
4     return 1;
5   }
6   b = 2;
7 }
```

Figure 9.2: Code after a `return` in `f`. No execution reaches lines 3 to 6, yet every line reaches the result of `f`: line 3 through `return 1`, line 6 through the final return edge.

The proof uses the solved nodes of procedures whose result is solved in the same context. An execution starts at the entry of `main`, which is solved because the result of `main` is the query (C1) and every node reaches its procedure's result. Intra edges and the step from a call site to its continuation stay inside a procedure. A call reads the callee's result in the context it computes, so by (C2) it is solved too, and executions never leave this set.

The return edge that ends every body is the implicit return of a body that reaches its end without a `return`. In a body that always returns, it starts in dead code, where the analyzer computes $\bot$, and adds nothing. Omitting it saves one dead node per body, but code after a `return` could end in a node without successors, and the proof would need a syntactic description of the code no `return` cuts off, so the compiler always emits the edge.

In Isabelle, the nodes of a procedure are `prog_node`. `prog_node_reaches` proves that each reaches the procedure's result, and `prog_node_entry`, `prog_node_intra`, and `prog_node_calls` that executions stay among them. The restricted set is `live_unknowns`, and `live_unknowns_cover` proves from a well-formed program and a terminating solve alone that it contains the entry and is closed under execution steps. The final theorem therefore needs no premise about which unknowns the solve visited.

9.3.3. The solver as a parameter thy

The soundness argument uses only four facts about a solve that returns, and none of them depends on how the solver works. They are: the result is a post-solution on the solved set (the certificate), the solved set is finite, the run lies in the domain on which the first two facts hold (Section 9.6), and the executable run returns that same result. Finiteness lets Chapter 10 combine the finitely many contexts solved at a node into one verdict. Since the proof uses nothing else, the solver is a parameter of the analysis, and another solver, for instance one with local side effects (Section 16.3.1), attaches to Voblint by proving the same four facts. In Isabelle, the analysis locale `dg_analysis` takes the solver as a parameter and extends `certified_solver`, which states the four facts. `td_certified_solver` proves them for the vendored solver under every update rule.

9.4. Merging contributions to global unknowns thy

A global unknown such as a seed receives contributions from several places. The solver merges each new contribution into the value it holds, and this merge, the *update rule*, decides both precision and termination. Joining keeps every contribution exactly but may grow forever. Widening stops the growth but loses precision. Stemmler et al. compare existing update rules and propose new ones [44, §3–4], and Tilscher et al. formalize a generic interface for them and prove five rules sound against it [4]. Voblint exposes all five.

Every rule keeps one record per origin, the right-hand side that sent the contribution (Section 2.7). In the analyzer, each right-hand side's contributions to one target arrive already joined (Section 9.1.4). The rules differ in what they record and whether they widen:

- *join* joins each contribution into the value;
- *join per origin* records each origin's latest contribution and reports the join of the records;
- *warrow* records the same, and warrows the value toward the join of the records;
- *warrow per origin* warrows the origin's old record with the new contribution, then joins the records;
- *bounded narrowing* does the same, but also counts how often an origin has switched from widening to narrowing. Once the count reaches a bound, every later switch to narrowing takes only its first narrowing step.

Table 9.4 runs all five on one sequence of contributions, and the project site runs them on a program with several shared callee entries. In `run_voblint` the rule decides how a callee's entry state accumulates across call sites and, with flow-insensitive program globals (Section 6.4), how a global accumulates its writes.

No rule is more precise on every program, and the rules also differ in termination. Section 13.3 compares them on the programs of Table 13.1.

rule, vendored update	**1:** A sends `[0,3]`	**2:** B sends `[4,4]`	**3:** A sends `[1,1]`	**4:** A sends `[0,8]`
join `update_global_always_join`	`[0,3]`	`[0,4]`	`[0,4]`	`[0,8]`
join per origin `update_global_per_origin`	`[0,3]`	`[0,4]`	`[1,4]`	`[0,8]`
warrow `update_global_warrowing_apinis`	`[0,3]`	**`[0,+∞]`∇**	`[0,4]`	**`[0,+∞]`∇**
warrow per origin `update_global_warrowing_per_origin`	`[0,3]`	`[0,4]`	`[0,4]`	**`[0,+∞]`∇**
bounded narrowing, bound 5 `update_global_bounded_narrowing`	`[0,3]`	`[0,4]`	`[0,4]`	**`[0,+∞]`∇**

Table 9.4: One global unknown under each update rule, from $\bot$, after four interval contributions from origins A and B. Join per origin reports the join of each origin's latest contribution, so at step 3 A's `[1,1]` replaces `[0,3]`. Warrow widens at step 2, because `[0,4]` is not below `[0,3]`, and narrows at step 3. Warrow per origin widens only when A's own contribution grows. Bounded narrowing agrees with warrow per origin here, since A switches to narrowing once, and the bound 5 is never reached. A widened bound is set bold and marked ∇. The cells are read from `update_rules_example`, which evaluates the vendored rules with Voblint's interval operators; the contributions separate the rules and come from no program.

9.5. Making abstract states executable thy

The pointwise analyses of Chapter 5 are specified over abstract states $\mathrm{Var} \to A$, total functions on variable names (the relational order analysis has its own state type, `relc`). This suits proofs, where lookup is function application and the lattice operations are pointwise. The solver, however, cannot compute with it: after every evaluation it decides whether an unknown changed, and deciding $f \leq g$ for two such functions means checking $f(x) \leq g(x)$ for infinitely many names x. Nipkow and Klein address the same problem in the abstract interpreter of Concrete Semantics by refining total function states to a finite representation with a default [6, §13.6]: a state is a finite list of variables with their values, every unlisted name reads as $\top$, and two states are compared on the listed names. Such a representation works like a default dictionary, a finite dictionary that answers every missing key with a default. Voblint follows the same idea but needs two defaults. It represents a total function exactly and therefore loses no precision. We call such a representation an *executable state carrier* (*carrier* for short, distinct from the carrier of a domain, Chapter 2).

VIMP initializes globals to zero, as C does for objects with static storage (§6.7.9p10), and leaves the locals of `main` arbitrary (Section 3.3). The initial state must give every local the value $\top$ and every global the value $0^\sharp$, the abstract value of the constant 0, without listing the declared globals. Voblint's carrier therefore keeps two default dictionaries, one for the names the program treats as local and one for the globals, each a default paired with a finite list of overrides. We write the state with local dictionary (d_l, ls) and global dictionary (d_g, gs) as $\langle\!\langle (d_l, \mathrm{ls}), (d_g, \mathrm{gs}) \rangle\!\rangle$. The initial state is $\langle\!\langle (\top, []), (0^\sharp, []) \rangle\!\rangle$, and the least element is $\langle\!\langle (\bot, []), (\bot, []) \rangle\!\rangle$.

Different representations can describe the same state: overrides of distinct names may appear in any order, and an override equal to its default changes nothing. A *quotient type* (Section 2.8) identifies the representations that answer every lookup alike:

```isabelle
(* src/Abstract_Interpreter/Exec/State/Default_St_Base.thy *)
quotient_type 'a default_st =
  "('a::bot) default_st_rep" / "eq_default_st_rep"
  morphisms rep_default_st Abs_default_st
  by (rule equivp_eq_default_st_rep)
```

Two carrier states are therefore equal exactly when every lookup agrees (`default_st_eq_iff`). Comparing the two defaults and the finitely many listed names decides the order (`le_default_st_rep_code_iff`), and equality tests it in both directions.

The program's global-variable classifier $\mathcal{G}$ (Section 3.2) assigns each name x its *location* $\ell(x)$, local or global. A lookup reads the dictionary of the variable's location: the override if there is one, the default otherwise. We write $d\langle l\rangle$ for the lookup at location l and $d\langle l := a\rangle$ for the update. Looking up every variable at its location gives the total function $\rho_{\mathcal{G}}(d)$ that a carrier state represents (`default_st_to_fun`). A carrier state means what its function means under the concretization `gamma_state` of Section 5.2.1, so its own concretization (`default_st_gamma`) is

$$\gamma(d) = \gamma(\rho_{\mathcal{G}}(d)) = \{s \mid \forall x.\, s(x) \in \gamma(d\langle \ell(x)\rangle)\}.$$

The solver never computes $\gamma(d)$. It uses only the executable lattice operations. Function states get order, join, and bottom pointwise from HOL, but no widening or narrowing. The carrier instantiates all of these classes on `default_st` whenever the values do, so the generic solver runs on it unchanged.

Distinct carrier states can still denote the same stores: every state whose local default is $\bot$ denotes $\emptyset$, yet the quotient keeps such states apart because they represent distinct functions.

Each lattice operation is computed with the value domain's operation on the two defaults and on each listed name, so under lookup it agrees with the pointwise operation on functions. For the join, `default_st_get_sup` states $(d \sqcup e)\langle l\rangle = d\langle l\rangle \sqcup e\langle l\rangle$, and analogous lemmas cover bottom, order, widening, and narrowing. The transfer functions must commute with the represented function in the same way,

$$\rho_{\mathcal{G}}(\mathrm{op}_{\mathrm{exec}}(d)) = \mathrm{op}_{\mathrm{abs}}(\rho_{\mathcal{G}}(d)),$$

and since $\gamma(d) = \gamma(\rho_{\mathcal{G}}(d))$, the soundness facts of Chapter 6 then transport from the abstract to the executable operation. The locale `dg_analysis_exec` states this commutation for the transfer on nonempty states and for procedure entry. For primitives proved sound it holds without a per-domain proof, because the abstract and executable steps are derived from the same operations (`sound_nonrelational_ops.tf_st_for_commute`, Section 11.1).

Emptiness, on which `DEAD` rests (Chapter 5), is the one operation the listed overrides do not decide: an unlisted variable can make the state empty through its default. The carrier's test inspects the local default, the overrides that the represented function reads, and each declared global. The globals are listed explicitly because a program has finitely many of them, so the global default may describe no variable at all. `default_st_is_bot_for_gamma_iff` proves the test exact,

$$\texttt{default_st_is_bot_for}\ \ \text{globals}\ \ d \iff \gamma(d) = \emptyset,$$

provided the list enumerates exactly the globals of $\mathcal{G}$. Because the test is exact, collapsing the states it finds empty to $\bot$ commutes with the represented function, as the specification's collapse requires, and a state it keeps is nonempty, which is where the numeric transfer commutes (`live_default_st`).

9.6. Why termination is not proved

The previous sections assume that the solve returns, and no theorem shows that every solve does. HOL admits only terminating recursion, so Isabelle defines the solver with a domain predicate (`solve_dom`, Section 2.8), the arguments on which its recursion terminates, and proves the certificate of Section 9.3 on that domain. The analyzer runs the executable form `solve_c`, which returns only when the recursion finishes, and `solve_dom_of_solve_c` turns a returned run into domain membership, so the theorems about `run_voblint` have no termination premise.

Under the unit context the unknowns of a compiled program form a finite set (`compiled_unit_vars_finite`), and bounded call strings over a compiled program form a finite space (`compiled_call_strings_finite`). Under entry-state contexts we expect termination to fail for some programs. With Interval, a recursion that changes its argument at every level demands a fresh context at every level, and widening bounds the values of existing unknowns but not their number (`01-unbounded_context_chain_diverges`). Goblint's configuration for the SV-COMP competition avoids this explosion by omitting intervals from the contexts of recursive functions [31, §3.4.3]. Finitely many unknowns do not suffice either: without contexts, the interval recursion `f(x) { f(x + 1) }` entered with $x = 0$ makes the one seed of `f` grow through $[0, 0], [0, 1], [0, 2], \ldots$, and the joining update rules never widen this chain (`01-recursion_grows_join_diverges`, Table 13.1). Both programs exceed their time limits, and we expect, without proof, that they diverge (Section 12.5). Both are recursive. For partial tabulation over abstract calling contexts, Schulze Frielinghaus et al. give a local solver that terminates on every non-recursive program, even over lattices with infinite ascending or descending chains [75].

The vendored termination theorems cover top-down solvers without side effects over a finite type of unknowns. The solver with separate widening and narrowing phases terminates under this condition, given widening and narrowing operators whose iterations are ultimately stable [76, Thm. 2], and the warrowing solver additionally needs a precise widening and monotonic right-hand sides and dependencies [76, Cor. 1]. Voblint's unknowns pair nodes of an infinite type with contexts, so these theorems do not apply to any of its configurations. Seidl and Vogler prove on paper that their side-effecting solver terminates on every system in which side effects target only unknowns without a right-hand side, as long as only finitely many unknowns are encountered [60, §9, Thm. 5], for widening and narrowing operators whose iteration sequences are ultimately stable [60, §3]. The joining update rules never widen, so this result does not carry over to them. Mechanizing such a result for the vendored solver is future work, so the end-to-end theorem (Section 10.4) is a partial-correctness result about every report the analyzer returns.

10. What a Report Guarantees

This chapter answers what the analyzer's output guarantees about the executions of the source program. Solving the equation system of Chapter 8 with the solver of Chapter 9 yields an abstract state for every pair of a CFG node and a context the solver reached, and a value for every global unknown. By Section 9.3.2, these states cover every execution. A user, however, reads neither states nor contexts. The user reads a report with one verdict for every `__voblint_check` and a diagnostic wherever a division may fail. The chapter shows how the analyzer derives this report from the solution and why every verdict in it is sound. Neither verdict claims its check is reached. `PROVED` says the condition holds whenever a run reaches the check, and `REFUTED` says that it fails there (Section 10.5).

10.1. From solution to report thy

The constructions of the previous chapters are packaged behind one HOL function, `run_voblint`. It takes an analysis configuration and a VIMP program. The configuration (`analysis_config`) packages the choices of the earlier chapters: the analyses that run together (Chapter 7), the update rule (Section 9.4), the context policy (Section 8.2), and the placement of program globals (Section 6.4). The analyzer compiles the program to its graph `prog_cfg` p (Chapter 3), builds the equation system of the configured analyses and context policy (Chapter 8), solves it from the exit of `main` (Chapter 9), and builds a report from the solution.

```
(* src/Executable_Surface/CLI/Analysis_Run.thy *)                    Isabelle
definition run_voblint ::
  "analysis_config ⇒ imp_prog ⇒ analysis_report analysis_answer"
where
  "run_voblint config p =
     (if ¬ valid_config config then Invalid_Activation
      else if ¬ wf_program_compile_input_exec p then Malformed_Program
      else case analysis_report_of config p of
             None ⇒ No_Answer
           | Some res ⇒ Analysed res)"
```

Before it compiles anything, `run_voblint` checks its two inputs, so that a report exists only where the soundness proofs apply. `valid_config` requires a nonempty list of distinct analyses, and `wf_program_compile_input_exec` decides the structural conditions that the compilation theorems of Chapter 3 assume. An input that fails is answered with `Invalid_Activation` or `Malformed_Program`, and `No_Answer` stands for a solve that produces no result. Every answer `Analysed` res thus comes from inputs that pass both tests (`run_voblint_report_contract`), and the executable test implies the well-formedness premise of the proofs (`wf_program_compile_input_exec_sound`, `run_voblint_wf`). The source-level theorem of Section 10.4 therefore needs no premise about the configuration or the program.

The report that Analysed carries (`analysis_report`) keeps the solver's abstract states, each under the CFG node and the context it was solved for, and builds on them what the user reads: a verdict for every check (Section 10.2) and the arithmetic diagnostics (Section 10.5). It also keeps the values of the global unknowns.

The solver may analyze a procedure several times, once per context (Section 8.2), so a CFG node can carry several states. The playground draws both pictures for the program of Figure 10.1. Its Analysis view shows one node per pair of a program point and a context, and its Control-flow view shows each node of the compiled CFG once, with the states of all its contexts grouped there. The report takes the second view. At a node v it describes every store that one of its states there describes, aggregated over all contexts the solver solved v in (`report_states_at`, `report_sem`):

$$\mathcal{R}_{\mathsf{res}}\ \mathsf{v} = \bigcup_{\mathsf{d} \in \mathsf{report_states_at\ res\ v}} \gamma_{\mathsf{res}}\ \mathsf{d},$$

where γ_{res} is the concretization of the configured analyses.

The union drops the contexts on purpose. A context is not part of the program's state. It is a label the analysis attaches to procedure activations, possibly several to one activation, to keep different calls apart, and the context policy decides which labels exist (Section 8.2). A source execution consists of the remaining program, a store, and a stack of frames, so there is no context it could be compared with. A user asks what holds at a program point, whichever call led there, and a guarantee per CFG node answers exactly this question. Inside the proof the contexts still matter: each per-context state over-approximates the activation collecting semantics of its context (Section 9.3.2), and the chain of Section 10.3 passes through this fact. The report keeps the per-context states for inspection, and the Analysis view displays them.

10.2. Verdicts

The command-line tool prints the report through its checks: each check at v receives one of four verdicts, `PROVED`, `REFUTED`, `UNKNOWN`, or `DEAD`. The browser playground shows the same verdicts inline in the editor and beside the graph, together with the state of every node and context, and a replay of the solve (Section 12.3, Section 12.4). Each verdict is printed at the source position of its check, and the text report checks that these positions are unique (`run_voblint_labelled_check_sound`).

How a verdict is computed. Each state d in `report_states_at res v` gives its own verdict for a check:

```
(* src/Abstract_Interpreter/Framework/Checks/Contextual_Check_Report.thy *)   Isabelle
fun classify_point ::
  "(exp ⇒ 'a ⇒ check_result) ⇒ exp ⇒ 'a lifted ⇒ contextual_verdict"
where
  "classify_point classify c Bot = Dead"
| "classify_point classify c (Lifted st) = Decided (classify c st)"
```

The state Bot gives `DEAD`. Any other state is classified by `mcp_classify`, which asks every active analysis for the truth value of the check's condition through the query system of Section 7.1 and combines the answers by meet: an exact 1 gives `PROVED`, an exact 0 gives `REFUTED`, and any other answer gives `UNKNOWN`. A definite answer holds in every store the state describes (`mcp_classify_proved`, `mcp_classify_refuted`).

The verdict of the CFG node soundly joins the per-context verdicts of its states (`aggregate_verdicts`). If every state other than Bot gives `PROVED`, the node is `PROVED` if every such state gives `REFUTED`, it is `REFUTED`. If the states disagree, or one of them gives `UNKNOWN`, the node is `UNKNOWN`. Disagreement cannot be resolved otherwise. If one context proves `x > 0` and another refutes it, the stores reaching v in the first satisfy the condition and those in the second violate it, so no single definite verdict holds for every store at the node. States classified `DEAD`, the Bot states, are neutral in this aggregation, so a node at which the report holds no state, or only Bot states, is `DEAD`.

What a verdict claims about stores. The chain of Section 10.3 has to end in something an end user reads, and an end user reads the verdicts rather than the states. To state that the verdicts are correct as an inclusion of sets, we turn the verdicts at v into the set of stores consistent with them, the *verdict set* $\mathcal{V}_{\text{res}}$ `v`. If v carries the check `x > 0` with verdict `PROVED`, the verdict set is $\{s \mid s(x) > 0\}$: the verdict tells the user that x is positive there and nothing else. `REFUTED` gives the stores in which the condition fails, and `UNKNOWN` gives every store:

```
(* src/Executable_Surface/CLI/Analysis_Report.thy *)                    Isabelle
fun verdict_holds :: "check_result ⇒ exp ⇒ store ⇒ bool" where
  "verdict_holds Check_Proved e s = truthy (⟦e⟧ₑ s)"
| "verdict_holds Check_Refuted e s = (¬ truthy (⟦e⟧ₑ s))"
| "verdict_holds Check_Unknown e s = True"
```

A `DEAD` check says nothing about the values in a store. It claims that no store arrives at v, which is a statement about reachability, and Section 10.5 treats it. As a set of stores it therefore admits every store and leaves the claims of the other checks at v untouched:

```
(* src/Executable_Surface/CLI/Analysis_Report.thy *)                    Isabelle
definition check_stores :: "result_check ⇒ store set" where
  "check_stores c = (case check_verdict c of
                       Decided r ⇒ {s. verdict_holds r (check_exp c) s}
                     | Dead ⇒ UNIV)"
```

A store is consistent with what the user reads at v only if no verdict there contradicts it, so the verdict set keeps exactly the stores that satisfy every check at v:

```
(* src/Executable_Surface/CLI/Analysis_Report.thy *)                    Isabelle
definition verdict_stores :: "analysis_report ⇒ pp ⇒ store set" ("𝒱_") where
  "𝒱_res v = (⋂c ∈ report_checks_at res v. check_stores c)"
```

The intersection states this for any number of checks at a node. In a compiled program each check has a node of its own, and the verdict set is then the set of that single check.

The verdict set can be much larger than $\mathcal{R}_{\text{res}}$ `v`, because a verdict keeps only the truth value of one condition (last row of Figure 10.1). Soundness needs only that every store reaching v lies in it.

Every verdict is backed by the states it was computed from: each store that a state at v describes satisfies every definite verdict at v, that is, $\mathcal{R}_{\text{res}}$ `v` $\subseteq$ $\mathcal{V}_{\text{res}}$ `v` (`analysis_report_verdicts_sound`). The theorem assumes that each node verdict is the join of its states' verdicts, which holds for every report `run_voblint` returns (`run_voblint_consistent`).

10.3. The chain at one check

Let res be a report `run_voblint` returns for a program p, g = `prog_cfg` p its compiled graph, $\mathcal{G}$ = `declared_global` p its global-variable classifier, and S = `cinit_stores` $\mathcal{G}$ its initial-store set (Section 3.3, Chapter 4). A store is *reached* if some finite run of `main`, started from a store in S, arrives at it. The argument that a verdict is sound places a reached store at a CFG node v and follows a chain of inclusions between sets of stores there. It ends in the two sets of the report, $\mathcal{R}_{\text{res}}$ `v` and $\mathcal{V}_{\text{res}}$ `v`. Each step is a theorem:

a store s a source run reaches, at a node v related to the run

$$\in \quad \mathcal{C}_{\mathcal{G},g,S}\ v$$
source_reaches_node_collect

$$= \quad \bigcup_c \mathcal{A}_{\mathcal{G},\text{adm},c_0,g,S}\ v\ c$$
node_collect_eq_Union_activation_collect

$$\subseteq \quad \mathcal{R}_{\text{res}}\ v$$
run_voblint_covers

$$\subseteq \quad \mathcal{V}_{\text{res}}\ v$$
analysis_report_verdicts_sound

The first step places a store that a finite source run reaches in the node collecting semantics, at a node v the simulation relates to the run. On the elaboration this is every node at which the run is observed (`csim_point_compile_prog`, Section 3.6). The second step holds for a policy with a cover that meets all five obligations of Chapter 4: `node_collect` is the union of `activation_collect` over the contexts `adm` admits, with `main` in c_0 (Section 4.3). Contexts enter the argument only at this step. The source-level theorem itself goes from $\mathcal{C}$ directly to the report. The third inclusion is the analyzer's soundness result. It bounds the stores collected at v, over all contexts, by the union of the report's states at v, without saying which state covers a given store. The fourth is the inclusion of Section 10.2 and depends only on the report and its consistency.

The program of Figure 10.1 calls `f` twice. The first call passes 1. The second passes an input x with $1 \le x \le 3$, if the input is in that range. So `m` is 2, 4, or 6 at the checks: `m >= 2` always holds, and `m == 2` holds at the first call and fails at the second call when x is 2 or 3.

```
1  fun f(n) {                                   VIMP ↗
2    m = 2 * n;
3    __voblint_check(m >= 2); v
4    __voblint_check(m == 2);
5    return m;
6  }
7
8  fun main() {
9    a = f(1);
10   x = __voblint_nondet_int();
11   if (0 < x && x < 4) {
12     b = f(x);
13   }
14 }
```

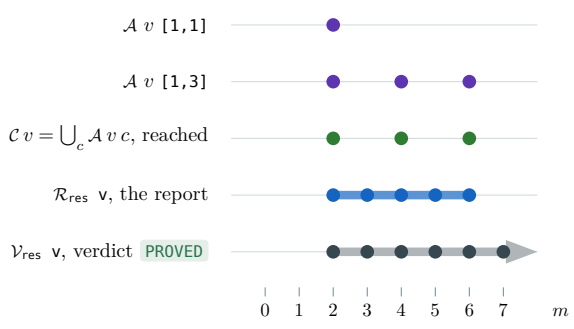


Figure 10.1: Left: the program. Right: the chain at the node v of the check `m >= 2` (line 3), projected on m, with Interval and entry-state contexts. Each row is a set of stores. Both per-context rows lie inside the union row, and each later row lies inside the rows below it. The report admits 3 and 5, which no run reaches, and the verdict set of `PROVED` is unbounded. The check `m == 2` (line 4) sees the same stores. Report and verdict are analyzer output (claim `chain-split-entry`); the first three rows are derived by hand.

`f` is analyzed in two contexts, one per entry state. The check `m >= 2` holds for every admitted store in both, so its verdict is `PROVED`. The check `m == 2` sees the same sets, but the contexts disagree. The first call enters with $n = 1$, and the check is `PROVED` there. The second enters with $n \in [1, 3]$. Its state `[2,6]` also admits 3 and 5, and the check is `UNKNOWN`, the only sound answer, since the run with $x = 1$ satisfies it and the runs with $x = 2$ and $x = 3$ violate it.

The report does not say which context covers a reached store, so the node verdict joins the context verdicts: `PROVED` and `UNKNOWN` give `UNKNOWN`. Without contexts, both calls feed the one seed of `f`, which the warrowing update rule widens, and the check is `UNKNOWN` at `interval: m=⊤` (claim chain-split-none). All of these verdicts are sound. They differ only in how much the analysis keeps apart, which the context policy decides.

10.4. The source-level theorem thy

One theorem states the chain at one store for the exported analyzer. It quantifies over an arbitrary analysis configuration and program, the arguments of `run_voblint` (Section 10.1). Whenever the analyzer returns a report, every store a finite source execution reaches is covered by that report and satisfies its definite verdicts. The theorem is about `run_voblint` itself, the constant that code generation exports, so it concerns the analyzer the tools run, up to the trusted code generator and target toolchains (Section 12.1, Section 12.5).

Theorem 10.1 (Source-level soundness of the analyzer, `run_voblint_source_sound`)**.**

```
(* src/Executable_Surface/CLI/Analysis_Certified.thy *)                    Isabelle
theorem run_voblint_source_sound:
  fixes p :: imp_prog and s0 s :: store
  defines G_def: "𝒢 ≡ declared_global p"
      and Pi_def: "Π ≡ prog_table p"
      and g_def: "g ≡ prog_cfg p"
  assumes s0: "s0 ∈ cinit_stores 𝒢"
      and run: "𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)"
      and ans: "run_voblint config p = Analysed res"
  shows "(∃v. p, s0 ⊢ (residual, s, frs) ▷ v)
       ∧ (∀v. p, s0 ⊢ (residual, s, frs) ▷ v ⟶
                 s ∈ 𝒞_{𝒢,g,cinit_stores 𝒢} v
               ∧ s ∈ ℛ_res v
               ∧ s ∈ 𝒱_res v)"
```

Here $\mathcal{G}$ and g are as in Section 10.3, and Π is the procedure table `prog_table` p. The theorem assumes three premises:

(P1) `s0 ∈ cinit_stores 𝒢`: the run starts from a store in `cinit_stores`, the initial stores S of Chapter 4, which zero every global, as C does for objects with static storage duration, and leave the locals unconstrained.

(P2) `𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)`: a finite `pstep` execution of p runs from the body of `main` to a configuration with store s. Any stopping point is allowed, so nonterminating programs are covered through their prefixes.

(P3) `run_voblint` config p = Analysed res: the analyzer returned a report. Input validity and solver termination are therefore not separate premises (Section 10.1), and the theorem is a partial-correctness result: termination is not proved for every program (Section 9.6).

The conclusions are stated at graph nodes, but a source configuration names none. Its residual command can occur at several places of the program, so the node has to come from the run. The relation $p, s_0 \vdash (\text{residual}, s, \text{frs}) \rhd v$ of Section 3.6 (`at_node`) supplies it. It holds when some run of the elaboration from s_0 reaches an annotated command that erases to the residual and names v, and the simulation `csim` relates that configuration to v with the same store. We then say that the configuration is *observed* at v. The relation is not functional, since a configuration reachable along two paths can be observed at two nodes.

The theorem has two halves. The first states that the configuration is observed at some node. The second states that at every node v where it is observed

(S1) $\mathtt{s} \in \mathcal{C}_{\mathcal{G},\mathtt{g},\mathtt{s}}\ \mathtt{v}$: the store is collected at that node;

(S2) $\mathtt{s} \in \mathcal{R}_{\mathtt{res}}\ \mathtt{v}$: some state the report holds at v, under one of its contexts, describes s;

(S3) $\mathtt{s} \in \mathcal{V}_{\mathtt{res}}\ \mathtt{v}$: every `PROVED` check listed at v holds in s, and every `REFUTED` check is false in s.

Whenever the analyzer returns a report, every store that a run of `main` reaches is thus collected, covered by the report and consistent with its verdicts at every node where the run is observed. This is the chain of Section 10.3 read at one store.

The statement mentions no activation and no context. Source executions carry neither, and the report states its guarantee per CFG node (Section 10.1), so both appear only inside the proof, at the second inclusion of the chain. `run_voblint_spine` keeps them visible: it states the chain once for every context policy, together with a representing activation trace and one context it carries whose set contains the reached store, and each shipped policy discharges its two assumptions.

10.5. What a verdict says about executions thy

What the four verdicts mean. By the theorem of Section 10.4, each verdict of a report that `run_voblint` returned becomes a claim about *covered executions*: finite runs of `main` from an initial store in `cinit_stores`, stopped at any point. Because the report's states may also describe stores that no execution reaches, a verdict can only claim what holds whenever an execution reaches its check.

- `PROVED`: whenever a covered execution reaches the check, the condition holds (`run_voblint_check_sound`).
- `REFUTED`: whenever a covered execution reaches the check, the condition fails (`run_voblint_check_sound`). It is not a verified counterexample: it establishes no reaching execution.
- `UNKNOWN`: the analysis decides neither.
- `DEAD`: no covered execution reaches the check: nothing is collected at its node (`run_voblint_dead_check_unreached`), and a check that an execution reaches is never `DEAD` (`run_voblint_check_sound`).

`PROVED` and `REFUTED` hold vacuously when no execution reaches the check, and only `DEAD` claims anything about reachability. The corresponding theorem for `PROVED` and `REFUTED` is:

```
(* src/Executable_Surface/CLI/Analysis_Certified.thy *)
theorem run_voblint_check_sound:
  fixes p :: imp_prog and s0 s :: store
  defines G_def: "𝒢 ≡ declared_global p"
      and Pi_def: "Π ≡ prog_table p"
      and g_def: "g ≡ prog_cfg p"
  assumes s0: "s0 ∈ cinit_stores 𝒢"
      and run: "𝒢, Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)"
      and chk: "next_check residual = Some (l, e)"
      and ans: "run_voblint config p = Analysed res"
  shows "∃c ∈ set (report_checks res). check_label c = l ∧ check_exp c = e
           ∧ s ∈ 𝒞𝒢,g,cinit_stores 𝒢 (check_point c)
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (⟦e⟧ₑ s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (⟦e⟧ₑ s))"
```

Isabelle

Truth and reachability are separate. In the program below, $x = 3n$ is never 1 or 2, so no run reaches the check. Interval cannot see this. It keeps `interval: x=[1,2]` there, where `x > 0` holds, and reports `PROVED`. The verdict is sound: it claims `x > 0` only for runs that reach the check. Int also tracks $x \equiv 0 \pmod 3$, finds the state empty, and reports `DEAD`.

```
fun main() {
  n = __voblint_nondet_int();
  x = 3 * n;
  if (0 < x && x < 3) {
    __voblint_check(x > 0);
  }
}
```

VIMP ↗

So an unreached check is not necessarily `DEAD`. The analyzer reports `DEAD` only where its emptiness test recognizes every state at the node as describing no store. The test is sound (`sound_emptiness_DEAD`) but not exact: two analyses can each admit stores while no store satisfies both (`mcp_empty_v_not_exact`, `DEAD_not_exact`, the latter on a constructed report). An unreached check can thus miss `DEAD` because its state describes stores that no run reaches, as Interval's state above does, or because the test does not recognize that it describes none. The verdict is sound either way.

Arithmetic diagnostics. Division by zero needs no check in the program. At every division and remainder the analyzer asks the query system of Section 7.1 whether the divisor can be zero. It reports an error if the analyses show that it is zero, a warning if they cannot exclude zero, and nothing otherwise. An error is sound as well. Whenever a covered execution reaches its node, the divisor is zero (`run_voblint_arithmetic_refuted`). A warning means only that the analysis could not exclude a zero divisor. At a node v without a diagnostic, every divisor in an expression on an intraprocedural edge leaving v is nonzero in every store collected at v (`run_voblint_arithmetic_safe` also covers the arguments of calls leaving v):

```
(* src/Executable_Surface/CLI/Analysis_Certified.thy *)
corollary run_voblint_arithmetic_intra_safe:
  assumes "run_voblint config p = Analysed res"
    and "s ∈ 𝒞declared_global p,prog_cfg p,cinit_stores (declared_global p) v"
    and "∀d ∈ set (report_diagnostics res). diagnostic_point d ≠ v"
    and "(v, action, w) ∈ intra (prog_cfg p)"
    and "e ∈ set (edge_expressions action)"
    and "divisor ∈ expression_divisors e"
  shows "⟦divisor⟧ₑ s ≠ 0"
```

Isabelle

PART V

Instances and Practice

11. Five Domains and an Order Analysis

Chapter 10 proved that a report is sound regardless of which analyses produced it. A user can choose among the numeric domains Sign, Interval, Parity, and Congruence, their combination Int, and the relational order analysis. The four base domains are textbook material, and the project site lets readers try them interactively, so this chapter treats them briefly. Most of it describes Int, which combines the four base domains and gains precision by letting them exchange facts. The formalization proves that this exchange preserves a value's meaning and shows why narrowing must skip it.

11.1. The implemented domains

The four base domains, Sign thy , Interval thy , Parity thy , and Congruence thy , are the classical non-relational domains [49]; congruences go back to Granger [77]. Figure 11.1 shows one value of each and the integers it stands for. Sign and Parity are finite, and a congruence class cannot grow forever either: growing from a single integer gives it a positive modulus, and every later strict growth replaces that modulus by a proper divisor [49, §4.8]. This argument is not mechanized here. All three use the join as their widening and keep the old value as their narrowing. Only Interval has a widening that jumps ahead and a narrowing that refines (Section 5.1).

domain	value	the integers it denotes
		−6 −4 −2 0 2 4 6 8 10
Sign	≥0	…
Interval	[-2,5]	…
Parity	1+2ℤ (odd)	…
Congruence	2+3ℤ	…
Int	+, [1,9], 1+2ℤ, 1+4ℤ	…

Figure 11.1: One value of each implemented domain, as the analyzer prints it, and the integers it denotes between −6 and 10. Congruence and Parity values print as $c + m\mathbb{Z}$, the integers congruent to c modulo m ($m\mathbb{Z}$ when $c = 0$). An end cell is filled when the set continues beyond the window. The values are illustrative. The Int value, the reduced state of x in Table 11.2, denotes the intersection of its components' meanings, here $\{1, 5, 9\}$.

Adding a domain mainly requires class instances for its carrier and a soundness proof for its operations: it supplies the record `nonrelational_ops` of Section 5.6 and interprets `sound_nonrelational_ops`. The generic construction then derives the abstract and executable transfer functions and their agreement, and `sound_nonrelational_ops.dg_analysis_execI` makes the domain a sound component of the combined state (Chapter 7). Only one of the lemma's premises concerns the domain's values: its initial state must cover the initial stores (for Parity, `parity_cinit_gamma`).

11.2. One loop, five answers

The loop below is the fixture `05-mine_ex410_stride2_parity.vimp`, adapted from Goblint's regression test `56-witness/29-mine-tutorial-ex4.10.c` (revision `d155e9e`) after Example 4.10 of Miné's tutorial [49]. In every run v takes the odd values $1, 3, \ldots, 51$ and the loop exits with $v = 51$, so every check holds.

```
1  fun main() {                                                    VIMP ↗
2    v = 1;
3    while (v < 51) {
4      __voblint_check(v >= 1);
5      v = v + 2;
6      __voblint_check(v <= 52);
7    }
8    __voblint_check(v >= 51); line 8
9    __voblint_check(v <= 52);
10   __voblint_check(v == 51); line 10
11 }
```

domain	records	state at the exit	`v >= 51` line 8	`v == 51` line 10
sign	sign of a value; finite	`v=+`	UNKNOWN	UNKNOWN
interval	bounds; infinite ascending chains	`v=[51,52]`	PROVED	UNKNOWN
parity	evenness	$v=1+2\mathbb{Z}$	UNKNOWN	UNKNOWN
congruence	residue modulo m	$v=1+2\mathbb{Z}$	UNKNOWN	UNKNOWN
int	all four, reduced	`v=51`	PROVED	PROVED

Table 11.1: The stride-2 loop under each domain (claims `dom-stride2-*`), at the two highlighted checks of the listing. Only Int, which combines the bound and the oddness, proves `v == 51`.

Each base domain records a different property of v. Interval finds the bound through widening and narrowing, but its exit state still allows 52, and an interval has no way to say "odd". Parity and Congruence know that v is odd but nothing about its size, and Sign knows only that v is positive. Every answer is sound, yet none of the four proves `v == 51`. Only Int, which puts the bound and the oddness together, does (Section 11.3).

11.3. The reduced product thy

An Int value (`int_dom`) holds one value of each base domain and stands for the integers that all four allow. Running the four side by side would be sound, but each would work with its own value only, so a fact that needs two of them would not be found. Table 11.1 shows this: no base domain alone proves `v == 51`. In the program below, the guards keep x between -2 and 10, and the assignment makes x one more than a multiple of 4. Only 1, 5, and 9 satisfy both, so `x >= 1 && x <= 9` holds. Yet the interval alone still allows -2 and the residue class alone allows -3, so neither component can decide the check. A *reduction* lets the components sharpen each other, as in the reduced product [46], [49, Def. 6.3, Ex. 6.2].

Here the residue class moves the interval's bounds to the nearest values that are 1 modulo 4, giving $[1, 9]$, and the now positive interval makes the sign positive. The reduced value stands for the same integers and decides the check (Table 11.2). Goblint uses a similar reduced-tuple design for integers. Its `IntDomTuple` combines up to six optional components, among them definite values with exclusion sets, intervals, enumerations, and congruences, each enabled by an option. Its `ana.int.refinement` setting offers the same three reduction modes as Int. Goblint defaults to no reduction, and Voblint defaults to the fixpoint mode.

```
fun main() {                                                    VIMP ↗
  n = __voblint_nondet_int();
  x = 4 * n + 1;
  if (x >= -2) {
    if (x <= 10) {
      __voblint_check(x >= 1 && x <= 9);
    }
  }
}
```

component of x	without reduction	with reduction
Sign	$\top$	+
Interval	[-2,10]	[1,9]
Parity	1+2ℤ	1+2ℤ
Congruence	1+4ℤ	1+4ℤ
verdict of the check	UNKNOWN	PROVED

Table 11.2: The value of x at the check of the program above, without reduction and with the default reduction (claims `int-reduction-never` and `int-reduction-fixpoint`). Both columns describe the integers 1, 5 and 9 in the guard's range, but only the reduced value decides the check.

Reduction is also what proves `v == 51` in Table 11.1. Inside the loop, the guard `v < 51` leaves $[1, 50]$, and together with "odd" the reduction shrinks this to $[1, 49]$. The increment then gives $[3, 51]$ instead of Interval's $[3, 52]$, the loop head settles at $[1, 51]$, and after the loop only 51 remains.

A reduction round has two steps. `refine_interval` first combines the range information of Sign and Interval and uses the result to sharpen Sign, Interval, and Parity. `refine_congruence` then uses the residue class, intersected with the parity, to sharpen Interval, Parity, and Congruence. Through `refine_mode` the user chooses whether to skip reduction, reduce once, or reduce until nothing changes. The last mode is the default. Every mode keeps the integers a value stands for (`refine_exact`) and only ever makes the value smaller (`refine_reductive`).

The default mode repeats rounds until the value stops changing. In Isabelle this is a total function: if the rounds never settle, it returns its input. The generated OCaml instead keeps iterating, and no proof shows that it always stops. On some input the analyzer might therefore run forever instead of returning a report, just as it would if the solver did not terminate (Section 12.5). Soundness is unaffected, because the theorem only speaks about reports that are returned.

Int reduces the result of every arithmetic operation and of the intersections and inverse operators of the guard filter. Join, widening, and narrowing work componentwise, and for join and widening Goblint makes the same choice. For narrowing, Voblint's solver contract forces the choice. The verified solver relies on the bracket laws $b \leq a\Delta b \leq a$ for $b \leq a$ (Section 2.5), and reduction, which only makes values smaller, can push a narrowed value below b (`post_narrow_refinement_would_violate_narrow_ge`, checked by evaluation). Goblint's `IntDomTuple` does reduce after narrowing. No proof obliges its lattices to satisfy the bracket laws, so Goblint can make this choice and Voblint cannot.

11.4. The order analysis thy

None of these domains can express that $x \leq y$, because each describes the variables one at a time. The order analysis keeps a set of such pairs, the relational state of Section 5.2.2. A guard between two variables adds a pair. An assignment first forgets the pairs involving the changed variable. It then asks the shared query channel how the new value relates to each variable, and every active analysis, the order analysis included, may contribute an answer (Section 7.4). In the program below, the guard `x < y` tells Interval nothing about either variable, so it keeps `interval: x=⊤, y=⊤` and reports `UNKNOWN` at the first check. The order analysis records `order: {x≤y}` and reports `PROVED`.

VIMP

```
fun main() {
  x = __voblint_nondet_int();
  y = __voblint_nondet_int();
  z = __voblint_nondet_int();
  if (x < y) {
    __voblint_check(x <= y);
    if (y <= z) {
      __voblint_check(x <= z);
    }
    y = y + 1;
    __voblint_check(x <= y);
  }
  w = x;
  __voblint_check(w == x);
}
```

Because the analysis does not combine pairs transitively, it cannot prove `x <= z` under `y <= z`: it holds `order: {x≤y ∧ y≤z}` there and answers `UNKNOWN` (Section 5.2.2). The assignment `y = y + 1` throws away every pair involving y, leaving `order: ⊤`. A copy, in contrast, yields pairs in both directions. At `w = x` the assignment asks, for the variable x itself, whether `x <= x` holds in each direction. The order analysis answers yes to both, because it treats a variable as ordered with itself, so it records `order: {w≤x ∧ x≤w}` and proves `w == x`, where Interval answers `UNKNOWN`. The analysis contract of Chapter 6 accepts this state as it is, even though it is not a map from variables to values.

The base domains record different properties, Int lets them share these facts, and the order analysis proves relations that no domain over single variables can express. The order analysis can also run together with the numeric domains in one combined state (Chapter 7). At each assignment it then asks them how the new value compares with the other variables, and a check is decided by the meet of all their answers. Section 13.3 shows that Interval and the order analysis together prove a check that neither proves alone.

12. From Theorem to Tool

Chapter 10 proved that every report `run_voblint` returns is sound. A user, however, does not evaluate a HOL function. The user writes source text and reads a report in a terminal or a browser tab. This chapter shows how the proved function becomes the analyzer that runs, and where the proof stops between the source text and the displayed report.

12.1. From theorem to code thy

Chapter 9 replaced the two objects of the argument that cannot run with executable versions (Section 9.5, Section 9.6). Every function that `run_voblint` uses therefore has code equations, and Isabelle's code generator [47] can export it. One `export_code` declaration emits `run_voblint`, `render_report`, and the constructors and selectors a caller needs into one OCaml file, `Voblint_Generated.ml`. This module contains the analyzer core that both delivered tools run. The command-line tool and the playground reach it only through the facade `cli/voblint.ml`, and no handwritten code takes part in computing an analysis result.

The command-line tool links the module with handwritten adapters into a native program. For the playground, `wasm_of_ocaml` compiles the same code to WebAssembly, which a web worker in the reader's browser calls (Figure 12.1). Both tools thus run the generated code of `run_voblint`, the function the theorems are about, up to the code generator and the compilers (Section 12.5).

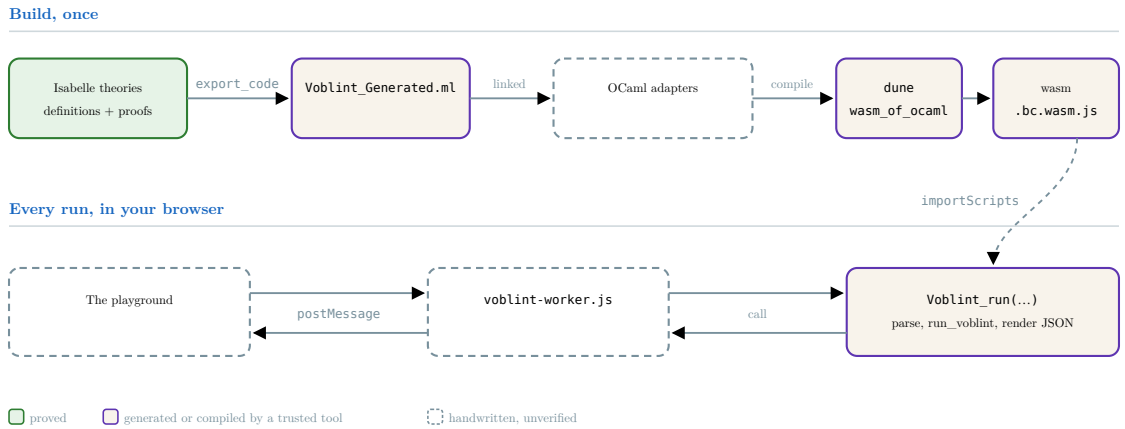


Figure 12.1: How the generated `run_voblint` reaches the browser. Build: `export_code` writes `run_voblint` into `Voblint_Generated.ml`, which `dune` and `wasm_of_ocaml` compile with the handwritten adapters into a WebAssembly bundle. Run: a web worker calls the bundle and returns the answer as JSON. Adapted from the project site.

12.2. The generated code

The exported module, `Voblint_Generated.ml`, runs, but it follows the proofs rather than the habits of OCaml programmers. It has 14,164 lines. Type classes become records of operations passed as arguments, so polymorphic operations such as equality go through an indirect call. Finite sets are unsorted lists, and the finite maps the code uses are association lists, so membership, insertion, and lookup scan them. The solver keeps its valuation as a function. Each update wraps the previous valuation in another closure, so a lookup may traverse a chain as long as the number of earlier updates, and superseded closures stay reachable.

The proofs guarantee that this code computes the proved function, but they say nothing about its running time or how it computes the result. One such cost was a defect. A sampling profile of a program of 256 chained assignments, taken on one machine, attributed most of the CPU time to the compiler, which ran once per equation evaluation. The cause was `init_publications`. On every equation evaluation it tests whether the unknown is the program's entry, and its definition names the entry as `cfg_entry (prog_cfg p)`. The generated code evaluates this term as written, so it compiled the whole program to find a node that is always the entry of `main` (Figure 12.2).

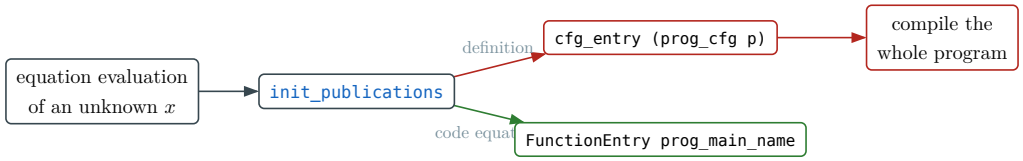


Figure 12.2: The entry test of `init_publications` in the generated code. Generated from the definition (red), it compiled the program at every equation evaluation. Generated from the proved code equation `init_publications_code` (green), it compares with a constant.

Isabelle lets a theory replace the equation that code generation uses for a constant with any proved equation for it. `init_publications_code` states `init_publications` with the entry node `FunctionEntry prog_main_name` written out and follows from `cfg_entry_compile_prog` by simplification. The definition and every theorem about it stay unchanged, and no trust is added (Section 12.5). The trace hook of Section 12.4, by contrast, adds a mapping to handwritten OCaml to the trusted base. The proof establishes that the equations agree, and the unchanged corpus and solver traces only confirm that the code generator uses the new one. On the same machine, the generated chain of 256 assignments then ran 13 times faster, and a program of 512 assignments in procedures of eight statements 18 times faster (recorded measurement).

The remaining cost is spread over report construction, printing, and dictionary calls. Three routes would remove more of it, at increasing price. Further proved code equations change the code without changing a theorem. Isabelle's library implements sets by red-black trees, but only for element types with a linear order. Abstract values, check results, and entry-state contexts have only their lattice order. Replacing the solver's function-valued table with a finite map would remove the closure chains, but it would change the state of the vendored solver and require its proof to be redone. This thesis aims at a sound analyzer rather than efficient generated code, so the code was not measured further.

12.3. The unverified shell

The generated `run_voblint` takes a configuration and a syntax tree and returns a typed answer, so unverified code has to surround it on both sides. Before it, an `ocamllex` lexer and a Menhir parser turn the program text a user writes into an `imp_prog` and write each check's source position into its label. After it, `render_report` turns the report's states into text. Handwritten code then prints the command-line report, each check at the position its label carries, or serializes the answer as JSON, from which the playground draws the graph and places the verdicts and arithmetic diagnostics in the editor. Isabelle's code export maps HOL's mathematical integers to arbitrary-precision Zarith integers, so the generated code computes with VIMP's integers (Section 3.3). The command-line tool and the playground share the parser, the generated module, and the code that turns a request into an analysis configuration. They differ in their surroundings: the command-line tool manages processes and files, and the playground's worker exchanges JSON with the page.

For the program below, Figure 12.3 shows what crosses this boundary: the call `run_voblint` receives and its answer after `render_report` has turned the states into text, serialized by the same code the playground uses.

```
fun main() {
  k = 10;
  q = 100 / k;
  d = __voblint_nondet_int();
  y = 100 / d;
  __voblint_check(0 < q);
}
```

VIMP

call

```
{
 "as": ["Interval_Analysis"],
 "rule": "Globals_Warrow",
 "ctx": "Ctx_None",
 "pg": "Program_Globals_Flow_Sensitive",
 "p": {
  "declared_global_vars": [],
  "prog_main": {
   "Seq": [...×2]
  },
  "prog_table": []
 }
}
```

json

run_voblint

answer

```
{
 "Analysed": {
  "res_cfg": {...},
  "res_contexts": [...×1],
  "res_states": [...×8],
  "res_routes": [],
  "res_checks": [
   {
    "check_point": {"Statement":4},
    "check_label": [6,3],
    "check_exp": {"Less":[{"N":0},{"V":"q"}]},
    "check_verdict": {"Lifted":"Check_Proved"}
   }
  ],
  "res_globals": [...×1],
  "res_diagnostics": [...×1]
 }
}
```

json

Figure 12.3: What the generated analyzer receives and returns for the program above (claim `shell-json`). The call carries the configuration and the syntax tree `p` the parser built, of the kind Figure 3.3 shows in full; the answer carries the report with its states rendered as text, here with its one check in full. Everything before the call, and the rendering from the answer on, is unverified. `{...}` and `[...×n]` fold parts of the JSON.

The playground is a static page. Its toolbar selects the configuration argument of `run_voblint` and, independently, whether the surrounding tracer is on. `run_voblint_source_sound` quantifies over that configuration whenever `run_voblint` returns an Analysed report. Figure 12.4a shows one run.

12.4. Tracing the exported solver thy

To explain a run, the command-line tool and the playground can trace the solver's steps: which unknown is queried when, which update destabilizes which unknown (Table 9.3), where widening applies, as Goblint's tracing of its top-down solvers reports them. Tracing must leave the definitions and proofs of the vendored solver [4] untouched. Voblint therefore supplies traced versions of its code equations that report each step through one constant, `trace_event`, which is () in the logic. Because it is (), each traced equation is proved equal to the one it replaces (for example `solve_rec_c_traced`; `cs_route_traced`, `trace_route`, and `trace_run` do the same for routing and the reading of the result). Code export replaces the original equations with these, so one generated analyzer serves traced and untraced runs alike. In OCaml, `trace_event` is mapped to a hook that records the event when tracing is on. This mapping adds one target mapping to the trusted base: the hook must return, raise nothing, and leave the solver's values alone. The trace, and the playground's replay of it, are unverified observations and never feed back into a result. Figure 12.4b shows one step of the replay.

12.5. The trust boundary

Three questions decide what a run of the delivered analyzer establishes, and Figure 1.4 places the main components on each side.

What is proved. `run_voblint_source_sound` and the verdict theorems of Section 10.5 hold for every report `run_voblint` returns, from the syntax tree it receives to the semantic report. The proof covers the compiler, the well-formedness test, the vendored solver with its executable refinement and the executable state carrier.

What is trusted. Two groups lie outside the proof. The first is the handwritten code around the generated analyzer: the lexer, the parser, the translation of a request into a configuration, and the presentation, including the placement of arithmetic diagnostics. `render_report` is generated and passes verdicts and diagnostics through unchanged (`render_report_columns`), but no theorem relates its state strings to the semantic values. The second is the foundation: Isabelle's kernel, its code generator with the target mappings (`code_printing`), the trace hook among them, the OCaml and WebAssembly toolchains, Zarith or, in the browser, a JavaScript replacement of it, and the browser. Only the tests of Section 13.4 cover the handwritten code.

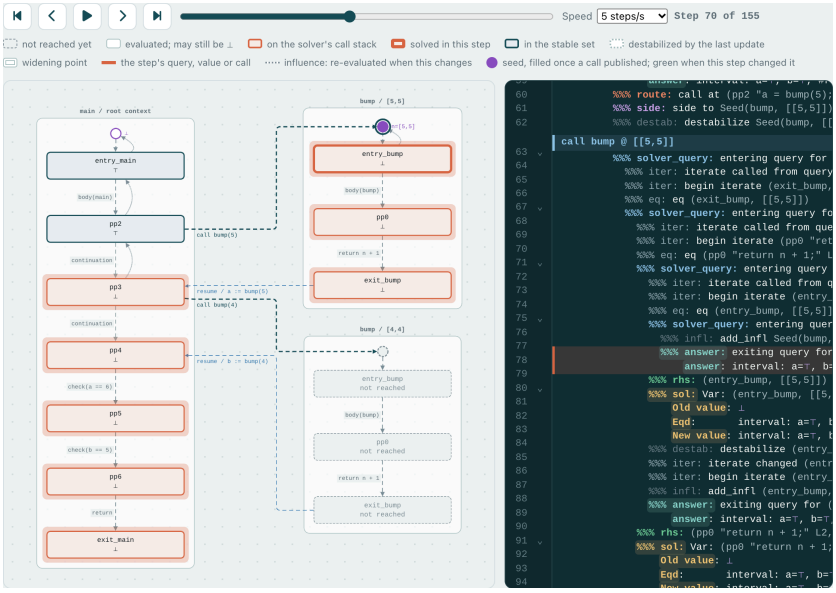
Whether the definitions are adequate. A proof establishes consequences of `pstep`. It cannot show that `pstep` is the language a reader has in mind; Section 3.3 argues this for the fragment of C that VIMP models.

What the soundness theorem does not establish. A parse error, an `Invalid_Activation`, `Malformed_Program`, or `No_Answer` answer, a timeout, an aborted code equation, and a run that never returns yield no Analysed report, so the theorem gives no verdict. A hang may come from the solve, from the fixpoint reduction of Int, whose generated code may loop where the HOL function returns (Section 11.3), or from the toolchain.

Both tools thus run generated code for the `run_voblint` that Chapter 10 proves sound.



(a) one run in three views: the check's verdict in the editor, the state inspector at the check, and the solved graph. Settings `analysis interval, globals warrow, context none` open this run.



(b) one step of the solver replay for the calls of `bump` (Table 9.3): the graph shows each unknown's value at this step, and the trace beside it marks the step's line. Settings `analysis interval, globals warrow, context entry-state` open this run.

Figure 12.4: The playground. Both panes run the generated analyzer of Figure 12.1 in the reader's browser. The screenshots are illustrative.

13. Evaluation

Chapter 10 proved that every report `run_voblint` returns is sound. A machine-checked theorem does not need testing to show that it holds, but it can hold for uninteresting reasons. Its premises may never be met together, its obligations may be weaker than they look, or it may exclude only errors that no real analyzer makes. Following J. Kim [78, §6], this chapter first asks two questions about the theorem: whether its premises can hold together, and whether its obligations exclude real errors. It then shows on concrete programs what precision the analyzer reaches under the theorem, and how the code outside the theorem is tested.

The evidence differs in strength. A *machine-checked* result is an Isabelle theorem and holds for every input its statement ranges over, under its premises. An *evaluated* result is an Isabelle fact proved by `eval`, which runs the generated code of a concrete solve: Isabelle checks it, but it trusts the code generator and concerns one input. An *executable* result records that one program, run with fixed analysis settings, produces one answer. Its run also passes through the parser and renderer outside the theorem (Section 12.5). A *source inspection* or *repository measurement* supports statements about the structure or size of the development.

13.1. Non-vacuity thy

Evidence: machine-checked, with the concrete solves evaluated. A theorem whose premises are unsatisfiable holds vacuously. The end-to-end theorem assumes an initial store, a source run, and an `Analysed` answer, which the analyzer gives only if its solve terminates. D. Marmsoler [79, §9] shows with one small executable instance that the locale assumptions of a formal model can be satisfied. We use the same kind of instance for the premises of the end-to-end theorem.

The instance is the two-call program of Figure 3.1 under Interval, entry-state contexts, and warrowing. The theory builds the source run that returns from `bump(5)` and `bump(4)` and stops before the first check. It computes the answer by evaluation: both checks are `PROVED`. `nv_source_certified` instantiates `run_voblint_source_sound` with every premise discharged. Its conclusion gives a node at which the run is observed (`at_node`) together with the three memberships there. In the same way, `nv_check_proved_sound` instantiates `run_voblint_check_sound` at the check `a == 6`. For the reachability claim, a second program sets `x = 1` and guards a check by `x < 0`; the analyzer marks it `DEAD`, and `nv_dead_unreached` instantiates `run_voblint_dead_check_unreached` to conclude that the collecting semantics at that node is empty. The source runs are proved by rule application, and the compiled graphs and the answers are proved by `eval`. The instances cover a definite truth verdict, `PROVED`, and the reachability verdict `DEAD`.

13.2. Do the obligations exclude real errors?

Evidence: machine-checked, and executable against a defect documented upstream. The coverage contract and the domain obligations are long, so a reader may ask whether each is needed. For two coverage conditions the theories give a counterexample. Reading a callee's result at the caller's context meets the remaining obligations (`ret_weak_obligations`) but misses a store a run reaches (`return_at_caller_context_unsound`). Dropping TOTAL admits a claim that meets the others (`tot_weak_obligations`) but misses a store collected at a continuation (`total_dropped_unsound`, Section 4.4). These are conditions we selected, so they do not show that every premise of the development is needed.

A domain obligation, in turn, excludes a defect that occurred in Goblint. Chapter 1 opened with issue 1156, in which Goblint claimed `c % 2 == 1` for $c \in \{-5, -7\}$, although C's truncating remainder gives -1 for both values. Pull request 1161 restricted the cases in which the congruence domain returns a constant remainder and added the program as regression test `37-congruence/14-negative.c` [13], which marks the first check as unknown and the second, `c % 2 == -1`, as not yet provable.

In Voblint, the pre-fix remainder cannot be proved sound. The Congruence remainder must meet `congruence_mod_sound`: for any two concrete values, their remainder `c_mod` lies in the concretization of the abstract remainder. `c_mod` truncates toward zero, as C11 specifies (Section 3.3). The pre-fix answer, the constant 1 for $(1 + 2\mathbb{Z}) \bmod 2$, violates the obligation at -5 (`prefix_congruence_mod_unsound`). The theorem holds for every abstract remainder that returns this constant. Pull request 1161 reports that Goblint's pre-fix operator returned it, so an obligation of this form would have rejected that operator as well. Our fixture transliterates the test into VIMP, whose remainder also truncates toward zero (Figure 13.1). Congruence alone knows only $c \in 1 + 2\mathbb{Z}$ and decides neither check. Int also knows that c lies in $[-7, -5]$, refutes the first check, and proves the second.

```
1 fun main() {                                  VIMP ↗
2   top = __voblint_nondet_int();
3   c = -5;
4   if (top) {
5     c = -7;
6   }
7   __voblint_check(c % 2 == 1);
8   __voblint_check(c % 2 == -1);
9 }
```

check	Congruence	Int

`c % 2 == 1`	UNKNOWN
	`congruence: c=1+2ℤ`
		REFUTED
		`int: c=signs:-`
		`intervals:[-7,-5]`
		`parities:1+2ℤ`
		`congruences:1+2ℤ`

`c % 2 == -1`	UNKNOWN
	`congruence: c=1+2ℤ`
		PROVED
		`int: c=signs:-`
		`intervals:[-7,-5]`
		`parities:1+2ℤ`
		`congruences:1+2ℤ`

Figure 13.1: Goblint's regression test for pull request 1161 in VIMP, and the verdict and state at each check with Congruence alone and with the Int product, both without contexts. Concretely, `c % 2` evaluates to -1 on both executions.

13.3. Precision and cost on concrete programs

The theorem says nothing about precision, since an analyzer that answers `UNKNOWN` everywhere satisfies it (`unknown_everywhere_sound`). We therefore show on concrete programs what each mechanism gains. Each example below fixes the concrete behavior first, then shows what one mechanism keeps or loses, and supports a claim about its program only.

Contexts. *Evidence: executable, and evaluated.* Figure 8.3 showed which calls each context policy separates. `sign_k2_strictly_more_precise_than_k1_at_g` proves a strict separation on one program: the Sign value of a parameter at a procedure entry is strictly lower under call strings of length 2 than under length 1, with the component values computed by `eval`. The theorem uses Sign because its widening is its join and its narrowing returns the current value, so the difference cannot come from widening.

The gain from contexts is not proportional to their cost. `down(100)` recurses to `down(0)`, so `n >= 0` holds at every call. Call strings of length 100 give 102 procedure copies, and the check is PROVED. Length 99 gives 101 copies, but one context still stands for the deepest calls, widening removes the lower bound there, and the check is UNKNOWN (claims `cost-down-k99` and `cost-down-k100`).

Modularity. *Evidence: source inspection.* Adding the order analysis required three proofs about its local specification (`order_spec_sound`, `single_entry_order_spec`, `relc_qry_sound`), a lattice instance, an entry in the analysis manifest (Chapter 14), and a few class instances for the command-line layer. No theory of the framework or of the numeric domains refers to it outside document text. A new numeric domain proves its lattice laws and its forward, backward, and query operations sound once (`sound_nonrelational_ops`), and the generic construction derives its transfer functions and their soundness (Section 11.1).

program	join	join per origin	warrow	warrow per origin	bounded narrowing
two call sites: `p(1); p(2);`, check `x <= 2` in `p`	PROVED `interval: x=[1,2]`	PROVED `interval: x=[1,2]`	UNKNOWN `interval: x=[1,+∞]`	PROVED `interval: x=[1,2]`	PROVED `interval: x=[1,2]`
call in a loop: `g(x)` for `x` from 0 to 9, check `x < 10` in `g`	PROVED `interval: x=[0,9]`	PROVED `interval: x=[0,9]`	UNKNOWN `interval: x=[0,+∞]`	PROVED `interval: x=[0,9]`	PROVED `interval: x=[0,9]`
growing recursion: `f(x)` calls `f(x + 1)` from `f(0)`, check `x >= 0`	no answer in 5 s	no answer in 5 s	PROVED `interval: x=[0,+∞]`	PROVED `interval: x=[0,+∞]`	PROVED `interval: x=[0,+∞]`
shrinking recursion: `f(a)` calls `f(9 / (a + 2))` while `a < 4`, from `f(-1)`, check `a <= 9`	PROVED `interval: a=[-1,9]`	PROVED `interval: a=[-1,9]`	PROVED `interval: a=[-1,9]`	UNKNOWN `interval: a=⊤`	UNKNOWN `interval: a=⊤`

Table 13.1: Update rules on four programs (Interval, no contexts; bounded narrowing at its default bound 5), as on the project site, where each cell opens its run in the playground. Each cell is the verdict and state of the check row. Every check holds in every execution, so an `UNKNOWN` is lost precision. "No answer in 5 s" means the command-line `--timeout 5` stopped the solve.

Update rules. *Evidence: executable, and evaluated for a system with two writers to one global.* Table 13.1 runs the five update rules of Section 9.4 on four programs whose checks hold in every execution. In the first row `p(1)` and `p(2)` publish to the same entry seed. Warrow joins the two contributions before widening, so the second looks like growth, and the upper bound goes to $+\infty$. Per-origin warrowing widens each call site's contribution separately. Each origin writes one constant, so nothing grows. `two_writer_slot_warrow_loses_upper_bound` and

`two_writer_slot_warrow_per_origin_exact` prove the same effect on a system with two writers to one global by `eval`. No rule is best on every row. In the last row the fixture header attributes the loss to the recursive call's own contribution: per-origin warrowing widens it to $-\infty$, `a + 2` may then be zero, and the division loses all information. We attribute Warrow's result to the fact that it joins in `main`'s argument -1 first, which keeps the lower bound. On the growing recursion the joining rules give no answer within the limit. The argument grows without bound, so we expect, without a proof, that their solve diverges (Section 9.6); the timeout alone does not show this (Section 12.5). Bounded narrowing, at its default bound 5, reaches the verdicts of per-origin warrowing on all four programs.

Refinement inside Int. *Evidence: executable.* Int combines the four base domains in one value and refines them against each other (Section 11.3). In the program below, `q` is $4n + 1$ for some $n \in [0, 20]$, so `r` is 1.

VIMP

```
fun main() {
  n = __voblint_nondet_int();
  if (0 <= n && n <= 20) {
    q = (12 * n + 3) / 3;
    r = q % 4;
    __voblint_check(r == 1);
    __voblint_check(r > 0);
    __voblint_check(r % 2 == 1);
  }
}
```

Interval alone bounds `r` to `[0,3]` and reports `UNKNOWN` at `r == 1`. Congruence alone keeps `r` in `1+4ℤ`, which admits -3 for a negative `q`, and reports `UNKNOWN`. Int intersects the two to `1` and proves the check (claims `int-refine-*`). Figure 13.1 and Table 11.1 show the same effect.

Queries between analyses. *Evidence: executable, and evaluated.* Separate analyses refine each other only through queries (Chapter 7). In the following program they do so in both directions:

VIMP

```
fun main() {
  c = __voblint_nondet_int();
  if (0 < c) { x = 0; y = 10; } else { x = 20; y = 30; }
  z = (x <= y);
  __voblint_check(z == 1);
}
```

Int alone joins the branches, so `z` lies only in `[0,1]` and the check is `UNKNOWN`. The order analysis alone cannot compare a variable with a constant and reports `UNKNOWN`. Together, at both assignments to `y` the order analysis asks Int whether x is at most the assigned constant and records the pair (x, y) on both branches. At `z = (x <= y)` Int asks the query `EvalInt` $(x \leq y)$, the order analysis answers 1, and the check is `PROVED` (claims `coop-mutual-*`). Int asks at every assignment, and at `z = (x <= y)` the answer is decisive. On a check of `x <= y` itself, the order analysis would answer the check directly, though it still needs Int's answers at the assignments to `y`. On two related programs with Interval in place of Int, `coop_demo_needs_both`, `order_asks_interval_alone`, `order_asks_order_alone`, and `order_asks_needs_both` prove by `eval`, one direction each, that the check is proved only with both analyses.

13.4. Tests of the unverified parts

Evidence: executable, and repository measurement. For Voblint's handwritten code around the generated analyzer (Section 12.5), tests are the only project-local evidence. The theories under `src/` contain 58,262 physical lines in 203 files, including comments and document text but excluding generated theories, and the analyzer that runs is the 14,164-line generated OCaml module. Around it lie 4,904 lines of handwritten OCaml under `cli/`, not counting the lexer, parser, and printer that a script generates from a grammar description.

A regression corpus runs the analyzer end to end, and property tests exercise the frontend. Part of the corpus is adapted from Goblint's regression tests, whose origin the fixture header records. The corpus holds 377 VIMP fixtures in 26 groups. Each fixture states its command-line flags and the verdict expected at each check. Cases in `precision/` pin the exact verdict at each check, which is usually definite. In `soundness/`, the program has executions on both sides of the check, so `UNKNOWN` is the only sound answer. In `known-imprecision/`, the concrete result is fixed, but the abstraction cannot establish it, and the header names the mechanism that loses the information. The playground's *Examples* menu opens every fixture with its settings, so a reader can inspect its solve and change the analysis parameters. The remaining 37 fixtures pin output other than a definite verdict: snapshots of the rendered graph or state, rejection of malformed input, and solves expected not to finish within their time limit. The runner matches results by source line, which tests the position bookkeeping of Section 12.3, and distinguishes a missing report row from a `DEAD` one, so a check the compiler dropped cannot pass as proved unreachable. Some fixtures pin the conventions of Table 3.1 at the analyzer's output, for instance `10-arithmetic/precision/07-signed_division_totalization.vimp` (truncating division and remainder). They check verdicts, not concrete runs, since the development has no executable form of `pstep`. Golden files pin the solver trace (Section 12.4) of three programs step by step and expose changes in the generated solver's execution order. When pull request 256 published a call's seed before reading its exit at nodes where one call returns, as Goblint does at every call, the trace of the running example (Interval, entry-state contexts, warrowing) shrank from 241 to 155 solver steps. The playground replays such a trace on the graph (Figure 12.4b).

The parser is trusted to hand the analyzer the program the user wrote. A parser bug could make the analyzer report verdicts for a different program. Property tests therefore print random syntax trees built from the constructors exported from Isabelle, the type the generated analyzer receives, and require the parser to read back the same tree. They also require the parser to finish on mutated programs without a crash. A round trip cannot detect a misreading of the grammar that parser and printer share, and no test relates the parsed tree to the program a user meant to write. CI runs the corpus and the property tests on every pull request and push to the main branch.

Together, these results show that the end-to-end theorem applies to a concrete run, that two selected coverage conditions cannot simply be dropped, and that one domain obligation excludes a real defect. Each precision example concerns one program, the tests reach only what the corpus covers, and none of this shows that `pstep` is the intended semantics of VIMP (Section 16.2).

14. Engineering and Tooling

A formalization of this size needs work around the proof assistant as well as inside it. Its theories must be formatted and checked for unfinished proofs, its names must stay consistent with the prose that cites them, and a new contributor must be able to install everything the build needs. For Isabelle, comparatively little project-level tooling is available outside the distribution itself. Isabelle users have noted the resulting gap as well, for instance in a comment on a recent blog post by Paulson [80], which observes that Isabelle's own development uses almost no standard tooling [81].

The need for such tooling grows with AI assistance. Many proofs and a large share of the surrounding code of this development were written by agents (see the statement on the use of generative AI). An agent can produce a plausible change quickly, but it cannot be trusted to keep the cross-references of a large development consistent, and a reviewer cannot read every line it writes. Deterministic tooling around the agent made its work usable at this scale. An agent's change counts as done only when these checks pass, so the checks, not the agent, decide whether the repository is consistent.

This chapter describes the published repository, and the tools and upstream changes made for this thesis and published separately. They are engineering contributions: nothing in them is proved.

14.1. The Voblint repository

Everything this thesis describes is published in one repository: the Isabelle theories, a submodule reference to the vendored solver, whose repository is private (Section 14.3), the exported analyzer with its command-line tool and playground, the regression corpus, and the sources of this thesis. Each release is archived on Zenodo with a citable DOI [48]. The project site hosts the playground, the explainer, and the rendered theories that every Isabelle name in this thesis links to.

A development of this size drifts when one fact is written down twice. The repository therefore keeps a few sources and derives everything else from them (Figure 14.1). One grammar file yields the Isabelle syntax of VIMP, the analyzer's parser and its printer. The parser lies outside the proof and hands the proved analyzer its syntax tree (Section 12.5), but it and the Isabelle syntax are generated from the same language description. An analysis manifest names each analysis, its value type and the theories that prove it sound. From it, a generator writes the registration of each numeric domain (Section 11.1) and the combined state of Chapter 7, and Isabelle checks every generated proof. The generator exists because the repetition lies outside what a theory can express: each domain is registered once per context policy it supports, for every update rule, and the combined state has one field per analysis, of differing types. With dependent types, a heterogeneous list indexed by these types could express it inside the logic [82, ch. 9]; HOL's simple types cannot. `run_voblint` itself reaches OCaml through `export_code` (Section 12.1). Because the derived files are

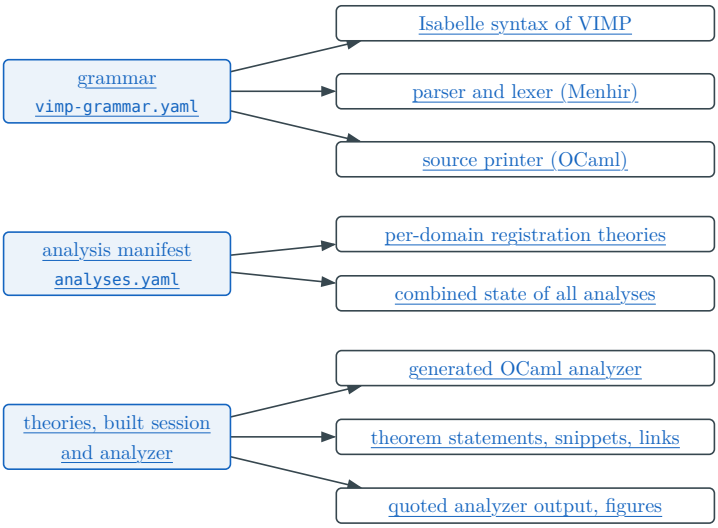


Figure 14.1: The sources of the repository (blue) and the files derived from them; each box links to its file. Each derived file is generated during the build or committed with a check that regenerates it; the link map is also checked against the published site.

checked against their sources, a stale name, statement, or quoted output fails the build instead of reaching the reader (Figure 14.1).

This thesis is held to the same discipline. Each Isabelle name in its Typst sources goes through a helper that a check resolves against the theories and links to the rendered theories. Theorem statements are lifted from the theories and compared with what the built session proves, quoted analyzer results are rerun against the analyzer, each VIMP listing's playground link is decoded from the PDF and compared with the listing, and repository figures come from a measurement script. A rename, a changed statement, or a changed result therefore fails the build, and another check warns when the text shows a statement before the entities it uses.

The checks run in layers ordered by cost (Figure 14.2), from hooks triggered by the staged files to continuous integration, which reruns the checks on a fresh runner, reusing cached results for unchanged inputs, together with the tests of Section 13.4. Only the main branch is deployed, and only after the rendered theories, the formalization PDF, and this thesis build. All layers invoke the same-named Pixi tasks, and a further check keeps the local `verify` task and continuous integration from diverging.

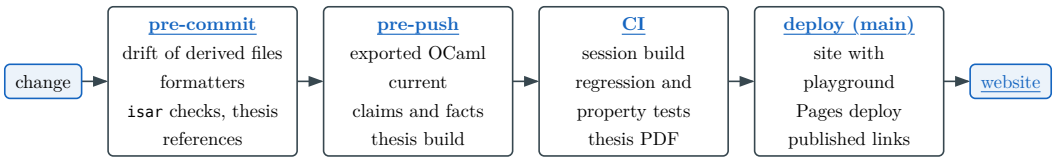


Figure 14.2: The layers of checks between a change and the published site, with representative checks; the published links are checked after deployment. Each box links to its configuration.

14.2. `isar-tools`: source tooling for Isabelle

Isabelle checks what a theory means, but not how a project is maintained: whether every theory is reached by a session, whether a proof was left unfinished, whether two lemmas state the same fact, or whether a locale still says what it said before a rename. `isar-tools` [83] answers such questions from the sources alone. Written for this thesis and published separately, it reads sources, build logs, and rendered HTML as text and never runs Isabelle, so it also reads theories that do not yet check and runs all its checks in a pre-commit hook.

Its formatter is designed to change only layout. Its checks, grouped by topic, report unfinished and unclosed proofs, theories no session reaches, malformed `ROOT` files, and, when named, unused or duplicated lemmas, unused imports, leftover proof-search commands, and broken links. This repository runs all of them. Project views list the sessions, the import graph, the class and locale hierarchy, and every named declaration, and statistics report sizes and proof counts. An extraction layer was built for the remaining tooling of this repository, in particular to keep the website and this thesis in sync with the theories: the theorem statements of this thesis come from `isar project extract`, the notation tables of the thesis and the explainer from `isar project notation`, and the links into the rendered theories from `isar project anchors`. Among related tools, the Isabelle Linter [84] runs inside Isabelle and lints proof style, and `isabelle-query` [85] parses the sources without Isabelle and answers queries about entries, call graphs, and `sorry`s.

14.2.1. A silent failure in locale headers

One check addresses a mistake the batch build cannot see. Inside the header of a `locale`, an identifier Isabelle does not know is read as a free variable and generalized. An assumption that cites a renamed or misspelled constant therefore still builds, but now quantifies over an arbitrary function. In Figure 14.3, the assumption cites a misspelling of the constant defined above it, and `isar check locales` reports the identifier.

The check flags identifiers in a locale header that appear to be neither parameters nor locally bound and have no matching declaration. It approximates Isabelle's inner syntax lexically and is therefore a heuristic.

```
1 definition gamma_sign :: "int ⇒ int set" where                                Isabelle
2   "gamma_sign x = {x}"
3 locale sound_domain =
4   fixes step :: "int ⇒ int"
5   assumes sound: "x ∈ gamma_sgn y ⟹ step x ∈ gamma_sign (step y)"
```

```
Demo.thy:8:27: locale-free-variable: gamma_sgn in the header of locale sound_domain is no parameter, bound
variable, or name used elsewhere; Isabelle generalizes over it as a free variable
```

Figure 14.3: A locale assumption citing a misspelled constant, which Isabelle accepts, and the report of `isar check locales` on this theory, which a test reruns.

14.2.2. A hidden cost: re-elaborated theories

A second problem also leaves the build green. A session inherits only the heaps of its parent chain, so a library theory that is not an ancestor is elaborated again in every session that imports it. In this repository, this once cost 19% of a clean build. `isar stats build` finds it in a build log, and continuous integration enforces a budget per library session.

14.3. Changes to the verified solver

Voblint vendors the top-down solver formalization of Tilscher et al. [4] as a submodule. Four changes made for this thesis, each useful to any downstream user, were proposed upstream.

- The first restores the build under Isabelle2025-2 [86].
- The second removes the well-foundedness assumptions from the solver's `widening` and `narrowing` classes; termination is proved in locales that state their own well-foundedness assumptions [87]. Those assumptions say that widening and narrowing stabilize, which only termination proofs use. Partial correctness needs the four order laws alone, so a domain does not have to provide a stabilizing widening (Section 5.1), and no domain instance has to prove a property that no soundness theorem uses (Section 9.6).
- The third renames short record fields and constructors, such as `c`, `W` and `N`, that took these names away from downstream theories, and adds one interface theory per solver family [88].
- The fourth adds solver facts that Voblint had proved on its side, although they speak only about the solver's own definitions [89], among them that a returned run of the executable solver lies in the solver's domain (`solve_dom_of_solve_c`) and that a terminating solve returns a finite set of stable unknowns (`finite_stabl_solve`), both used by `td_certified_solver` (Section 9.3.3).

At the time of writing, all four pull requests await review. The vendored copy carries the second to fourth on an older upstream revision; the first repairs a later upstream change. The solver's repository is private and accessible only by invitation, so the development cannot yet be built from public sources alone. There are plans to publish the solver in the Archive of Formal Proofs, which would make the whole development buildable by anyone.

PART VI

Assessment

15. Related Work

Section 1.2 compared Voblint by what its theorem covers, first with unverified analyzers and then with mechanized ones (Table 1.1, Table 1.2). This chapter compares its mechanisms with the closest lines of work: verified analyzers, mechanized abstract interpretation, verified fixpoint solvers, context sensitivity, and Goblint together with local-trace semantics. The comparisons are based on a web and OpenAlex search with full-text reads of the closest papers, not on a systematic review.

15.1. Verified analyzers: Verasco and the CompCert line

CompCert and Verasco state their theorems about the tool that users run, and Voblint follows their pattern for a language with recursive procedures and context-sensitive calls.

CompCert is a compiler from Clight, a large subset of C, to PowerPC assembly, programmed and proved correct in Coq, now Rocq [32]. Its theorem is semantic preservation: the compiled code behaves as the source semantics specifies, so safety properties proved of the source hold for the executable. The executable compiler is extracted from Coq to Caml and linked with handwritten, unverified parts, and Leroy lists what remains trusted: the semantics of Clight and of the PowerPC assembly, the unverified CIL-based parser, the assembler and the linker, and Coq's extraction together with the Caml compiler and run-time system [32, §5]. Voblint's guarantee has the same shape: a theorem about the function the tool runs, code generated from the prover, and an explicit list of trusted components (Section 12.5). Compiler correctness preserves the behavior of a program. Analyzer soundness only over-approximates it, so a coarser result remains sound.

Verasco is a Coq-verified analyzer for most of ISO C99, excluding recursion and dynamic allocation, that proves the absence of run-time errors [33]. Its abstract interpreter iterates over the structure of C#minor, a CompCert intermediate language. Modular interfaces separate the abstract state from an extensible combination of numerical domains, and CompCert's semantic preservation extends the guarantee to compiled code.

Verasco covers a far larger language, so the comparison concerns design only. Voblint's language is small, but recursion and context-sensitive calls are central to it: calls become equations over (program point, context) unknowns, and a generic solver computes their solution. Verasco reanalyzes functions at every call site and raises an alarm at a possible recursive call (Table 1.2, [33, §4]). Its extracted analyzer runs with unbounded fuel, and termination is proved only on paper [90, §§3.1.2, 4.3.1], so its theorem, like Voblint's (Section 9.6), covers only the results it returns. Both state the soundness of abstract operations through their concretization.

The earlier value analysis of Blazy et al. [36] does not verify its fixpoint iterator. An untrusted OCaml implementation of Bourdoncle's iteration strategy [61] computes a candidate, and a verified checker accepts it as a post-fixpoint or falls back to top [36, §4]. Voblint instead runs a solver whose partial correctness is proved [4]. The checker leaves the

iteration heuristics free and tests every result at run time. A verified solver needs no such test, and its proof holds for that one algorithm.

Witness validation also checks results instead of the analyzer. A verifier that claims a program correct exports a *correctness witness*, invariants at program locations, and an independent validator re-establishes the result from them. In the 2016 software verification competition, ten of 13 verifiers claimed correctness for some programs known to contain a bug [91, §1]. Saan et al. [92] build such a validator on Goblint's abstract interpreter. Such a validator confirms one result at a time and, unlike the checker of Blazy et al., is itself unverified.

CompCert hides its dataflow solvers behind a common Coq module type whose clients rely only on fixpoint properties of the result [93], and La Spina et al. [94] verify two solvers based on weak topological orderings against it, and CompCert's dataflow analyses then run with these solvers. Both works are intraprocedural. Voblint consumes its solver through a fixpoint guarantee in the same way, for side-effecting, context-indexed systems, where the certificate `part_post_solution` also accounts for contributions to global unknowns.

15.2. Mechanized abstract interpretation

Abstract interpreters have been mechanized in several provers, with different ways of factoring their proofs for reuse. Voblint follows the design of Nipkow and Klein's generic interpreter for its domains and splits the rest into analysis, context policy, and solver.

Nipkow and Klein develop abstract interpretation in Isabelle/HOL over annotated commands of the While language IMP [6], [53]. A collecting semantics annotates each program point with a set of states, and an abstract interpreter annotates it with an abstract state. The development is syntax-directed and intraprocedural. Its interpreter is parametric in a domain of abstract values: the domain supplies abstract operations and inverse operations with their soundness laws, and the interpreter derives forward evaluation, the backward filtering of guards, and the step of each command generically [6, Sects. 13.5, 13.7]. Voblint's non-relational domains follow this pattern (Section 5.6). Voblint keeps the separation between collecting semantics and abstraction. Its collecting semantics ranges over activation traces of a procedure-aware control-flow graph (Chapter 4), because a return must know which caller resumes.

Michelland et al. [95], Keidel et al. [96], and Darais et al. [97] make soundness proofs reusable by sharing one interpreter between the concrete and the abstract semantics, through monadic handlers, an arrow-based interface, or monad transformers. Voblint instead splits analysis, context policy, and solver for a constraint-based analyzer and composes them in one theorem, `activation_collect_dg_sound`, instantiated for every shipped analysis configuration.

D. Cachera and D. Pichardie [38] program and prove in Coq an abstract interpreter for a While language without procedures, proved correct through an intermediate collecting semantics and extracted to OCaml [38, §§1, 6]. Voblint uses the same stepping stone, a collecting semantics between execution and abstraction, but over activation traces of a graph and with a separately verified solver instead of a syntax-directed iterator. Franceschino et al. [41] verify a syntax-directed interpreter in F* with SMT automation, and their analyzer runs in a browser, so an executable or interactive verified analyzer is not new with this thesis.

Interprocedural analyses have been verified in Isabelle before, but not with a context-indexed collecting semantics like Voblint's. P. Lammich and M. Müller-Olm [40] prove soundness and, for every program, precision of a conflict analysis for programs with recursive procedures, threads, and monitors, interpreting calls and returns exactly. D. Wasserrab [98] proves an interprocedural slicer correct over an abstract control-flow graph with matched calls and returns. J. Breitner [99] proves Shivers' control-flow analysis correct in Isabelle/HOLCF, parametric in its contours through a type class, which plays the role a context policy plays here. None of these entries claims an extracted analyzer, and none of their semantics represents one activation with its caller chain, the object Voblint's context relation reads. Code generated from an Isabelle proof has, however, replaced an unverified compiler component: Buchwald et al. [100] replace the SSA construction of CompCertSSA by OCaml code extracted from an instantiation of their locale-based construction.

15.3. Verified fixpoint solvers

A verified solver guarantees a property of its result for arbitrary equations, and the analyzer must give the equations their meaning. Voblint reuses the solver of Tilscher et al. [4] and supplies that meaning.

The bytecode verifier of Klein and Nipkow [101, §3] is an early Isabelle instance of a verified solver consumed through a specification. A function is a bytecode verifier if it reports no error exactly when a stable, error-free method type, a post-fixpoint of the flow functions, exists above the initial state. Kildall's algorithm is proved to be one for every semilattice with the ascending chain condition and monotone, bounded flow functions [101, Thm. 1]. Their system is finite and intraprocedural, and the specification also demands completeness. Voblint's solver explores a context-indexed system with side effects on demand, and Voblint uses only the soundness direction of its certificate.

Hofmann et al. [62] verify the local generic solver RLD in Coq; like the top-down solver, it records dependencies between unknowns while it evaluates right-hand sides. De Vilhena et al. [102] verify in Iris a local generic solver that discovers dependencies dynamically and computes the least fixpoint of a monotone system. Stade et al. [3] prove the top-down solver partially correct in Isabelle/HOL, first for a simpler recursive formulation and then for the optimized solver, which agrees with it on termination and results. Tilscher et al. [4] extend the line to mixed flow-sensitive analyses. They verify $\mathrm{TD}_{\mathrm{side}}$, which accepts contributions to global unknowns during iteration, against a generic interface for update rules, prove five update rules of Stemmler et al. [44] sound, and refine the solver to executable code. For precise updates without widening and narrowing, they also prove that the solver returns the least partial post-solution, provided it terminates and the equation system satisfies their monotonicity conditions. Tilscher et al. [76] treat the termination of the solver without side effects; Section 9.6 explains why these results do not apply to Voblint's equations.

Voblint includes a copy of the solver of [4]. The algorithm, the update rules, and their proofs belong to that work. The copy carries local refactorings, such as renamings and an interface theory (Section 14.3). Voblint gives the equations their meaning: the certificate `part_post_solution` implies coverage of the concrete traces, and compiler correctness transfers the coverage to source executions. The solver Voblint runs warrows every local unknown at a widening point, regardless of which of the five selectable update rules merges

the global contributions. The least-solution theorem therefore does not apply, and Voblint uses only partial correctness, for all five rules (Section 9.4), and claims no optimality.

15.4. Context sensitivity and trace partitioning

Calling contexts have been defined by call history, by procedure summaries, and as an index of traces. Voblint's contexts index a collecting semantics, as trace partitioning does, and one activation may belong to several of them.

M. Sharir and A. Pnueli [56] introduce the two classical approaches to interprocedural analysis. The functional approach establishes input-output relations for procedures and treats calls as super operations, and the call-string approach tags propagated information with an encoded history of the calls encountered, so that a return propagates only information whose history matches [56, §1]. For recursive programs, both may fail to yield an effective solution for problems such as constant propagation [56, §1]. J. Knoop and B. Steffen [103] extend the coincidence theorem to recursive procedures with local variables. Their return function combines the information before the call with the callee's exit information [103, §4.1]. Voblint's return step has the same shape, combining the caller's pre-call state with the callee's result (Section 6.1). Reps et al. [104] solve interprocedural problems with finite fact sets and distributive transfer functions precisely, as reachability over interprocedurally valid paths, and state that semantic correctness is an orthogonal issue [104, §2]. Voblint needs neither finite nor distributive domains and proves no precision result.

Trace partitioning abstracts a set of traces by indexing it with control history before abstracting states [105]. Rival and Mauborgne define both partitions and coverings, in which one trace may belong to several indices, and present calling-context sensitivity as one such indexing. Keeping the full stack amounts to inlining and works only for nonrecursive calls. They also note that the function mapping tokens to tokens in a covering may be replaced by a relation [105, Rem. 3.2.4]. Voblint's context-indexed collecting semantics is a covering of this relational kind, since a context policy may admit one activation in several contexts. Voblint mechanizes the indexing over activation traces and proves that the solved, routed result bounds the activation collecting semantics of every admitted context.

Contexts also determine how many unknowns a solve creates. The context lifters of Erhard et al. [72] bound the number of contexts on the fly and guarantee finitely many contexts whenever the solver updates each unknown finitely often [72, Abstract]. Voblint's entry-state policy has no such bound, which is one reason the termination of the solve is not proved (Section 9.6).

15.5. Goblint, local traces, and thread-modular analysis

Voblint follows Goblint's architecture and adapts the local-trace semantics of Schwarz et al. This section names what it adopts and where it departs.

15.5.1. Architecture

Side-effecting constraint systems let one equation both define a local unknown and publish contributions to global unknowns [42], [43]. Seidl et al. [2] describe how Goblint uses them

to decouple a mixed flow-sensitive analysis from the solver, with digests on the analysis side and update rules on the solver side recovering precision. Voblint adopts the split: an analysis supplies local and global transfer behavior, and the generator and solver organize their interaction.

The correspondence with Goblint's implementation is architectural. We compared Voblint with the framework interface of Goblint's constraint generator at revision `5320a6b7` by source inspection. There, a call runs the analysis's `enter`, selects the callee context from the entered state, publishes the callee entry, and combines the callee's exit with the caller. Voblint's equations have the same four steps, with enter and combine from the D/G specification and the context chosen by the routing policy. The project site's side-by-side comparison (site) lists 19 Goblint constructs: 3 are modeled, 12 simplified, and 4 not modeled. No row claims that the two compute the same fixpoint.

Three simplifications affect how results transfer to Goblint. The callee entry is published to a global seed and read back by the entry's local unknown, where Goblint writes the local entry directly; under the warrowing rules the seed itself can be widened, so widening is placed differently, in a direction we have not proved. Call targets are resolved statically. The analyses share one combined state and one query kind without globals, where Goblint's MCP also passes events, spawns, and per-analysis globals. No agreement rate between the verdicts of the two analyzers is reported: the corpus has not been run through Goblint, and the adapted fixtures record Goblint's annotations only as prose.

15.5.2. Combining analyses

Goblint combines its analyses at run time in its MCP (Section 7.3), which also answers a query cycle with the top element, caches answers per transfer, and raises `Deadcode` when one analysis finds a point unreachable (`mCP.ml` at revision `5320a6b7`). Voblint's combination follows this design for one query kind and for analyses without globals, and adds a proof obligation per analysis that quantifies over every sound query channel. The reduced product [16, §5.1.2], [46] combines domains through a reduction on the product carrier, as the Int domain of Section 11.3 does. The combined state keeps separate carriers and exchanges facts only through answers, so no reduction operator has to be defined or proved for a pair of analyses.

Verasco, whose channels Voblint's follow (Section 7.2), threads them so that the second domain can query the state the first has just computed [33, §7]. Voblint's answers describe the predecessor state only, as Goblint's do. Voblint combines any list of components on the fields of one state and proves the combination sound for calls, returns, and context-sensitive equations.

15.5.3. Local traces and contexts

Local traces give each thread of a concurrent program a semantics against which thread-modular analyses are proved sound and compared [5], later extended to relational analyses [34], with detailed proofs in [35, Ch. 6]. The digest framework uses abstractions of execution histories to decide which observations may interact [106]. Voblint adapts the local view to sequential procedure activations: a trace covers one activation and records its suspended caller, so a return reads its caller from the trace instead of choosing one. A calling context becomes a projection of the trace instead of a component of the state. Context policies can

therefore be proved against one fixed concrete semantics. Activations do not interfere, so no concurrency result of that work transfers.

P. Sotin and B. Jeannet [107] also replace a stack semantics by a local one, in which every instruction acts on the top activation record only. They prove the two semantics equivalent with respect to reachable stacks [107, §3] and derive a relational interprocedural analysis from the local semantics. Their motive is pointers into the stack, which VIMP lacks. Voblint proves for activation traces only that every graph run is represented by a valid trace (Section 4.2.2).

A digest is a total function on local traces that splits the unknowns $[u]$ into $[u, A]$ already in the concrete semantics [35, §2.3]. The local-trace semantics itself has no procedures, and Schwarz lists procedures among the features that Goblint implements but the local-trace semantics does not yet support [35, §8, p. 277]. In Voblint, the context relation `activation_context_rel` (Section 4.3) plays a similar role for sequential activations, with two differences. A digest is a function, so every trace has exactly one digest and the traces split into disjoint groups [35, §2.3, Def. 4, Thm. 4]. One activation may carry several contexts instead, so Voblint's groups may overlap, and together they still contain every collected store (`node_collect_eq_Union_activation_collect`). A digest is also updated at every step of a trace, whereas an activation's contexts are fixed when it is called. Voblint therefore distinguishes activations only by how they were entered.

15.6. Where Voblint sits

Two mechanized analyzers come closest to Voblint's combination of mixed flow sensitivity and calling contexts. Cachera et al. [37] prove in Coq that a constraint-based analysis of Carmel, an intermediate representation of Java Card bytecode, is sound with respect to a small-step operational semantics. It keeps one flow-insensitive heap and flow-sensitive local variables and operand stacks per program point, is context-insensitive, and solves ordinary inequations with a verified round-robin solver extracted to OCaml [37, §§2, 3.2]. F. Dabrowski and D. Pichardie [39] prove in Coq that a context-sensitive points-to analysis with a flow-insensitive heap, a component of their certified data race analyzer, is sound. They instrument the concrete semantics with contexts through a Coq functor over a module type of contexts, whose call-context function computes the context of a callee from the call site, the caller's context, and the receiver, and they instantiate it, among others, with k-object sensitivity. The authors state that the specification is not executable: the analyses are specified only as sets of constraints, and no solver computes them [39, §7].

The concrete semantics of Dabrowski and Pichardie, which carries contexts and is parameterized by a context policy, is the closest precedent for Voblint's. Voblint's differs in three respects. Its contexts are admitted by a relation, the totality condition `call_context_total_on` makes the context-indexed collection exhaustive (`node_collect_eq_Union_activation_collect`), and the indexing is connected to an executable solver.

Voblint thus takes its trust pattern from CompCert and Verasco, its domain interface from Nipkow and Klein, its solver from Tilscher et al., and its architecture from Goblint, and combines these with what we did not find mechanized elsewhere: a context-indexed collecting semantics over activation traces, covered by equations that a verified side-effecting solver computes.

16. Conclusion

The thesis shows that the soundness of a Goblint-style analyzer with contexts, side effects, and a verified solver can be machine-checked from source executions to the verdicts of the exported function. The result holds for a scalar language with recursive procedures and is a partial-correctness result, so it covers every answer the analyzer returns. The proof follows one chain. Graph runs simulate source executions, and valid activation traces represent graph runs. Each activation carries the contexts its policy admits, and the activation collecting semantics groups the traces by these contexts, so one trace may lie in the sets of several contexts. Context-indexed equations with routed calls cover this semantics at every context, provided the analysis and the context policy meet their separate obligations. The solver enters only through a post-solution certificate, and the source-level theorem reads the verdicts off its result.

The central contribution is this composition. A context-free activation semantics, relational context routing, analysis-local soundness obligations and a verified side-effecting solver together yield one guarantee from source executions to verdicts. The proved function is the analyzer that the command-line tool and the browser playground run (Chapter 12).

16.1. Discussion

The design states its choices as obligations, and each choice buys generality at a known price. Contexts admitted by a relation let entry-state routing read contexts off the analysis's result. The price is totality (TOTAL), which functional policies meet directly (`context_policy_of_fun`). Analysis globals in the analysis contract let the flow-insensitive placement give each program global its own unknown. The price is a frame obligation and lost precision on the `set`/`get` program (Section 6.4). Consuming the solver only through its post-solution certificate makes soundness independent of the update rule (Section 9.4). In exchange, nothing is said about termination or about which post-solution the solver returns (Section 9.3). Exporting one dispatcher makes the constant of the theorem the one the tools run (Section 12.1).

How far the results carry beyond VIMP depends on the layer. Goblint analyzes C after CIL normalization, with pointers, a heap, threads, machine integers, and further update rules. The certificate is stated over right-hand sides and unknowns and mentions no VIMP construct. The coverage contract has one obligation per rule of `valid_activation_trace`. It refers to VIMP only through the graph, its stores, and three step functions, `edge_step`, `call_enter`, and `combine_collect` (Section 4.4). We therefore expect three parts to carry over to a language whose activations do not interfere: the shape of the coverage contract, the theorem that the contexts exhaust the node collecting semantics, and the composition of Section 8.3. The step functions and every proof that unfolds them would have to be redone.

Machine integers fit this case. They change the step functions, the generic derivation in `sound_nonrelational_ops`, and each domain's soundness proofs for its primitives. Routing and the solver certificate do not mention arithmetic. For signed overflow, a domain could keep its ideal-integer bounds and report an overflow when they exceed the type, as Goblint does [31, §3.3.2]. Unsigned wrap-around would need domain variants, since classical numeric domains describe ideal integers [108]. A C-like treatment of division by zero changes more. The semantics would need an error outcome, and the verdicts of Section 10.5 would need a meaning for it.

Pointers into the stack break the assumption that activations do not interfere. A return keeps the caller's locals (Section 6.1), and this fails once a callee can write them through a pointer. P. Sotin and B. Jeannet [107] introduce their local semantics for this case. A heap adds a store component that caller and callee share, so RETURN, the entry pairs, and the one value per variable in the domains (Section 3.1) would all change. Threads interleave activations, which `valid_activation_trace` cannot express. The local traces of Schwarz et al. [5] give threads a semantics, but that semantics has no procedures [35, §8, p. 277], and the interface would also need synchronization, which it lacks. A new update rule that meets the vendored interface requires comparatively little change, since only `update_rule_update_global_of` splits on the rule. A C front end such as CIL would join the parser in the trust boundary unless it is verified.

16.2. Limitations

The first limits concern what the theorem covers. It is a partial-correctness result and covers every answer the analyzer returns. Termination is not proved for the solve or for the reduction of Int, and regression programs exist whose solves do not finish (Section 9.6, Section 11.3). The theorem concerns VIMP, a scalar language without pointers, heap, or memory model. VIMP has unbounded integers, defined division by zero, and zero-initialized callee locals, so a verdict about a VIMP program does not transfer to a C program with the same text (Section 3.3). We argue, but do not prove, that `pstep` models the intended language. The simulation from source runs to graph runs and the representation of graph runs by valid traces are proved in the forward direction only (Section 3.6, Section 4.2.2). Within the analyzer, only the combined state as a whole publishes to analysis globals. `rel_order_spec` is therefore selectable only in its local form, and one update rule serves every global unknown of a run (Section 6.4, Section 11.4, Section 9.4).

The delivered tools rest on the trusted components of Section 12.5, and every theorem proved by evaluation trusts the code generator. No test runs VIMP programs concretely, since the development has no executable form of `pstep`. No time or memory measurement is reported. The executable keeps proof-oriented data representations, and no data refinement to efficient sets and maps has been carried out (Section 12.2). A refinement of the solver's state would require the proof of the vendored solver to be redone. The rest of the chain uses the solver only through its certificate (Section 9.3) and would be unaffected.

No general precision, optimality, or completeness theorem is proved. Completeness would mean that every check that holds in all executions is reported as proved. Each precision example concerns one program with fixed analysis settings (Section 13.3). The regression corpus is small and was written or adapted for this work, and its expected verdicts were set by hand, not checked against an independent analyzer (Section 13.4). The Goblint defect of

Section 13.2 was not re-run. The correspondence with Goblint is architectural. No theorem transfers to its OCaml implementation, and no agreement rate between the two analyzers is measured (Section 15.5). The comparison with prior work rests on a targeted search rather than a systematic review (Chapter 15).

16.3. Outlook

16.3.1. Extending Voblint

A richer source language, up to a subset of C, needs the new obligations that Section 16.1 names for machine integers, pointers, a heap, and threads. It also needs a semantics and a preservation argument per construct at every layer.

Such a language would extend the elaboration of Section 3.6. For VIMP, elaboration only records where the compiler places each statement, and erasure relates the elaborated runs to the runs of the program as written. A front end for C would use the same annotated syntax to resolve names, types, and expressions and to make conversions explicit, with erasure again connecting the result to the source.

The adequacy of `pstep` is argued and checked against example programs (Section 3.3). A fuel-bounded evaluator whose runs are proved to be `pstep` runs would make this check systematic. A run that violates a check reported as `PROVED` would then point to a fault in the trusted base. Comparing the evaluator's runs with a C compiler on the common subset would test the adequacy argument itself.

Richer analyses need their own soundness proofs. A relational domain with stronger transfer functions than the order analysis could also answer the numeric analyses at branches (Section 11.4). Further query kinds would each need a truth relation, and analyses with their own globals would need the combination to keep these globals apart, as Goblint's MCP does (Section 7.4).

Every shipped analysis answers a single entry pair per call, although the interface admits several (Section 6.1). Path-sensitive analyses that split a call into cases therefore remain future work. The contexts of an activation depend only on how the activation was entered. Digests refine unknowns by other abstractions of a local trace (Section 15.5). Generalizing `activation_context_rel` to such abstractions of activation traces would allow path- or history-sensitive unknowns, each with its own admissibility conditions.

Voblint adapts its equations to the interface of the vendored solver (Section 9.1). Goblint's solvers publish to local unknowns and join the writes of one evaluation per target. A version of the vendored solver that does the same would let callers publish into callee entries directly and make activation seeds and buffering unnecessary. Its partial-correctness proof would have to be redone for local side effects.

A termination theorem would guarantee an answer for every program. The total-correctness result of Tilscher et al. covers the top-down solver without side effects. It assumes finitely many unknowns, a precise widening, and monotonic right-hand sides with monotonic dependencies [76, Cor. 1]. Their widening stabilizes every widening sequence by definition [76, Def. 2]. The vendored class `warrowing` states no stabilization law (Section 2.5), so even finitely many unknowns may take values that increase forever (Section 9.6).

16.3.2. Agent-assisted development

Each new analysis only has to meet its own obligations, which mention neither the other analyses nor contexts, the solver, or the equations (Section 7.4). This suits development with AI agents (see the statement on the use of generative AI). Formal verification and agents complement each other here. An agent may propose definitions and proofs, but Isabelle checks every proof, and the batch build and the drift checks of Chapter 14 reject anything that does not check. Counterexample theorems for weakened obligations (Section 13.2, Section 16.1) act as negative tests. Human review can therefore concentrate on what no proof settles: whether the base definitions and the obligations state the intended meaning (Section 3.3), and whether the trusted base of Section 12.5 is acceptable.

The same division applies beyond single analyses. Extending the framework with a construct from C, or with a Goblint feature such as path-sensitive entry alternatives or further query kinds, changes definitions at several layers. Isabelle then reports every proof the change breaks, at whichever layer it sits, so an agent can work through the failures one by one while a reviewer checks the new definitions, above all the extended source semantics. Goblint supplies both the design to follow and test material: its regression programs can be adapted as fixtures, as part of the corpus already is (Section 13.4), and its documented defects can be replayed as counterexample theorems (Section 13.2). We expect this to make larger extensions practical.

16.3.3. Voblint and Goblint

Within the architectural correspondence of Section 15.5, Voblint can serve as an executable specification of the architecture with proved soundness obligations. In one direction, a Goblint feature gives the design that a Voblint extension follows, as the query mechanism did for Chapter 7. A documented Goblint defect can be replayed as a regression fixture and as a counterexample theorem, as the congruence remainder was (Section 13.2). The same obligations could be consulted before a change to Goblint lands. A proposed domain operation with a counterpart in Voblint would be transliterated by hand and checked against the obligation that its counterpart meets, as `congruence_mod_sound` states it for the remainder. Operations on pointers or threads have no such counterpart.

In the other direction, a verified component exported as OCaml could replace its unverified counterpart in Goblint. That needs more than the export. It needs the data refinement of Section 16.2, a correspondence between the component's interface and Goblint's, and soundness for C's integers, since the component's proofs assume VIMP's unbounded ones (Section 3.3).

Voblint verifies neither Goblint nor an analyzer for C. It establishes a smaller result. For a Goblint-style architecture with recursive procedures, routed contexts, cooperating analyses, and a generic side-effecting solver, one machine-checked chain connects source executions to the verdicts of executable code. Its remaining gaps are a richer language, termination, efficient representations, and the handwritten code around the analyzer. The same proof structure can be extended to each of them.

Bibliography

[1] V. Vojdani, K. Apinis, V. Rõtov, H. Seidl, V. Vene, and R. Vogler, "Static Race Detection for Device Drivers: The Goblint Approach," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, ACM, 2016, pp. 391–402. doi: 10.1145/2970276.2970337.

[2] H. Seidl, V. Vojdani, J. Erhard, and M. Schwarz, "Mixed Flow-Sensitive Static Analysis: Engineering Modularity," in *FM 2026: Formal Methods*, 2026, pp. 446–470. doi: 10.1007/978-3-032-26220-2_22.

[3] Y. Stade, S. Tilscher, and H. Seidl, "The Top-Down Solver Verified: Building Confidence in Static Analyzers," in *Computer Aided Verification*, 2024, pp. 303–324. doi: 10.1007/978-3-031-65627-9_15.

[4] S. Tilscher, A. Graß, and H. Seidl, "Verifying a Solver for Mixed Flow-Sensitive Analyses," in *NASA Formal Methods (NFM 2026)*, in LNCS. Springer, 2026, pp. 48–69. doi: 10.1007/978-3-032-28079-4_3.

[5] M. Schwarz, S. Saan, H. Seidl, K. Apinis, J. Erhard, and V. Vojdani, "Improving Thread-Modular Abstract Interpretation," in *Static Analysis*, in LNCS, vol. 12913. 2021, pp. 359–383. doi: 10.1007/978-3-030-88806-0_18.

[6] T. Nipkow and G. Klein, *Concrete Semantics with Isabelle/HOL*. Springer, 2014. doi: 10.1007/978-3-319-10542-0.

[7] ISO/IEC JTC 1/SC 22/WG 14, "ISO/IEC 9899:2011, Information technology – Programming languages – C." Accessed: Sep. 21, 2026. [Online]. Available: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[8] Leiden Declaration Working Group, "Leiden Declaration on Artificial Intelligence and Mathematics." [Online]. Available: https://leidendeclaration.ai/

[9] AutoCorrode contributors, "AutoCorrode: Verification Infrastructure for the Isabelle/HOL Interactive Proof Assistant." Accessed: Sep. 21, 2026. [Online]. Available: https://github.com/awslabs/AutoCorrode

[10] K. Kappelmann, M. Schäffeler, L. Stevens, M. Abdulaziz, A. Popescu, and D. Traytel, "Just Type It in Isabelle! AI Agents Drafting, Mechanizing, and Generalizing from Human Hints." [Online]. Available: https://arxiv.org/abs/2604.15713v3

[11] D. Bryant, J. J. H. y Munive, C. Kaliszyk, and J. Urban, "Munkres' General Topology Autoformalized in Isabelle/HOL." Accessed: Sep. 22, 2026. [Online]. Available: https://arxiv.org/abs/2604.07455v1

[12] M. Schwarz, "Improve Precision for %." Accessed: Sep. 22, 2026. [Online]. Available: https://github.com/goblint/analyzer/issues/1156

[13] M. Schwarz, "Congruences: Make % Sound by Restricting Cases Where We Return a Constant." Accessed: Sep. 21, 2026. [Online]. Available: https://github.com/goblint/analyzer/pull/1161

[14] L. C. Paulson, T. Nipkow, and M. Wenzel, “From LCF to Isabelle/HOL,” *Formal Aspects of Computing*, vol. 31, no. 6, pp. 675–698, 2019, doi: 10.1007/s00165-019-00492-1.

[15] G. Klein *et al.*, “seL4: Formal Verification of an OS Kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009)*, ACM, 2009, pp. 207–220. doi: 10.1145/1629575.1629596.

[16] X. Rival and K. Yi, *Introduction to Static Analysis: An Abstract Interpretation Perspective*. MIT Press, 2020. [Online]. Available: https://mitpress.mit.edu/9780262043410/introduction-to-static-analysis/

[17] H. G. Rice, “Classes of Recursively Enumerable Sets and Their Decision Problems,” *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953, doi: 10.1090/S0002-9947-1953-0053041-6.

[18] B. Livshits *et al.*, “In Defense of Soundiness: A Manifesto,” *Communications of the ACM*, vol. 58, no. 2, pp. 44–46, 2015, doi: 10.1145/2644805.

[19] P. Cousot *et al.*, “Combination of Abstractions in the ASTRÉE Static Analyzer,” in *Advances in Computer Science – ASIAN 2006. Secure Software and Related Issues*, in LNCS, vol. 4435. Springer, 2007, pp. 272–300. doi: 10.1007/978-3-540-77505-8_23.

[20] D. Bühler, P. Cuoq, and B. Yakobowski, “The Eva Plug-in, for Frama-C 33.0 (Arsenic).” Accessed: Sep. 29, 2026. [Online]. Available: https://www.frama-c.com/download/frama-c-eva-manual.pdf

[21] M. Journault, A. Miné, R. Monat, and A. Ouadjaout, “Combinations of Reusable Abstract Domains for a Multilingual Static Analyzer,” in *Verified Software. Theories, Tools, and Experiments (VSTTE 2019)*, in LNCS. Springer, 2020, pp. 1–18. doi: 10.1007/978-3-030-41600-3_1.

[22] G. Brat, J. A. Navas, N. Shi, and A. Venet, “IKOS: A Framework for Static Analysis Based on Abstract Interpretation,” in *Software Engineering and Formal Methods (SEFM 2014)*, in LNCS. Springer, 2014, pp. 271–277. doi: 10.1007/978-3-319-10431-7_20.

[23] Q. L. Le, A. Raad, J. Villard, J. Berdine, D. Dreyer, and P. W. O'Hearn, “Finding Real Bugs in Big Programs with Incorrectness Logic,” *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, p. 81:1–81:27, 2022, doi: 10.1145/3527325.

[24] S. Saan *et al.*, “Goblint: Autotuning Thread-Modular Abstract Interpretation (Competition Contribution),” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2023)*, in LNCS, vol. 13994. Springer, 2023, pp. 547–552. doi: 10.1007/978-3-031-30820-8_34.

[25] B. Blanchet *et al.*, “A Static Analyzer for Large Safety-Critical Software,” in *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2003, pp. 196–207. doi: 10.1145/781131.781153.

[26] Mopsa developers, “Mopsa User Manual.” Accessed: Sep. 29, 2026. [Online]. Available: https://mopsa.gitlab.io/mopsa-analyzer/user-manual/

[27] IKOS developers, “IKOS Analyzer README.” Accessed: Sep. 29, 2026. [Online]. Available: https://github.com/NASA-SW-VnV/ikos/blob/ac7f7c1738976cabc58c6a53413df6e458995c38/analyzer/README.md

[28] P. Cuoq *et al.*, "Testing Static Analyzers with Randomly Generated Programs," in *NASA Formal Methods (NFM 2012)*, in LNCS. Springer, 2012, pp. 120–125. doi: 10.1007/978-3-642-28891-3_12.

[29] C. Klinger, M. Christakis, and V. Wüstholz, "Differentially Testing Soundness and Precision of Program Analyzers," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2019, pp. 239–250. doi: 10.1145/3293882.3330553.

[30] D. Kaindlstorfer, A. Isychev, V. Wüstholz, and M. Christakis, "Interrogation Testing of Program Analyzers for Soundness and Precision Issues," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, ACM, 2024, pp. 319–330. doi: 10.1145/3691620.3695034.

[31] S. Saan, "Correctness Witnesses for Thread-Modular Program Analysis," Doctoral dissertation, 2026. [Online]. Available: https://dspace.ut.ee/server/api/core/bitstreams/00ec3799-947c-48eb-99e3-f43c9646bbcb/content

[32] X. Leroy, "Formal Verification of a Realistic Compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009, doi: 10.1145/1538788.1538814.

[33] J.-H. Jourdan, V. Laporte, S. Blazy, X. Leroy, and D. Pichardie, "A Formally-Verified C Static Analyzer," in *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2015, pp. 247–259. doi: 10.1145/2676726.2676966.

[34] M. Schwarz, S. Saan, H. Seidl, J. Erhard, and V. Vojdani, "Clustered Relational Thread-Modular Abstract Interpretation with Local Traces," in *Programming Languages and Systems (ESOP 2023)*, in LNCS, vol. 13990. 2023, pp. 28–58. doi: 10.1007/978-3-031-30044-8_2.

[35] M. B. Schwarz, "Thread-Modular Abstract Interpretation: The Local Perspective," Doctoral dissertation, Technische Universität München, 2025. [Online]. Available: https://mediatum.ub.tum.de/doc/1765098/n8cop1l4luxwzq7hmzpv376a6.tmai-local-perspective-schwarz.pdf

[36] S. Blazy, V. Laporte, A. Maroneze, and D. Pichardie, "Formal Verification of a C Value Analysis Based on Abstract Interpretation," in *Static Analysis*, 2013, pp. 324–344. doi: 10.1007/978-3-642-38856-9_18.

[37] D. Cachera, T. Jensen, D. Pichardie, and V. Rusu, "Extracting a Data Flow Analyser in Constructive Logic," *Theoretical Computer Science*, vol. 342, no. 1, pp. 56–78, 2005, doi: 10.1016/j.tcs.2005.06.004.

[38] D. Cachera and D. Pichardie, "A Certified Denotational Abstract Interpreter," in *Interactive Theorem Proving (ITP 2010)*, in LNCS. Springer, 2010, pp. 9–24. doi: 10.1007/978-3-642-14052-5_3.

[39] F. Dabrowski and D. Pichardie, "A Certified Data Race Analysis for a Java-like Language," in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, in LNCS. 2009, pp. 212–227. doi: 10.1007/978-3-642-03359-9_16.

[40] P. Lammich and M. Müller-Olm, "Formalization of Conflict Analysis of Programs with Procedures, Thread Creation, and Monitors," *Archive of Formal Proofs*, Dec. 2007, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Program-Conflict-Analysis.html

[41] L. Franceschino, D. Pichardie, and J.-P. Talpin, "Verified Functional Programming of an Abstract Interpreter," in *Static Analysis (SAS 2021)*, in LNCS. Springer, 2021, pp. 124–143. doi: 10.1007/978-3-030-88806-0_6.

[42] H. Seidl, V. Vene, and M. Müller-Olm, "Global Invariants for Analysing Multi-threaded Applications," *Proceedings of the Estonian Academy of Sciences. Physics. Mathematics*, vol. 52, no. 4, pp. 413–436, 2003, doi: 10.3176/phys.math.2003.4.05.

[43] K. Apinis, H. Seidl, and V. Vojdani, "Side-Effecting Constraint Systems: A Swiss Army Knife for Program Analysis," in *Programming Languages and Systems (APLAS 2012)*, in LNCS, vol. 7705. Springer, 2012, pp. 157–172. doi: 10.1007/978-3-642-35182-2_12.

[44] F. Stemmler, M. Schwarz, J. Erhard, S. Tilscher, and H. Seidl, "Taking Out the Toxic Trash: Recovering Precision in Mixed Flow-Sensitive Static Analyses," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 1167–1193, 2025, doi: 10.1145/3729297.

[45] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1977, pp. 238–252. doi: 10.1145/512950.512973.

[46] P. Cousot and R. Cousot, "Systematic Design of Program Analysis Frameworks," in *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1979, pp. 269–282. doi: 10.1145/567752.567778.

[47] F. Haftmann and T. Nipkow, "Code Generation via Higher-Order Rewrite Systems," in *Functional and Logic Programming (FLOPS 2010)*, in LNCS, vol. 6009. Springer, 2010, pp. 103–117. doi: 10.1007/978-3-642-12251-4_9.

[48] M. Lerchner, "Voblint: A Verified Goblint-Style Static Analysis Pipeline in Isabelle/HOL." Zenodo, 2026. doi: 10.5281/zenodo.23260696.

[49] A. Miné, "Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation," *Foundations and Trends in Programming Languages*, vol. 4, no. 3–4, pp. 120–372, 2017, doi: 10.1561/2500000034.

[50] A. Tarski, "A Lattice-Theoretical Fixpoint Theorem and Its Applications," *Pacific Journal of Mathematics*, vol. 5, no. 2, pp. 285–309, 1955, doi: 10.2140/pjm.1955.5.285.

[51] H. Seidl, R. Wilhelm, and S. Hack, *Compiler Design: Analysis and Transformation*. Berlin, Heidelberg: Springer, 2012. doi: 10.1007/978-3-642-17548-0.

[52] P. Cousot and R. Cousot, "Comparing the Galois Connection and Widening/Narrowing Approaches to Abstract Interpretation," in *Programming Language Implementation and Logic Programming (PLILP 1992)*, in LNCS, vol. 631. Springer, 1992, pp. 269–295. doi: 10.1007/3-540-55844-6_142.

[53] T. Nipkow, "Abstract Interpretation of Annotated Commands," in *Interactive Theorem Proving (ITP 2012)*, in LNCS. Springer, 2012, pp. 116–132. doi: 10.1007/978-3-642-32347-8_9.

[54] G. A. Kildall, "A Unified Approach to Global Program Optimization," in *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1973, pp. 194–206. doi: 10.1145/512927.512945.

[55] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[56] M. Sharir and A. Pnueli, "Two Approaches to Interprocedural Data Flow Analysis," in *Program Flow Analysis: Theory and Applications*, S. S. Muchnick and N. D. Jones, Eds., Prentice-Hall, 1981, ch. 7.

[57] K. Apinis, H. Seidl, and V. Vojdani, "How to Combine Widening and Narrowing for Non-Monotonic Systems of Equations," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2013, pp. 377–386. doi: 10.1145/2491956.2462190.

[58] K. Apinis, H. Seidl, and V. Vojdani, "Enhancing Top-Down Solving with Widening and Narrowing," in *Semantics, Logics, and Calculi*, vol. 9560, in LNCS, vol. 9560., Springer, 2016, pp. 272–288. doi: 10.1007/978-3-319-27810-0_14.

[59] A. Graß, "Towards the Verification of Top-Down Solvers," Master's thesis, Technische Universität München, 2024. Accessed: Sep. 22, 2026. [Online]. Available: https://www.tcs.ifi.lmu.de/staff/alexandra-grass/master-thesis-alexandra-grass.pdf

[60] H. Seidl and R. Vogler, "Three improvements to the top-down solver," *Mathematical Structures in Computer Science*, vol. 31, no. 9, pp. 1090–1134, 2021, doi: 10.1017/S0960129521000499.

[61] F. Bourdoncle, "Efficient Chaotic Iteration Strategies with Widenings," in *Formal Methods in Programming and Their Applications*, in LNCS, vol. 735. Springer, 1993, pp. 128–141. doi: 10.1007/BFb0039704.

[62] M. Hofmann, A. Karbyshev, and H. Seidl, "Verifying a Local Generic Solver in Coq," in *Static Analysis (SAS 2010)*, in LNCS. Springer, 2010, pp. 340–355. doi: 10.1007/978-3-642-15769-1_21.

[63] F. Haftmann and M. Wenzel, "Constructive Type Classes in Isabelle," in *Types for Proofs and Programs (TYPES 2006)*, in LNCS. Springer, 2007, pp. 160–174. doi: 10.1007/978-3-540-74464-1_11.

[64] C. Ballarin, "Locales: A Module System for Mathematical Theories," *Journal of Automated Reasoning*, vol. 52, no. 2, pp. 123–153, 2014, doi: 10.1007/s10817-013-9284-7.

[65] B. Huffman and O. Kunčar, "Lifting and Transfer: A Modular Design for Quotients in Isabelle/HOL," in *Certified Programs and Proofs (CPP 2013)*, in LNCS. Springer, 2013, pp. 131–146. doi: 10.1007/978-3-319-03545-1_9.

[66] L. C. Paulson, "Broken Proofs and Broken Provers." Accessed: Sep. 22, 2026. [Online]. Available: https://lawrencecpaulson.github.io/2026/01/15/Broken_proofs.html

[67] P. Lammich and S. Wimmer, "IMP2 – Simple Program Verification in Isabelle/HOL," *Archive of Formal Proofs*, Jan. 2019, [Online]. Available: https://isa-afp.org/entries/IMP2.html

[68] N. Schirmer, "A Sequential Imperative Programming Language Syntax, Semantics, Hoare Logics and Verification Environment," *Archive of Formal Proofs*, Feb. 2008, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Simpl.html

[69] G. D. Plotkin, "A Structural Approach to Operational Semantics," *Journal of Logic and Algebraic Programming*, vol. 60–61, pp. 17–139, 2004, doi: 10.1016/j.jlap.2004.05.001.

[70] G. C. Necula, S. McPeak, S. P. Rahul, and W. Weimer, "CIL: Intermediate Language and Tools for Analysis and Transformation of C Programs," in *Compiler*

Construction (CC 2002), in LNCS, vol. 2304. Springer, 2002, pp. 213–228. doi: 10.1007/3-540-45937-5_16.

[71] M. Brecknell *et al.*, "AutoCorres2," *Archive of Formal Proofs*, Apr. 2024, Accessed: Oct. 04, 2026. [Online]. Available: https://isa-afp.org/entries/AutoCorres2.html

[72] J. Erhard, J. F. Schinabeck, M. Schwarz, and H. Seidl, "Context Gas and friends: taming context-sensitivity on the fly," *International Journal on Software Tools for Technology Transfer*, vol. 27, no. 2, pp. 289–307, 2025, doi: 10.1007/s10009-025-00803-3.

[73] P. Cousot, "The Calculational Design of a Generic Abstract Interpreter," in *Calculational System Design*, vol. 173, M. Broy and R. Steinbrüggen, Eds., in NATO ASI Series F, vol. 173., IOS Press, 1999, pp. 421–505.

[74] K. Apinis, "Frameworks for Analyzing Multi-threaded C," Doctoral dissertation, 2014. [Online]. Available: https://nbn-resolving.org/urn:nbn:de:bvb:91-diss-20140606-1189191-0-9

[75] S. Schulze Frielinghaus, H. Seidl, and R. Vogler, "Enforcing Termination of Interprocedural Analysis," *Formal Methods in System Design*, vol. 53, no. 2, pp. 313–338, 2018, doi: 10.1007/s10703-017-0288-5.

[76] S. Tilscher, A. Graß, H. Seidl, and Y. Stade, "Proving Total Correctness of Top-Down Solvers with Widening and Narrowing," *Journal of Automated Reasoning*, vol. 70, no. 2, p. 15, 2026, doi: 10.1007/s10817-026-09756-x.

[77] P. Granger, "Static Analysis of Arithmetical Congruences," *International Journal of Computer Mathematics*, vol. 30, no. 3–4, pp. 165–190, 1989, doi: 10.1080/00207168908803778.

[78] J. Kim, "Mechanizing Typed Regulatory Actions for Security Tokens: Semantics, Falsification, and Bounded EVM Evidence." [Online]. Available: https://arxiv.org/abs/2608.29134v3

[79] D. Marmsoler, "Isabelle/STARK: A Formalization of zk-STARK in Isabelle/HOL." [Online]. Available: https://arxiv.org/abs/2608.01965v1

[80] L. C. Paulson, "Why not just use Lean?." Accessed: Sep. 27, 2026. [Online]. Available: https://lawrencecpaulson.github.io/2026/04/23/Why_not_Lean.html

[81] halbGefressen, "Comment on 'Why not just use Lean?'." Accessed: Sep. 27, 2026. [Online]. Available: https://github.com/lawrencecpaulson/lawrencecpaulson.github.io/issues/62#issuecomment-4333878708

[82] A. Chlipala, *Certified Programming with Dependent Types*. MIT Press, 2013. [Online]. Available: http://adam.chlipala.net/cpdt/

[83] M. Lerchner, "isar-tools: Source Tooling for Isabelle/Isar Projects." Accessed: Sep. 27, 2026. [Online]. Available: https://github.com/ManuelLerchner/isar-tools

[84] Y. Megdiche and others, "Isabelle Linter." Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/isabelle-prover/isabelle-linter

[85] A. Salamon, "isabelle-query: Querying Isabelle/Isar Projects." Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/ott2/isabelle-query

[86] M. Lerchner, "Fix Isabelle2025-2 compatibility after phase-state refactor." Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/15

[87] M. Lerchner, "refactor: separate operator laws from termination assumptions." Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/13

[88] M. Lerchner, "Rename short constructors and record fields that leak into downstream theories." Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/14

[89] M. Lerchner, "Add solver facts downstream users had to prove themselves." Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/16

[90] J.-H. Jourdan, "Verasco: a Formally Verified C Static Analyzer," Doctoral dissertation, 2016. [Online]. Available: https://theses.hal.science/tel-01327023

[91] D. Beyer, M. Dangl, D. Dietsch, and M. Heizmann, "Correctness Witnesses: Exchanging Verification Results between Verifiers," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, ACM, 2016, pp. 326–337. doi: 10.1145/2950290.2950351.

[92] S. Saan, M. Schwarz, J. Erhard, H. Seidl, S. Tilscher, and V. Vojdani, "Correctness Witness Validation by Abstract Interpretation," in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2024)*, in LNCS, vol. 14499. Springer, 2024, pp. 74–97. doi: 10.1007/978-3-031-50524-9_4.

[93] CompCert developers, "CompCert Module Kildall: Solvers for Dataflow Inequations." Accessed: Sep. 22, 2026. [Online]. Available: https://compcert.org/doc/html/compcert.backend.Kildall.html

[94] R. L. Spina, D. Demange, and S. Blazy, "Formal Verification of WTO-based Dataflow Solvers," in *Programming Languages and Systems (ESOP 2025)*, in LNCS. Springer, 2025, pp. 91–118. doi: 10.1007/978-3-031-91121-7_4.

[95] S. Michelland, Y. Zakowski, and L. Gonnord, "Abstract Interpreters: A Monadic Approach to Modular Verification," *Proceedings of the ACM on Programming Languages*, vol. 8, no. ICFP, 2024, doi: 10.1145/3674646.

[96] S. Keidel, C. B. Poulsen, and S. Erdweg, "Compositional Soundness Proofs of Abstract Interpreters," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, 2018, doi: 10.1145/3236767.

[97] D. Darais, M. Might, and D. V. Horn, "Galois Transformers and Modular Abstract Interpreters: Reusable Metatheory for Program Analysis," in *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2015)*, ACM, 2015, pp. 552–571. doi: 10.1145/2814270.2814308.

[98] D. Wasserrab, "Backing up Slicing: Verifying the Interprocedural Two-Phase Horwitz-Reps-Binkley Slicer," *Archive of Formal Proofs*, Nov. 2009, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/HRB-Slicing.html

[99] J. Breitner, "Shivers' Control Flow Analysis," *Archive of Formal Proofs*, Nov. 2010, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Shivers-CFA.html

[100] S. Buchwald, D. Lohner, and S. Ullrich, "Verified Construction of Static Single Assignment Form," in *Proceedings of the 25th International Conference on Compiler Construction (CC 2016)*, ACM, 2016, pp. 67–76. doi: 10.1145/2892208.2892211.

[101] G. Klein and T. Nipkow, "Verified bytecode verifiers," *Theoretical Computer Science*, vol. 298, no. 3, pp. 583–626, 2003, doi: 10.1016/S0304-3975(02)00869-1.

[102] P. E. de Vilhena, F. Pottier, and J.-H. Jourdan, "Spy Game: Verifying a Local Generic Solver in Iris," *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–28, 2020, doi: 10.1145/3371101.

[103] J. Knoop and B. Steffen, "The Interprocedural Coincidence Theorem," in *Compiler Construction (CC '92)*, in LNCS, vol. 641. Springer, 1992, pp. 125–140. doi: 10.1007/3-540-55984-1_13.

[104] T. Reps, S. Horwitz, and M. Sagiv, "Precise Interprocedural Dataflow Analysis via Graph Reachability," in *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 1995, pp. 49–61. doi: 10.1145/199448.199462.

[105] X. Rival and L. Mauborgne, "The Trace Partitioning Abstract Domain," *ACM Transactions on Programming Languages and Systems*, vol. 29, no. 5, 2007, doi: 10.1145/1275497.1275501.

[106] M. Schwarz and J. Erhard, "The digest framework: concurrency-sensitivity for abstract interpretation," *International Journal on Software Tools for Technology Transfer*, vol. 26, no. 6, pp. 727–746, 2024, doi: 10.1007/s10009-024-00773-y.

[107] P. Sotin and B. Jeannet, "Precise Interprocedural Analysis in the Presence of Pointers to the Stack," in *Programming Languages and Systems (ESOP 2011)*, in LNCS. Springer, 2011, pp. 459–479. doi: 10.1007/978-3-642-19718-5_24.

[108] A. Miné, "Abstract Domains for Bit-Level Machine Integer and Floating-point Operations," in *Joint Proceedings of the Workshops on Automated Theory eXploration and on Invariant Generation (ATx'12/WInG'12)*, in EPiC Series in Computing, vol. 17. EasyChair, 2013, pp. 55–70. doi: 10.29007/b63g.